

# Projekt-Dokumentation: ClioGraph

---

- **Git-Basis-Version:** v1-260729-2115
- **Dokumentations-Datei:** ClioGraph\_Stand\_v1-260729-2115.md
- **Export-Zeitstempel:** 29.07.2026 21:15:23

## 1. Aktuelle Ordnerstruktur

```
ClioGraph/
├── .gitignore
├── LICENSE
├── README.md
├── business/
│   ├── __init__.py
│   ├── biz.py
│   ├── commands/
│   │   ├── __init__.py
│   │   ├── cmd_api.py
│   │   ├── cmd_base.py
│   │   ├── cmd_community.py
│   │   ├── cmd_database.py
│   │   ├── cmd_dummy.py
│   │   ├── cmd_file.py
│   │   ├── cmd_help.py
│   │   ├── cmd_llm.py
│   │   ├── cmd_ml.py
│   │   ├── cmd_register.py
│   │   ├── cmd_source.py
│   │   ├── cmd_system.py
│   │   └── cmd_visual.py
│   ├── commands_user/
│   │   ├── __init__.py
│   │   └── cmd_hello_world.py
│   └── models.py
├── cliograph_system.log
├── database/
│   ├── __init__.py
│   └── db.py
├── gui/
│   ├── __init__.py
│   ├── designer.py
│   ├── factory.py
│   ├── ide_designer.py
│   ├── sidebar.py
│   ├── tabs.py
│   └── window.py
├── infrastructure/
│   ├── __init__.py
│   ├── cfg.py
│   └── errors.py
```

```
| | | exceptions.py
| | | i18n.py
| | | languages/
| | | | | __init__.py
| | | | | translator.pdf
| | | | | translator.py
| | | llm_ml/
| | | | | __init__.py
| | | | | llm.py
| | | menu_storage.py
| | | ui_state_manager.py
| | json_storage/
| | | backup_locales/
| | | | | .gitkeep
| | | dynamic_menu.json
| | | locales/
| | | | | de.json
| | | | | de_factory.json
| | | | | en.json
| | | | | en_factory.json
| | | | | start_status.json
| | logs/
| | | cliograph.log
| | main.py
| | requirements.txt
| | temp
| | tools/
| | | clioGraph_locales_review.txt
| | | convert_to_pdf.py
| | | json_syntax_fixer.py
| | | locale_cleaner.py
| | | pdf_konverter.reg
| | | system_actions.py
| | | test_ollama.py
```

## 2. Quellcode- und Konfigurationsdateien

File: **.gitignore**

```
# --- Python Systemdateien ---
__pycache__/
*.pyc
*.pyo
*.pyd
.Python
env/
venv/
.venv/

# --- IDE & Editor Einstellungen ---
.vscode/
```

```
.idea/
*.suo
*.ntvs*
*.njsproj
*.sln

# --- ClioGraph Framework-Ausschluss (KORRIGIERT!) ---
# Wir erlauben die Basis-Menüdatei auf GitHub, ignorieren aber lokale Sperr-Reste
json_storage/dynamic_menu.json.bak
json_storage/dynamic_menu.json.tmp

# WICHTIG: Kern-Sprachdateien, Werkshüllen UND start_status.json MÜSSEN auf GitHub
landen!
json_storage/locales/*.json
!json_storage/locales/de.json
!json_storage/locales/en.json
!json_storage/locales/de_factory.json
!json_storage/locales/en_factory.json
!json_storage/locales/start_status.json

# Struktur des Backup-Korbs erlauben, aber den temporären Inhalt ignorieren
json_storage/backup_locales/*
!json_storage/backup_locales/.gitkeep

# Wir ignorieren die temporären Makro-Dateien der Entwickler-Tests
business/commands_user/cmd_tab_*.py
!business/commands_user/__init__.py
!business/commands_user/cmd_hello_world.py

# --- Betriebssystem-Reste ---
.DS_Store
Thumbs.db
```

File: **main.py**

```
# Datei: main.py
"""Bootstrapper: Zentraler Einstiegspunkt für die ClioGraph-Anwendung"""

import sys
import os
import json
from PyQt6.QtWidgets import QApplication

# Import der Infrastruktur- und Hilfsdienste
from infrastructure.i18n import TranslationService
from infrastructure.errors import ErrorHandlingService
from infrastructure.menu_storage import MenuPersistenceManager
import infrastructure.cfg as cfg

# Import der Core-Schichten (DAL & KI-Infrastruktur)
```

```

from database.db import GraphRepository
from infrastructure.llm_ml.llm import LlmExtractionService
from business.biz import BusinessCoordinator

from gui import MainWindow

# from infrastructure.languages.translater import DynamicLanguageManager
from infrastructure.languages.translator import DynamicLanguageManager

def main():
    # 1. System-Applikation initialisieren
    app = QApplication(sys.argv)

    # 2. Unabhängige Basis-Infrastruktur starten
    i18n_service =
TranslationService(default_lang=cfg.APP_CONFIG["default_language"])
    error_service = ErrorHandlingService(i18n_service)

    # 3. Core-Schichten initialisieren (DAL & KI-Infrastruktur)
    db_repo = GraphRepository(db_path=cfg.DB_PATH)
    llm_service = LlmExtractionService(model_name=cfg.APP_CONFIG["default_model"])

    # 4. Central Service Locator Pool aufbauen & Persistenz laden
    persistence_manager =
MenuPersistenceManager(discover_func=cfg.discover_all_commands)

    # ÜBERSPRÜGE DEN LÖSCH-MODUS UND LADE DIE PERSISTENTEN MENÜS:
    if os.path.exists(cfg.JSON_MENU_FILE):
        try:
            # Lade die gespeicherten Menüs von der Festplatte
            with open(cfg.JSON_MENU_FILE, "r", encoding="utf-8") as f:
                saved_data = json.load(f)

                if "menu_structure" in saved_data:
                    # Aktualisiere das globale Laufzeit-Schema mit den geladenen
Daten
                    cfg.UI_SCHEMA["menu_structure"] = saved_data["menu_structure"]

                    # === REPARATUR DER PERSISTENZ: Reihenfolge beim Starten
mitladen ===
                    if "menu_order" in saved_data:
                        cfg.UI_SCHEMA["menu_order"] = saved_data["menu_order"]
                    else:
                        # Fallback: Falls die Liste in der JSON fehlt, aus den
Keys generieren
                        cfg.UI_SCHEMA["menu_order"] =
list(saved_data["menu_structure"].keys())

                    # === KERN-REPARATUR: Register/Tabs werden nun direkt beim
Booten geladen! ===
                    if "tab_structure" in saved_data:
                        cfg.UI_SCHEMA["tab_structure"] =
saved_data["tab_structure"]

```

```

# === NEU: REAKTIVIERUNG DER GESPEICHERTEN SPRACHEN ===
# 1. Aktivierte Sprachen zurück in den RAM-Pool holen
if "supported_languages" in saved_data:
    for lang_code in saved_data["supported_languages"]:
        if lang_code not in
cfg.APP_CONFIG["supported_languages"]:

cfg.APP_CONFIG["supported_languages"].append(lang_code)

# 2. Die Anzeigeliste für die GUI-Dropdowns absolut
zerstörungsfrei wiederherstellen
if "available_languages" in saved_data:
    for lang in saved_data["available_languages"]:
        # Füge die Sprache NUR hinzu, wenn sie nicht bereits
im System existiert
        if not any(l["code"] == lang["code"] for l in
cfg.AVAILABLE_LANGUAGES):
            cfg.AVAILABLE_LANGUAGES.append(lang)

# === REPARATUR DES SCHÖNHEITSFEHLERS: START-STATUS
GEGENPRÜFEN ===
start_status_pfad = os.path.join(cfg.JSON_LOCALES_DIR,
"start_status.json")
if os.path.exists(start_status_pfad):
    with open(start_status_pfad, "r", encoding="utf-8") as sf:
        sd = json.load(sf)
        if "active_languages" in sd:
            # Bekannte ISO-Mapping-Tabelle für DH-
Forschungssprachen
            iso_namen = {
                "fr": "Français", "it": "Italiano", "es":
"Español",
                "ar": "Arabic", "he": "Hebrew", "fa":
"Persian",
                "la": "Latin", "el": "Greek", "nl": "Dutch"
            }
            for code in sd["active_languages"]:
                # Im RAM-Pool registrieren
                if code not in
cfg.APP_CONFIG["supported_languages"]:
                    cfg.APP_CONFIG["supported_languages"].append(code)
                    # Im Dropdown-Pool registrieren
                    if not any(l["code"] == code for l in
cfg.AVAILABLE_LANGUAGES):
                        display_name = iso_namen.get(code,
code.upper())
                        cfg.AVAILABLE_LANGUAGES.append({"name":
display_name, "code": code})

print("✓ [SYSTEM] Dynamische Menüstruktur und Sprachen
erfolgreich geladen und synchronisiert.")
except Exception as e:

```

```

        print(f"❌ [SYSTEM] Fehler beim Laden der Menüdatei: {e}")
    else:
        # Falls die Datei noch nie existierte (erster Start), erstelle das
        Ausgangsgerüst
        print("🚧 [SYSTEM] Keine Menüdatei gefunden. Generiere Basis-Schema...")
        persistence_manager.save_menus(cfg.UI_SCHEMA)

# 5. Service Locator Pool mit Objekten befüllen
# Den Manager erschaffen
language_manager = DynamicLanguageManager(services=None)

# Das gesamte Services-Paket schnüren
services = {
    "i18n": i18n_service,
    "errors": error_service,
    "biz": BusinessCoordinator(repository=db_repo, llm_service=llm_service),
    "command_finder": cfg.discover_all_commands,
    "menu_persistence": persistence_manager,
    "language_manager": language_manager # <-- Hier legst du ihn hinein
}

# Dem Manager das fertige Paket zur Verfügung stellen
language_manager._services = services

# 6. Presentation Layer (GUI) aufbauen, Service-Pool und geladenes Schema
injizieren
main_window = MainWindow(services, cfg.UI_SCHEMA)

# 7. Sprache initial setzen und Fenster auf dem Bildschirm anzeigen
"""main_window._change_language_trigger(cfg.APP_CONFIG["default_language"])"""
main_window._change_language_trigger("de")
main_window.show()

# === NEU: REFRESH NACH DEM START FÜR DIE SIDEBAR-LISTEN ===
main_window.setup_ui()

# 8. Event-Loop der Anwendung starten
sys.exit(app.exec())

if __name__ == "__main__":
    main()

```

File: `business\__init__.py`

File: `business\biz.py`

```
# Datei: business/biz.py
"""Business Logic Layer (BLL): Koordination der Arbeitsabläufe für ClioGraph"""
from database.db import GraphRepository
from business.models import GraphDataWrapper
from infrastructure.exceptions import GraphValidationError

class BusinessCoordinator:
    """Der zentrale Dirigent der Geschäftslogik (Fassade/Coordinator Pattern)."""

    def __init__(self, repository: GraphRepository, llm_service):
        self._repo = repository
        self._llm = llm_service

    def process_input_data(self, text: str, source_ref: str) -> GraphDataWrapper:
        """Steuert die Kette aus LLM-Extraktion und Datenbank-Speicherung.

        Prüft die Integrität der extrahierten Daten für maximale Graph-RAG-
        Tauglichkeit.
        """
        # 1. LLM zur Extraktion aufrufen
        raw_llm_list = self._llm.extract_entities_and_relations(text)

        # 2. Aus den LLM-Daten ein valides Graph-Modell bauen
        try:
            # Erzeugt das Datenpaket (Knoten und Kanten)
            graph_data = GraphDataWrapper.from_llm_list(raw_llm_list, source_ref)
        except Exception as e:
            raise GraphValidationError(f"Die Struktur der extrahierten Daten ist
fehlerhaft: {str(e)}")

        # 3. Validierung: Für ein stabiles Graph-RAG dürfen keine verwaisten
        # Kanten existieren
        # (Dieser Check wird aktiv, sobald wir das Datenmodell im nächsten Schritt
        # reparieren)
        # Open Item: Gegen das in cfg.py definierte Schema (NODE_LABELS /
        # EDGE_TYPES)
        # validieren, bevor die Daten an die DAL übergeben werden.

        # 4. Daten an die DAL zur persistenten Speicherung übergeben
        self._repo.insert_graph_data(graph_data)

        return graph_data
```

File: `business\models.py`

```
# Datei: models.py
"""Domain Layer: Typsichere Datenmodelle und Konvertierungslogiken für Graph-
```

```

RAG"""
from dataclasses import dataclass, field
from typing import List, Dict, Any

@dataclass
class NodeModel:
    """Repräsentiert einen eindeutigen Knoten (Entität) im Wissensgraphen."""
    node_id: str
    label: str # Z.B. "Person", "Ereignis", "Ort"
    properties: Dict[str, Any] = field(default_factory=dict)

@dataclass
class EdgeModel:
    """Repräsentiert eine gerichtete Beziehung (Kante) zwischen zwei Knoten."""
    source_id: str
    target_id: str
    edge_type: str # Z.B. "GEBOREN_IN", "PARTNER_VON"
    properties: Dict[str, Any] = field(default_factory=dict)

@dataclass
class GraphDataWrapper:
    """Container für ein extrahiertes Teilnetzwerk (Teilgraphen)."""
    nodes: List[NodeModel] = field(default_factory=list)
    edges: List[EdgeModel] = field(default_factory=list)

    @classmethod
    def from_llm_list(cls, raw_data_list: List[Dict[str, Any]], source_ref: str) -
> 'GraphDataWrapper':
        """Konvertiert rohe LLM-Listen in ein valides, Graph-RAG-taugliches
        Datenmodell.

        Verhindert 'unknown_source'-Verweise durch korrekte ID-Verknüpfung.
        """
        wrapper = cls()

        # Falls die Liste leer ist (z.B. beim aktuellen Testdurchlauf), leeres
        Paket zurückgeben
        if not raw_data_list:
            return wrapper

        # ID-Mapping, um Kanten logisch und präzise mit den generierten Knoten zu
        verknüpfen
        for item in raw_data_list:
            # Beispielhafte Verarbeitungslogik für strukturierte LLM-Objekte
            # (Wird aktiv, sobald Ollama echtes JSON liefert)
            pass

        return wrapper

```

File: business\commands\\_\_init\_\_.py



```
# Datei: business/commands/__init__.py
# Zentrale Bereitstellung aller Menübefehle für den Service Locator

from .cmd_file import ProjectExportCommand, OllamaInfoCommand,
OllamaValidateCommand, LanguageEditorCommand
from .cmd_system import ResetSystemCommand

# Mapping für die dynamische Befehlsinstanziierung zur Laufzeit
COMMAND_MAPPING = {
    "menu.file.export": ProjectExportCommand,
    "menu.file.ollama": OllamaInfoCommand,
    "menu.file.validate": OllamaValidateCommand,
    "menu.file.edit_locales": LanguageEditorCommand,

    # Variante 1: Registrierung über die ID
    "menu.file.reset_system": ResetSystemCommand,

    # Variante 2: Registrierung über den Klassennamen (ERWEITERT FÜR DEINE BEIDEN
    # BUTTONS)
    "ResetSystemCommand": ResetSystemCommand,
    "OllamaValidateCommand": OllamaValidateCommand,
    "LanguageEditorCommand": LanguageEditorCommand
}
```

File: `business\commands\cmd_api.py`

File: `business\commands\cmd_base.py`

```
# Datei: business/commands/cmd_base.py
from abc import ABC, abstractmethod

class UICommand(ABC):
    """
    Erweiterte abstrakte Basisklasse für alle entkoppelten Menübefehle.
    Erzwingt Typsicherheit für die dynamische Menügenerierung.
    """

    @property
    @abstractmethod
    def menu_key(self) -> str:
        """
        Gibt den exakten I18N-Konfigurationsschlüssel des Befehls zurück.
        Muss von jeder Kind-Klasse überschrieben werden (z.B. 'menu.file.export').
        """
        pass
```

```
@abstractmethod
def execute(self, services: dict):
    """Führt den spezifischen Business-Befehl aus."""
    pass
```

File: `business\commands\cmd_community.py`

File: `business\commands\cmd_database.py`

File: `business\commands\cmd_dummy.py`

```
# Datei: business/commands/cmd_dummy.py
from business.commands.cmd_base import UICommand

class DynamischesTestFeatureCommand(UICommand):
    """Ein einfaches Test-Makro für unsere Plus-Schaltfläche."""

    @property
    def menu_key(self) -> str:
        return "dummy.test_feature"

    def execute(self, services: dict):
        print("\n" + " ⚡ "*15)
        print("[ERFOLG] Das dynamisch erstellte Feature wurde aufgerufen!")
        print(" ⚡ "*15 + "\n")
        return "Dynamisches Feature erfolgreich ausgeführt!"
```

File: `business\commands\cmd_file.py`

```
# Datei: business/commands/cmd_file.py
from abc import abstractmethod

from business.commands.cmd_base import UICommand
from PyQt6.QtCore import Qt
import infrastructure.cfg as cfg
import os
import json
```

```

class ProjectExportCommand(UICommand):
    """Kapselt die Fachlogik für den Export des aktuellen DH-Projekts."""
    @property
    def menu_key(self) -> str:
        return "menu.file.export"

    def execute(self, services: dict):
        print("[Datei-Container] Starte den Projektexport...")

class OllamaInfoCommand(UICommand):
    """Befehl zur Anzeige der lokalen Ollama-Offline-KI Hilfsinformationen."""

    @property
    def menu_key(self) -> str:
        return "menu.file.ollama"

    def execute(self, services: dict):
        print("🚀 [DEBUG] OllamaInfoCommand.execute wurde ERFOLGREICH
aufgerufen!")
        from PyQt6.QtWidgets import QMessageBox

        # Ein unzerbrechlicher Informationstext direkt im Quellcode
        titel = "Ollama-Dienst (KI-Infrastruktur)"

        text = (
            "<h3>Lokale Offline-KI für ClioGraph</h3>"
            "Dieses System nutzt Ollama, um historische Daten vollkommen "
            "lokal und datenschutzkonform auf Ihrem Rechner zu verarbeiten.<br>
<br>"
            "<b>Schritte zur Einrichtung:</b><br>"
            "1. Laden Sie Ollama für Ihr Betriebssystem herunter.<br>"
            "2. Installieren und starten Sie die Anwendung im Hintergrund.<br>
<br>"
            "<b>Download-Link:</b><br>"
            "<a href='https://ollama.com'>https://ollama.com</a>"
        )

        # Die Box wird völlig frei im Vordergrund geschaltet
        msg = QMessageBox()
        msg.setIcon(QMessageBox.Icon.Information)
        msg.setWindowTitle(titel)

        # JETZT KORREKT: Holt sich das Flag sicher aus dem Qt-Kernmodul
        from PyQt6.QtCore import Qt
        msg.setTextInteractionFlags(msg.textInteractionFlags() |
Qt.TextInteractionFlag.LinksAccessibleByMouse)

        msg.setText(text)
        msg.setStandardButtons(QMessageBox.StandardButton.Ok)
        msg.exec()

class OllamaValidateCommand(UICommand):
    """Zentraler Sprachen-Assistent mit hierarchischem Dropdown (EU & Intl) und

```

```

Hybridmodus."""
def __init__(self, main_win=None, i18n_service=None):
    # Das =None verhindert jetzt auch hier den Absturz beim Scannen!
    super().__init__()

    # Falls die GUI den Service nicht übergibt, holen wir das Singleton:
    if i18n_service is None:
        from infrastructure.i18n import TranslationService
        self.i18n = TranslationService()
    else:
        self.i18n = i18n_service

@property
def menu_key(self) -> str:
    return "menu.file.validate"

def execute(self, services: dict):
    i18n = services.get("i18n")
    lang_mgr = services.get("language_manager")
    if not i18n or not lang_mgr: return

    from PyQt6.QtWidgets import QMessageBox, QApplication, QDialog,
QVBoxLayout, QLabel, QCheckBox, QPushButton, QHBoxLayout, QComboBox
    from PyQt6.QtCore import QLocale
    from PyQt6.QtGui import QStandardItemModel, QIcon
    import json
    import os

    # 1. ERST DAS MAINWINDOW SUCHEN (Damit der Dialog ein stabiles Fundament
hat)
    main_win = None
    for widget in QApplication.topLevelWidgets():
        if widget.inherits("QMainWindow") or widget.__class__.__name__ ==
"MainWindow":
            main_win = widget
            break

    # 2. DIALOGFENSTER ERSTELLEN UND DAS MAINWINDOW ALS PARENT ÜBERGEBEN
    dialog = QDialog(main_win)
    dialog.setWindowTitle(i18n.text("dialog.assistant.title"))
    dialog.setMinimumWidth(480)

    # FIX: Das geerbte Kleeblatt-Icon für diesen Dialog entfernen
    dialog.setWindowIcon(QIcon())

    # HIER WIRD 'layout' DEFINIERT:
    layout = QVBoxLayout()
    dialog.setLayout(layout)

    lbl_info = QLabel(i18n.text("dialog.assistant.info"))
    lbl_info.setWordWrap(True)
    layout.addWidget(lbl_info)
    layout.addWidget(QLabel(" "))

```

```

# === UNZERBRECHLICHE EINLESEN AUS DEM START-STATUS ===
gespeicherte_aktive_sprachen = ["de", "en"]
start_status_pfad = os.path.join(cfg.JSON_LOCALES_DIR,
"start_status.json")

if os.path.exists(start_status_pfad):
    try:
        with open(start_status_pfad, "r", encoding="utf-8") as sf:
            sd = json.load(sf)
            if "active_languages" in sd:
                gespeicherte_aktive_sprachen = sd["active_languages"]
            for code in gespeicherte_aktive_sprachen:
                if code not in cfg.APP_CONFIG["supported_languages"]:
                    cfg.APP_CONFIG["supported_languages"].append(code)
    except Exception as e:
        print(f" Fehler beim Lesen des Start-Status: {e}")

# 2. DIALOGFENSTER ERSTELLEN UND DAS MAINWINDOW ALS PARENT ÜBERGEBEN
# GRUPPE A: Europäische Hauptsprachen (Häkchen mit Live-Existenzprüfung)
layout.addWidget(QLabel(f"<b>{i18n.text('dialog.assistant.group.eu')}
</b>"))
eu_sprachen = {
    "English (Standard)": ("en", "English"),
    "Français (CH)": ("fr", "French"),
    "Italiano (CH)": ("it", "Italian"),
    "Español": ("es", "Spanish")
}

checkbox_mapping = {}
for label_text, (code, name_en) in eu_sprachen.items():
    file_path = os.path.join(cfg.JSON_LOCALES_DIR, f"{code}.json")
    datei_existiert = os.path.exists(file_path)

    # Absolute Wahrheit aus unserer dedizierten Status-Datei
    ist_aktiv = (code in gespeicherte_aktive_sprachen)

    if code in cfg.MASTER_LANGUAGES:
        final_label = f"{label_text} ● (Systemkern)"
    elif datei_existiert and not ist_aktiv:
        final_label = f"{label_text} ● (Im Archiv vorhanden)"
    elif datei_existiert and ist_aktiv:
        final_label = f"{label_text} ● (Bereits aktiv)"
    else:
        final_label = f"{label_text} ● (Bereit zum Anlegen)"

    cb = QCheckBox(final_label)

    if code == "en" or ist_aktiv:
        cb.setChecked(True)
    else:
        cb.setChecked(False)

```

```

        layout.addWidget(cb)
        checkbox_mapping[cb] = (code, name_en)

    layout.addWidget(QLabel(" "))

    # GRUPPE B: Systemweiter ISO-Standard für das Dropdown
    layout.addWidget(QLabel(f"<b>{i18n.text('dialog.assistant.group.intl')}
</b>"))
    combo_intl = QComboBox()
    combo_intl.addItem(i18n.text("dialog.assistant.dropdown.none"), None)

    any_country = getattr(QLocale.Country, "AnyCountry", getattr(QLocale,
    "AnyCountry", None))
    if any_country is None:
        any_country = QLocale.Country(0) if hasattr(QLocale, "Country") else 0

    eu_amtssprachen = [
        "bg", "cs", "da", "et", "fi", "el", "ga", "hr",
        "lv", "lt", "mt", "nl", "pl", "pt", "ro", "sk", "sl", "sv", "la"
    ]
    intl_wissenschaftssprachen = ["ar", "he", "fa", "hi", "zh", "ja", "ko",
    "ru", "tr"]

    # Festplatte scannen, um archivierte Dateien für das Dropdown zu erkennen
    vorhandene_dateien = os.listdir(cfg.JSON_LOCALES_DIR) if
os.path.exists(cfg.JSON_LOCALES_DIR) else []
    archivierte_codes = [f.replace(".json", "") for f in vorhandene_dateien if
f.endswith(".json")]

    all_locales = QLocale.matchingLocales(QLocale.Language.AnyLanguage,
    QLocale.Script.AnyScript, any_country)
    added_iso_codes = set()

    eu_items = []
    intl_items = []

    for loc in all_locales:
        locale_name = loc.name()
        if not locale_name: continue

        parts = locale_name.split("_")
        iso_code = parts[0].lower()

        if len(iso_code) != 2 or iso_code in added_iso_codes:
            continue

        native_name = QLocale.languageToString(loc.language())

        # === DYNAMISCHE STATUS-ERMITTLUNG FÜR DIE DROPDOWN-ELEMENTE ===
        if iso_code in cfg.MASTER_LANGUAGES:
            status_str = "● Systemkern"
        elif iso_code in cfg.APP_CONFIG["supported_languages"]:
            status_str = "● Bereits aktiv"
        elif iso_code in archivierte_codes:

```

```

        status_str = "🟡 Im Archiv vorhanden"
    else:
        status_str = "🟢 Bereit zum Anlegen"

    display_str = f"{native_name} ({iso_code}) | {status_str}"
    data_tuple = (iso_code, native_name)

    if iso_code in eu_amtssprachen:
        eu_items.append((display_str, data_tuple))
        added_iso_codes.add(iso_code)
    elif iso_code in intl_wissenschaftssprachen:
        intl_items.append((display_str, data_tuple))
        added_iso_codes.add(iso_code)

eu_items.sort(key=lambda x: x[0])
intl_items.sort(key=lambda x: x[0])

model = combo_intl.model()
combo_intl.addItem("--- EU-AMTSPRACHEN ---", None)
if isinstance(model, QStandardItemModel):
    item = model.item(combo_intl.count() - 1)
    if item:
        font = item.font()
        font.setBold(True)
        item.setFont(font)
        item.setEnabled(False)

for display_str, data_tuple in eu_items:
    combo_intl.addItem(display_str, data_tuple)

# Sonderprüfung für Latein-Status
la_status = "🟢 Systemkern" if "la" in cfg.MASTER_LANGUAGES else ("🟢
Bereits aktiv" if "la" in cfg.APP_CONFIG["supported_languages"] else ("🟡 Im
Archiv vorhanden" if "la" in archivierte_codes else "🟢 Bereit zum Anlegen"))
combo_intl.addItem(f"Latin / Latein (la) | {la_status}", ("la",
"Latin"))
combo_intl.insertSeparator(combo_intl.count())

combo_intl.addItem("--- INTERNATIONALE SPRACHEN ---", None)
if isinstance(model, QStandardItemModel):
    item = model.item(combo_intl.count() - 1)
    if item:
        font = item.font()
        font.setBold(True)
        item.setFont(font)
        item.setEnabled(False)

for display_str, data_tuple in intl_items:
    combo_intl.addItem(display_str, data_tuple)

layout.addWidget(combo_intl)
layout.addWidget(QLabel(" "))

# 2. DIE AKTIONS-STEUERUNG (Buttons)

```

```

        btn_layout = QHBoxLayout()

        # Beenden (Abbruch-Weg)
        btn_cancel = QPushButton(i18n.text("dialog.assistant.btn.cancel"))
        btn_cancel.clicked.connect(dialog.reject)
        btn_layout.addWidget(btn_cancel)

        btn_layout.addStretch()

        # Das transaktionale OK-Muster zum Bestätigen der Sprachauswahl
        btn_ok = QPushButton(i18n.text("dialog.assistant.btn.ok"))
        btn_ok.setStyleSheet("background-color: #007bff; color: white; font-
weight: bold; padding: 6px 12px; border-radius: 4px;")
        btn_ok.clicked.connect(dialog.accept)
        btn_layout.addWidget(btn_ok)

        layout.addLayout(btn_layout)

        # Wenn der Forscher das Fenster schliesst oder Beenden klickt, brechen wir
ab
        if dialog.exec() != QDialog.DialogCode.Accepted:
            return

        self._action_mode = "manual"

        # 3. VERARBEITUNG DER ENTSCHEIDUNG
        main_win = None
        for widget in QApplication.topLevelWidgets():
            if widget.inherits("QMainWindow") or widget.__class__.__name__ ==
"MainWindow":
                main_win = widget
                break

        if main_win:
            main_win.setDisabled(True)

        ausgewaehlte_sprachen_log = []
        targets_to_process = []
        languages_to_deactivate = []

        for cb, (target_code, target_name_en) in checkbox_mapping.items():

... [HINWEIS: Datei wurde hier nach 300 Zeilen gekürzt] ...

```

File: `business\commands\cmd_help.py`



File: `business\commands\cmd_llm.py`

File: `business\commands\cmd_ml.py`

File: `business\commands\cmd_register.py`

File: `business\commands\cmd_source.py`

File: `business\commands\cmd_system.py`

```
# -*- coding: utf-8 -*-
"""Business Layer: System-Kommando zur Wiederherstellung der Werkseinstellungen"""

import os
import shutil
import json
from PyQt6.QtWidgets import QMessageBox
from business.commands.cmd_base import UICommand
import infrastructure.cfg as cfg

class ResetSystemCommand(UICommand):
    """Setzt das gesamte GUI-Framework und die Sprachdateien auf den Urzustand zurück."""

    @property
    def menu_key(self) -> str:
        return "menu.file.reset_system"

    def execute(self, services: dict):
        # LAZY IMPORT: Zirkuläre Importe und Side-Effects verhindern
        from PyQt6.QtCore import Qt
        from PyQt6.QtWidgets import QMessageBox, QApplication

        i18n = services.get("i18n")
        ui_state = services.get("ui_state_manager")
```

```

persistence = services.get("menu_persistence")

# Helfer für bilinguale Sonden
def get_text(key, fallback):
    return i18n.text(key) if i18n and hasattr(i18n, "text") else fallback

# Hauptfenster-Referenz für korrekte Zuweisung suchen
main_win = None
for widget in QApplication.topLevelWidgets():
    if widget.inherits("QMainWindow"):
        main_win = widget
        break

# Hilfsfunktion, um Nachrichten-Boxen ohne Kleeblatt-Icon zu erzeugen
def create_clean_msg_box(icon_type, title_text, body_text,
    buttons=QMessageBox.StandardButton.Ok):
    box = QMessageBox(main_win)
    box.setIcon(icon_type)
    box.setWindowTitle(title_text)
    box.setText(body_text)
    box.setStandardButtons(buttons)
    box.setWindowFlags(box.windowFlags() |
Qt.WindowType.CustomizeWindowHint)
    box.setWindowFlags(box.windowFlags() &
~Qt.WindowType.WindowSystemMenuHint)
    return box

# 1. Sicherheitsabfrage
title = get_text("dialog.reset.title", "System zurücksetzen")
msg = get_text("dialog.reset.text", "Möchten Sie das System wirklich auf
die Werkseinstellungen zurücksetzen?\nAlle dynamischen Menüs und hinzugefügten
Sprachen werden archiviert.")

ask_box = create_clean_msg_box(
    QMessageBox.Icon.Question,
    title,
    msg,
    QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No
)
ask_box.setDefaultButton(QMessageBox.StandardButton.No)

if ask_box.exec() != QMessageBox.StandardButton.Yes:
    return

try:
    # SONDE 1: Start
    box1 = create_clean_msg_box(QMessageBox.Icon.Information,
get_text("sonde.title.1", "Schritt 1/5"), get_text("sonde.msg.1", "Bereinigung
gestartet..."))
    box1.exec()

    # SONDE 2: Datei-Bereinigung
    box2 = create_clean_msg_box(QMessageBox.Icon.Information,
get_text("sonde.title.2", "Schritt 2/5"), get_text("sonde.msg.2", "Lösch

```

```

Konfigurationsdatei (dynamic_menu.json)..."))
    box2.exec()

# ABSOLUTE PFADSICHERUNG: Löscht die Datei direkt im json_storage-
Ordner
    target_menu_path = os.path.join(os.getcwd(), "json_storage",
"dynamic_menu.json")
    if os.path.exists(target_menu_path):
        try:
            os.remove(target_menu_path)
            print(f"[RESET] {target_menu_path} erfolgreich gelöscht.")
        except Exception as e:
            print(f"[RESET-FEHLER] Konnte dynamic_menu.json nicht löschen:
{e}")

# Bereinigung der generierten Benutzer-Makros (cmd_tab_*.py)
    user_cmd_dir = os.path.join(os.getcwd(), "business", "commands_user")
    if os.path.exists(user_cmd_dir):
        for filename in os.listdir(user_cmd_dir):
            if filename.startswith("cmd_tab_") and
filename.endswith(".py"):
                try:
                    os.remove(os.path.join(user_cmd_dir, filename))
                    print(f"[RESET] Benutzer-Makro gelöscht: {filename}")
                except Exception as e:
                    print(f"[RESET] Fehler beim Löschen von {filename}:
{e}")

# SONDE 3: Sprach-Archivierung
    box3 = create_clean_msg_box(QMessageBox.Icon.Information,
get_text("sonde.title.3", "Schritt 3/5"), get_text("sonde.msg.3", "Archiviere
zusätzliche Sprachdateien..."))
    box3.exec()

    locales_dir = cfg.JSON_LOCALES_DIR
    backup_dir = os.path.join(cfg.JSON_STORAGE_DIR, "backup_locales")
    os.makedirs(backup_dir, exist_ok=True)

    if os.path.exists(locales_dir):
        for filename in os.listdir(locales_dir):
            if filename.endswith(".json"):
                # KORREKTUR: Erweiterter Schutzschild für alle
Werksdateien

                protected_files = [
                    "de.json",
                    "en.json",
                    "de_factory.json",
                    "en_factory.json",
                    "start_status.json"
                ]

                if filename in protected_files:
                    continue

```

```

        src_path = os.path.join(locales_dir, filename)
        dst_path = os.path.join(backup_dir, filename)
        try:
            shutil.move(src_path, dst_path)
            print(f"[RESET] Fremdsprache archiviert: {filename}")
        except Exception as e:
            print(f"[RESET] Fehler beim Verschieben von
{filename}: {e}")

        # PRÄZISE POSITIONIERUNG: start_status.json am exakten Ort auf ("de",
"en") setzen
        status_path = os.path.join(cfg.JSON_LOCALES_DIR, "start_status.json")
        try:
            default_status = {"active_languages": ["de", "en"]}
            with open(status_path, "w", encoding="utf-8") as f:
                json.dump(default_status, f, ensure_ascii=False, indent=4)
            print("[RESET] start_status.json erfolgreich aktualisiert.")
        except Exception as e:
            print(f"[RESET] Fehler beim Schreiben der start_status.json: {e}")

        # OPTION A: Überschreibe die aktiven Sprachdateien radikal mit den
sauberen Werkshüllen
        for lang in ["de", "en"]:
            factory_path = os.path.join(cfg.JSON_LOCALES_DIR, f"
{lang}_factory.json")
            active_path = os.path.join(cfg.JSON_LOCALES_DIR, f"{lang}.json")
            if os.path.exists(factory_path):
                try:
                    shutil.copy2(factory_path, active_path)
                    print(f"[RESET] {lang}.json erfolgreich aus Werkshülle
wiederhergestellt.")
                except Exception as e:
                    print(f"[RESET-FEHLER] Konnte {lang}.json nicht
wiederherstellen: {e}")

        # SONDE 4: RAM-Reset (Vollständige Werkseinstellung inklusive
Sidebars)
        box4 = create_clean_msg_box(QMessageBox.Icon.Information,
get_text("sonde.title.4", "Schritt 4/5"), get_text("sonde.msg.4", "Setze
Systemkonfigurationen im RAM zurück..."))
        box4.exec()

        cfg.APP_CONFIG["supported_languages"] = ["de", "en"]
        cfg.AVAILABLE_LANGUAGES = [{"name": "Deutsch", "code": "de"}, {"name":
"English", "code": "en"}]

        # Vollständiges Werkseinstellungs-Schema mit allen Sidebars und dem
Plus-Tab
        cfg.UI_SCHEMA = {
            "menu_structure": {
                "file": {
                    "i18n_key": "menu.file",

```

```

        "items": [
            {"type": "action", "id": "menu.file.language",
"command_class": "ChangeLanguageCommand"},
            {"type": "action", "id": "menu.file.edit_locales",
"command_class": "LanguageEditorCommand"},
            {"type": "action", "id": "menu.file.ollama",
"command_class": "OllamaInfoCommand"},
            {"type": "action", "id": "menu.file.validate",
"command_class": "OllamaValidateCommand"},
            {"type": "action", "id": "menu.file.reset_system",
"command_class": "ResetSystemCommand"},
            {"type": "separator"},
            {"type": "action", "id": "menu.file.exit",
"command_class": "ExitCommand"}
        ]
    },
    "menu_order": ["file"],
    "tab_structure": [
        {
            "id": "tab.welcome",
            "i18n_key": "tabs.welcome.title",
            "label": "Willkommen",
            "elements": []
        }
    ]
}

if ui_state:
    if hasattr(ui_state, "_undo_stack"): ui_state._undo_stack.clear()
    if hasattr(ui_state, "_redo_stack"): ui_state._redo_stack.clear()

if persistence and hasattr(persistence, "save_menus"):
    payload = {
        "menu_structure": cfg.UI_SCHEMA["menu_structure"],
        "menu_order": cfg.UI_SCHEMA["menu_order"],
        "tab_structure": cfg.UI_SCHEMA["tab_structure"],
        "supported_languages": cfg.APP_CONFIG["supported_languages"],
        "available_languages": cfg.AVAILABLE_LANGUAGES
    }
    persistence.save_menus(payload)

# SONDE 5: UI-Neuaufbau
box5 = create_clean_msg_box(QMessageBox.Icon.Information,
get_text("sonde.title.5", "Schritt 5/5"), get_text("sonde.msg.5", "Initialisiere
Benutzeroberfläche neu..."))
box5.exec()

# Da sich die Widgets durch setup_ui ändern, suchen wir das Fenster
zur Sicherheit noch einmal
for widget in QApplication.topLevelWidgets():
    if widget.inherits("QMainWindow"):
        main_win = widget
        break

```

```

        if main_win and hasattr(main_win, "setup_ui"):
            main_win.setup_ui()

        # Abschlussmeldung
        box_done = create_clean_msg_box(QMessageBox.Icon.Information,
get_text("sonde.title.done", "Erfolg"), get_text("sonde.msg.done", "Das System
wurde erfolgreich in den Urzustand zurückgesetzt!"))
        box_done.exec()

    except Exception as e:
        box_err = create_clean_msg_box(QMessageBox.Icon.Critical, "Fehler beim
Reset", f"Absturz bei der Ausführung:\n{str(e)}")
        box_err.exec()

```

File: `business\commands\cmd_visual.py`

File: `business\commands_user\__init__.py`

File: `business\commands_user\cmd_hello_world.py`

```

# -*- coding: utf-8 -*-
from PyQt6.QtWidgets import QMessageBox

class HelloWorldCommand:
    """Ein einfaches Test-Makro zur Validierung der Befehls-Zuweisung."""

    def __init__(self, main_window):
        self.win = main_window

    def execute(self):
        """Wird aufgerufen, wenn der Forscher auf den zugewiesenen Menüpunkt
klickt."""
        # Ermittle die aktuelle GUI-Sprache für den bilingualen Test
        lang = getattr(self.win._i18n, "_current_lang", "de")

        title = "ClioGraph Test" if lang == "de" else "ClioGraph Test"
        msg = "Hallo Welt! Die Befehls-Zuordnung funktioniert einwandfrei!" if
lang == "de" else "Hello World! The command assignment works perfectly!"

```

```
QMessageBox.information(self.win, title, msg)
```

File: database\\_\_init\_\_.py

File: database\db.py

```
# Datei: database/db.py
"""Data Access Layer (DAL): Kapselung der Kùzu-Graphdatenbank"""
from infrastructure.exceptions import DatabaseConnectionError
import infrastructure.cfg as cfg

class GraphRepository:
    """Verwaltet den Zugriff auf die lokale Kùzu-Graphdatenbank."""

    def __init__(self, db_path: str = cfg.DB_PATH):
        self._db_path = db_path
        # Hier wird später die echte Kùzu-Verbindung instanziiert
        self._db = None
        self._session = None
        # Open Item: Schema-Erstellung (cfg.NODE_LABELS & cfg.EDGE_TYPES) bei der
        # Datenbank-Initialisierung dynamisch einlesen (geplant für Phase 2).

    def insert_graph_data(self, graph_data_wrapper) -> bool:
        """Speichert extrahierte Knoten und Kanten in der Datenbank.

        Fängt interne Kùzu-Ausnahmen ab und übersetzt sie in Business-Fehler.
        """
        try:
            # Platzhalter für die echte Kùzu-Transaktionslogik
            # Später: self._session.execute(...)
            print(f"[DAL] Speichere {len(graph_data_wrapper.nodes)} Knoten in
'{self._db_path}'...")
            return True
        except Exception as e:
            # Rohe C++/Treiber-Fehler abfangen und als sauberen Fehler hochreichen
            raise DatabaseConnectionError(f"Fehler beim Schreiben in die Kùzu-
Datenbank: {str(e)}")
```

File: gui\\_\_init\_\_.py

```
from .window import MainWindow
```

## File: gui\designer.py

```

# -*- coding: utf-8 -*-
"""
Presentation Layer: Integriertes Kontrollzentrum als linke Navigationsleiste
(Sidebar) mit funktionalen Gruppen für Hauptmenü, Items und Register.
Erweitert um Undo/Redo-Schutz und Positionsverschiebungen.
Bereinigt: Makro-Zuweisung in die Entwicklertools ausgelagert.
"""

import uuid
from PyQt6.QtWidgets import QDialog, QMessageBox, QWidget, QVBoxLayout,
QHBoxLayout, QPushButton, QLabel, QFrame
from PyQt6.QtCore import Qt
import infrastructure.cfg as cfg

class MenuDesigner:
    """Kapselt alle Laufzeit-Operationen zur Modifikation des UI-Schemas in einer
    linken Sidebar."""

    @staticmethod
    def inject_designer_sidebar_at(main_window, target_widget):
        """Erstellt das Layout des System-Designers innerhalb des übergebenen
        Ziel-Widgets."""
        panel_layout = QVBoxLayout()
        panel_layout.setContentsMargins(0, 0, 0, 0)
        panel_layout.setAlignment(Qt.AlignmentFlag.AlignTop)
        target_widget.setLayout(panel_layout)

        # INHALTS-BEREICH
        main_window._designer_content = QWidget()
        main_window._designer_content.setStyleSheet("""
        QWidget { border: none; background: transparent; }
        QLabel.group_title { font-weight: bold; color: #6c757d; font-size: 11px;
margin-top: 8px; margin-bottom: 2px; }
        QPushButton { background-color: #ffffff; border: 1px solid #ced4da;
border-radius: 4px; padding: 6px; text-align: left; font-size: 11px; }
        QPushButton:hover { background-color: #e9ecef; }
        """)

        content_layout = QVBoxLayout()
        content_layout.setContentsMargins(10, 5, 10, 10)
        content_layout.setSpacing(5)
        content_layout.setAlignment(Qt.AlignmentFlag.AlignTop)
        main_window._designer_content.setLayout(content_layout)

    def create_separator():
        line = QFrame()
        line.setFrameShape(QFrame.Shape.HLine)
        line.setFrameShadow(QFrame.Shadow.Sunken)
        line.setStyleSheet("color: #ced4da; margin-top: 4px; margin-bottom:
4px;")

```



```

        return line

    i18n = main_window._i18n

    # -----
    # SEKTION 0: SAFETY ZONE (Undo / Redo ganz oben)
    # -----
    lbl_safety = QLabel("SICHERHEITS-VERLAUF")
    lbl_safety.setStyleSheet("font-weight: bold; color: #1a73e8; font-size:
10px; margin-top: 5px;")
    content_layout.addWidget(lbl_safety)

    safety_layout = QHBoxLayout()
    btn_undo = QPushButton()
    btn_redo = QPushButton()

    i18n.register(btn_undo, "button", "sidebar.safety.undo")
    i18n.register(btn_redo, "button", "sidebar.safety.redo")

    btn_undo.clicked.connect(lambda: MenuDesigner._trigger_undo(main_window))
    btn_redo.clicked.connect(lambda: MenuDesigner._trigger_redo(main_window))
    safety_layout.addWidget(btn_undo)
    safety_layout.addWidget(btn_redo)
    content_layout.addLayout(safety_layout)
    content_layout.addWidget(create_separator())

    # -----
    # SEKTION 1: HAUPTMENÜ
    # -----
    lbl_g1 = QLabel()
    i18n.register(lbl_g1, "button", "sidebar.menu.group")
    content_layout.addWidget(lbl_g1)

    btn_add_menu = QPushButton()
    i18n.register(btn_add_menu, "button", "sidebar.menu.add")
    btn_add_menu.clicked.connect(main_window.create_new_main_menu)
    content_layout.addWidget(btn_add_menu)

    btn_del_menu = QPushButton()
    i18n.register(btn_del_menu, "button", "sidebar.menu.delete")
    btn_del_menu.clicked.connect(lambda:
MenuDesigner._live_remove_main_menu(main_window))
    content_layout.addWidget(btn_del_menu)

    btn_ren_menu = QPushButton()
    i18n.register(btn_ren_menu, "button", "sidebar.menu.rename")
    btn_ren_menu.clicked.connect(lambda:
MenuDesigner._live_rename_main_menu(main_window))
    content_layout.addWidget(btn_ren_menu)

    move_layout = QHBoxLayout()
    move_layout.setSpacing(4)
    btn_move_left = QPushButton()
    btn_move_right = QPushButton()

```

```

        i18n.register(btn_move_left, "button", "sidebar.menu.move_left")
        i18n.register(btn_move_right, "button", "sidebar.menu.move_right")
        btn_move_left.clicked.connect(lambda:
MenuDesigner._live_move_menu(main_window, -1))
        btn_move_right.clicked.connect(lambda:
MenuDesigner._live_move_menu(main_window, 1))
        move_layout.addWidget(btn_move_left)
        move_layout.addWidget(btn_move_right)
        content_layout.addLayout(move_layout)
        content_layout.addWidget(create_separator())

# -----
# SEKTION 2: SCHALTFLÄCHEN (ITEMS)
# -----

lbl_g2 = QLabel()
i18n.register(lbl_g2, "button", "sidebar.item.group")
content_layout.addWidget(lbl_g2)

btn_add_item = QPushButton()
i18n.register(btn_add_item, "button", "sidebar.item.add")
btn_add_item.clicked.connect(lambda:
MenuDesigner._live_add_menu_item(main_window))
content_layout.addWidget(btn_add_item)

btn_del_item = QPushButton()
i18n.register(btn_del_item, "button", "sidebar.item.delete")
btn_del_item.clicked.connect(lambda:
MenuDesigner._live_remove_menu_item(main_window))
content_layout.addWidget(btn_del_item)

btn_ren_item = QPushButton()
i18n.register(btn_ren_item, "button", "sidebar.item.rename")
btn_ren_item.clicked.connect(lambda:
MenuDesigner._live_rename_menu_item(main_window))
content_layout.addWidget(btn_ren_item)

# HIER WURDE DER REINIGER REINGESETZT: KERN-KORREKTUR ERFOLGT (btn_macro
ENTFERNT!)

item_move_layout = QHBoxLayout()
btn_item_up = QPushButton()
btn_item_down = QPushButton()
i18n.register(btn_item_up, "button", "sidebar.item.move_up")
i18n.register(btn_item_down, "button", "sidebar.item.move_down")
btn_item_up.clicked.connect(lambda:
MenuDesigner._live_move_menu_item(main_window, -1))
        btn_item_down.clicked.connect(lambda:
MenuDesigner._live_move_menu_item(main_window, 1))
        item_move_layout.addWidget(btn_item_up)
        item_move_layout.addWidget(btn_item_down)
        content_layout.addLayout(item_move_layout)
        content_layout.addWidget(create_separator())

# -----

```

```

# SEKTION 3: REGISTERKARTEN (TABS)
# -----
lbl_g3 = QLabel()
i18n.register(lbl_g3, "button", "sidebar.tab.group")
content_layout.addWidget(lbl_g3)

btn_add_tab = QPushButton()
i18n.register(btn_add_tab, "button", "sidebar.tab.add")
btn_add_tab.clicked.connect(lambda:
MenuDesigner._live_add_new_tab(main_window))
content_layout.addWidget(btn_add_tab)

btn_del_tab = QPushButton()
i18n.register(btn_del_tab, "button", "sidebar.tab.delete")
btn_del_tab.clicked.connect(lambda:
MenuDesigner._live_remove_tab(main_window))
content_layout.addWidget(btn_del_tab)

btn_ren_tab = QPushButton()
i18n.register(btn_ren_tab, "button", "sidebar.tab.rename")
btn_ren_tab.clicked.connect(lambda:
MenuDesigner._live_rename_tab(main_window))
content_layout.addWidget(btn_ren_tab)

tab_move_layout = QHBoxLayout()
btn_tab_left = QPushButton()
btn_tab_right = QPushButton()
i18n.register(btn_tab_left, "button", "sidebar.tab.move_left")
i18n.register(btn_tab_right, "button", "sidebar.tab.move_right")
btn_tab_left.clicked.connect(lambda:
MenuDesigner._live_move_tab(main_window, -1))
btn_tab_right.clicked.connect(lambda:
MenuDesigner._live_move_tab(main_window, 1))
tab_move_layout.addWidget(btn_tab_left)
tab_move_layout.addWidget(btn_tab_right)
content_layout.addLayout(tab_move_layout)

panel_layout.addWidget(main_window._designer_content)
i18n.translate_all()

# -----
# Steuerung aller Auswahldialoge für Hauptmenüs, Items und Tabs (einheitlich &
zentral)
# -----
@staticmethod
def _show_generic_selection_dialog(win, section_prefix, items_dict,
custom_title=None):
    """
    Zentraler, generischer Dialog für ALLE Sektionen.
    Nutzt entweder den custom_title für den blauen Balken oder fällt auf die
Gruppe zurück.
    """
    from PyQt6.QtWidgets import QInputDialog

```

```

        if custom_title:
            dialog_title = custom_title
        else:
            dialog_title = win._i18n.text(f"sidebar.{section_prefix}.group") or
section_prefix.upper()

        fallback_label = f"{win._i18n.text(f'sidebar.{section_prefix}.group') or
section_prefix.upper()} wählen:"
        raw_translation = win._i18n.text(f"dialog.{section_prefix}.select_label")

        if not raw_translation or raw_translation == f"dialog.
{section_prefix}.select_label":
            label_text = fallback_label
        else:
            label_text = raw_translation

        display_names = list(items_dict.keys())

        chosen_name, ok = QInputDialog.getItem(
            win,
            dialog_title,
            label_text,
            display_names,
            0,
            False
        )
        if ok and chosen_name:
            return items_dict[chosen_name]
        return None

# -----
# UNDO / REDO TRIGGER
# -----

# -----
-
# UNDO / REDO TRIGGER
# -----
@staticmethod
def _trigger_undo(win):
    if "ui_state_manager" in win._services:
        # 1. Merke dir das aktuell aktive Register, BEVOR wir alles abreißen
        current_tab_idx = win._tabs.currentIndex() if hasattr(win, "_tabs")
    else 0

    if win._services["ui_state_manager"].undo_action():
        win.setup_ui()

        # 2. Springe sofort wieder auf das Register zurück
        if hasattr(win, "_tabs") and current_tab_idx < win._tabs.count():
            win._tabs.setCurrentIndex(current_tab_idx)

        # === REPARATUR: SIGNALE DER SIDEBARS NEU VERDRAHTEN ===
        from gui.designer import MenuDesigner

```

```

        if hasattr(win, "ui") and hasattr(win.ui, "sidebar_container"):
            MenuDesigner.inject_sidebar_at(win, win.ui.sidebar_container)

        msg = win._i18n.text("notification.undo.success") or "Rückgängig
erfolgreich durchgeführt."
        win.statusBar().showMessage(msg, 2000)

    @staticmethod
    def _trigger_redo(win):
        if "ui_state_manager" in win._services:
            # 1. Merke dir das aktuell aktive Register, BEVOR wir alles abreißen
            current_tab_idx = win._tabs.currentIndex() if hasattr(win, "_tabs")
        else 0

        if win._services["ui_state_manager"].redo_action():
            win.setup_ui()

            # 2. Springe sofort wieder auf das Register zurück
            if hasattr(win, "_tabs") and current_tab_idx < win._tabs.count():
                win._tabs.setCurrentIndex(current_tab_idx)

            # === REPARATUR: SIGNALE DER SIDEBARS NEU VERDRAHTEN ===
            from gui.designer import MenuDesigner
            if hasattr(win, "ui") and hasattr(win.ui, "sidebar_container"):
                MenuDesigner.inject_sidebar_at(win, win.ui.sidebar_container)

            msg = win._i18n.text("notification.redo.success") or "Wiederholen
erfolgreich durchgeführt."
            win.statusBar().showMessage(msg, 2000)

# -----
# OPERATIVE LOGIK MIT SCHUTZSCHILDEN (HAUPTMENÜ)
# -----
    @staticmethod
    def _live_remove_main_menu(win):
        """Löscht ein Hauptmenü nach erfolgreicher, bilingualer
Sicherheitsabfrage."""
        target_uuid = MenuDesigner._select_main_menu_dialog(win,
action_type="delete")
        if not target_uuid:
            return
        # Absoluter Schutz des Kern-Menüs (Datei darf niemals gelöscht werden)
        if target_uuid == "file" or cfg.UI_SCHEMA["menu_structure"]
[target_uuid].get("i18n_key") == "menu.file":
            QMessageBox.warning(win, win._i18n.text("msg.protection.title"),
win._i18n.text("msg.protection.core_deny"))
            return

        i18n_key = cfg.UI_SCHEMA["menu_structure"][target_uuid].get("i18n_key",
f"menu.{target_uuid}")
        localized_menu_name = win._i18n.text(i18n_key)
        dialog_title = win._i18n.text("sidebar.menu.delete") or "Hauptmenü
löschen"

```

```

        raw_message = win._i18n.text("dialog.menu.delete.confirm") or "Möchten Sie
das Menü '{0}' wirklich löschen?"
        formatted_message = raw_message.format(localized_menu_name)

        reply = QMessageBox.question(
            win,
            dialog_title,
            formatted_message,
            QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
            QMessageBox.StandardButton.No
        )

        if reply == QMessageBox.StandardButton.Yes:
            if "ui_state_manager" in win._services:
                win._services["ui_state_manager"].save_checkpoint()
            del cfg.UI_SCHEMA["menu_structure"][target_uuid]

            if "menu_order" in cfg.UI_SCHEMA and target_uuid in
cfg.UI_SCHEMA["menu_order"]:
                cfg.UI_SCHEMA["menu_order"].remove(target_uuid)
            win.setup_ui()

... [HINWEIS: Datei wurde hier nach 300 Zeilen gekürzt] ...

```

File: `gui\factory.py`

```

# -*- coding: utf-8 -*-
"""Presentation Layer: Generische Widget-Fabrik mit Editor-Sprung und Größen-
Begrenzung"""

import os
import subprocess
from PyQt6.QtWidgets import (QLabel, QPushButton, QLineEdit, QTextEdit,
                             QComboBox, QCheckBox, QRadioButton, QGroupBox,
                             QVBoxLayout)
import infrastructure.cfg as cfg

class WidgetFactory:
    """Erzeugt und stylet UI-Elemente rein deklarativ anhand des Schemas."""

    @staticmethod
    def create_widget(elem_config: dict, services: dict, element_registry: dict):
        """Fabrik-Methode: Baut das passende Widget und wendet das rahmenlose
        Styling an."""
        i18n = services["i18n"]
        elem_type = elem_config["type"]
        elem_id = elem_config["id"]
        widget = None

        if elem_type == "label":
            widget = QLabel()

```

```

        i18n.register(widget, "headline", elem_config["i18n_key"])

    elif elem_type == "text_viewer":
        widget = QTextEdit()
        widget.setReadOnly(True)
        i18n.register(widget, "placeholder", elem_config["placeholder_i18n"])
        widget.setStyleSheet("border: 1px solid #ced4da; border-radius: 4px;
background-color: #f8f9fa; color: #495057;")
        element_registry[elem_id] = widget

    elif elem_type == "input_line":
        widget = QLineEdit()
        i18n.register(widget, "placeholder", elem_config["placeholder_i18n"])
        widget.setStyleSheet("border: 1px solid #ced4da; border-radius: 4px;
padding: 6px; background-color: #ffffff;")
        element_registry[elem_id] = widget

    elif elem_type == "input_text":
        widget = QTextEdit()
        i18n.register(widget, "placeholder", elem_config["placeholder_i18n"])
        widget.setStyleSheet("border: 1px solid #ced4da; border-radius: 4px;
padding: 6px; background-color: #ffffff;")
        element_registry[elem_id] = widget

    elif elem_type == "button":
        widget = QPushButton()
        i18n.register(widget, "button", elem_config["i18n_key"])
        widget.setStyleSheet("background-color: #ffffff; border: 1px solid
#ced4da; border-radius: 4px; padding: 7px; font-weight: bold;")

        if "macro_file" in elem_config and "macro_event" in elem_config:
            # Signal-Schutz: Alte Verbindungen kappen, um Blockaden zu
verhindern
            try: widget.clicked.disconnect()
            except Exception: pass
            widget.clicked.connect(
                lambda checked: WidgetFactory._trigger_live_macro(services,
elem_config, checked)
            )
            element_registry[elem_id] = widget

    elif elem_type == "combobox":
        widget = QComboBox()
        widget.setStyleSheet("border: 1px solid #ced4da; border-radius: 4px;
padding: 5px; background-color: #ffffff;")
        # Falls ein Platzhalter-Key im Schema definiert ist, als Typ
"placeholder" registrieren
        if "placeholder_i18n" in elem_config:
            i18n.register(widget, "placeholder",
elem_config["placeholder_i18n"])
        else:
            widget.addItem("Auswahl 1")
            widget.addItem("Auswahl 2")

```

```

        if "macro_file" in elem_config and "macro_event" in elem_config:
            try: widget.currentTextChanged.disconnect()
            except Exception: pass
            widget.currentTextChanged.connect(
                lambda text: WidgetFactory._trigger_live_macro(services,
elem_config, text)
            )
            element_registry[elem_id] = widget

    elif elem_type == "checkbox":
        widget = QCheckBox()
        # Registrierung als "checkbox"-Typ für dedizierte Text-Zuweisung
        i18n.register(widget, "checkbox", elem_config["i18n_key"])
        widget.setStyleSheet("background-color: transparent; padding: 4px;")

        if "macro_file" in elem_config and "macro_event" in elem_config:
            try: widget.toggled.disconnect()
            except Exception: pass
            widget.toggled.connect(
                lambda is_checked: WidgetFactory._trigger_live_macro(services,
elem_config, is_checked)
            )
            element_registry[elem_id] = widget

    elif elem_type == "radiobutton":
        widget = QRadioButton()
        # Registrierung als "radiobutton"-Typ
        i18n.register(widget, "radiobutton", elem_config["i18n_key"])
        widget.setStyleSheet("background-color: transparent; padding: 4px;")

        if "macro_file" in elem_config and "macro_event" in elem_config:
            try: widget.toggled.disconnect()
            except Exception: pass
            widget.toggled.connect(
                lambda is_checked: WidgetFactory._trigger_live_macro(services,
elem_config, is_checked)
            )
            element_registry[elem_id] = widget

    elif elem_type == "groupbox":
        widget = QGroupBox()
        # GroupBox benötigt ".setTitle()", daher als Typ "groupbox"
registrieren
        i18n.register(widget, "groupbox", elem_config["i18n_key"])
        group_layout = QVBoxLayout(widget)
        group_layout.setContentsMargins(8, 16, 8, 8)
        widget.setStyleSheet("QGroupBox { border: 1px solid #ced4da; border-
radius: 4px; margin-top: 6px; font-weight: bold; }")
        element_registry[elem_id] = widget

```

```

# =====
# UNIVERSAL-GRÖSSEN- UND POSITIONSTEUERUNG FÜR ALLE ELEMENTE
# =====

```



```

        if widget and "width" in elem_config and "height" in elem_config:
            widget.setFixedSize(int(elem_config["width"]),
int(elem_config["height"]))
        elif widget and "width" in elem_config:
            widget.setFixedWidth(int(elem_config["width"]))

        if widget and "x" in elem_config and "y" in elem_config:
            widget.move(int(elem_config["x"]), int(elem_config["y"]))

        # ONCE-ONLY: AUTOMATISCHER RECHTSKlick-SCHUTZ FÜR ALLE ELEMENTE
        if widget and "macro_file" in elem_config and "macro_event" in
elem_config:
            WidgetFactory._attach_developer_context_menu(widget, services,
elem_config)

        return widget

    @staticmethod
    def _trigger_live_macro(services: dict, elem_config: dict, event_data):
        """Zentrale Brücke, die Events an das Tab-Sammel-Makro weiterleitet und
die IDE öffnet."""
        importlib
        from PyQt6.QtWidgets import QApplication

        module_path = elem_config.get("macro_file")
        event_method_name = elem_config.get("macro_event")

        if not module_path or not event_method_name:
            return

        try:
            module = importlib.import_module(module_path)
            importlib.reload(module)
            file_absolute_path = getattr(module, "__file__", None)

            for attr_name in dir(module):
                if attr_name.startswith("TabControl_") or
attr_name.endswith("Command"):
                    cls = getattr(module, attr_name)
                    main_win = None
                    for w in QApplication.topLevelWidgets():
                        if w.inherits("QMainWindow"):
                            main_win = w
                            break

                    if main_win:
                        instance = cls(main_win)
                        if hasattr(instance, event_method_name):
                            method = getattr(instance, event_method_name)
                            method(event_data)

                            if file_absolute_path and
os.path.exists(file_absolute_path):
                                line_number = 1

```

```

        try:
            with open(file_absolute_path, "r",
encoding="utf-8") as f:
                for idx, line in enumerate(f, 1):
                    if f"def {event_method_name}" in line:
                        line_number = idx
                        break
        except Exception:
            pass

        # OPTIONALER EDITOR-SPRUNG MIT OK/CANCEL BUTTONS
        from PyQt6.QtWidgets import QMessageBox
        i18n = services.get("i18n")
        lang = getattr(i18n, "_current_lang", "de")

        btn_ok_text = i18n.text("ide.sonde.btn_edit") if
i18n else ("Code bearbeiten" if lang == "de" else "Edit Code")
        btn_cancel_text = i18n.text("ide.sonde.btn_close")
if i18n else ("Schließen" if lang == "de" else "Close")

        msg_box = QMessageBox(main_win)

        msg_box.setWindowTitle(i18n.text("ide.sonde.title") or "ClioGraph IDE")
        msg_box.setText(i18n.text("ide.sonde.question") or
("Möchten Sie das Makro in VS Code bearbeiten?" if lang == "de" else "Do you want
to edit the macro in VS Code?"))
        msg_box.setIcon(QMessageBox.Icon.Question)

        edit_button = msg_box.addButton(btn_ok_text,
QMessageBox.ButtonRole.ActionRole)
        close_button = msg_box.addButton(btn_cancel_text,
QMessageBox.ButtonRole.RejectRole)
        msg_box.exec()

        if msg_box.clickedButton() == edit_button:
            try:
                subprocess.Popen([cfg.EXTERNAL_EDITOR_COMMAND, "-g", f"{file_absolute_path}:
{line_number}"], shell=True)
            except Exception:
                pass

        break
    except Exception as e:
        print(f" [IDE-LIVE-MACRO] Fehler: {e}")

    @staticmethod
    def _attach_developer_context_menu(widget, services: dict, elem_config: dict):
        """Kapselt das bilinguale IDE-Kontextmenü für JEDES Widget mit sauberer
        Speicher-Trennung."""
        from PyQt6.QtCore import Qt
        from PyQt6.QtGui import QAction
        from PyQt6.QtWidgets import QMenu, QDialog, QLabel, QSpinBox, QPushButton,
        QFormLayout, QApplication
        import infrastructure.cfg as cfg

```

```

widget.setContextMenuPolicy(Qt.ContextMenuPolicy.CustomContextMenu)

def get_main_window():
    if services and "main_window" in services:
        return services["main_window"]
    for w in QApplication.topLevelWidgets():
        if w.inherits("QMainWindow"):
            return w
    return None

def refresh_ui(main_win):
    if not main_win:
        return
    from PyQt6.QtWidgets import QApplication, QScrollArea, QWidget
    import infrastructure.cfg as cfg

    persistence = main_win._services.get("menu_persistence")
    if persistence and hasattr(persistence, "save_menus"):
        persistence.save_menus(cfg.UI_SCHEMA)

    outer_widget = main_win._tabs.currentWidget()
    if not outer_widget:
        return

    if isinstance(outer_widget, QScrollArea):
        current_tab = outer_widget.widget()
    else:
        current_tab = outer_widget

    if current_tab:
        if hasattr(current_tab, "canvas_widget") and
current_tab.canvas_widget:
            for child in current_tab.canvas_widget.findChildren(QWidget):
                child.setParent(None)
                child.deleteLater()
            current_tab.canvas_widget.setMinimumSize(0, 0)

        if current_tab.layout():
            layout = current_tab.layout()
            for i in reversed(range(layout.count())):
                item = layout.itemAt(i)
                if item and item.widget():
                    w = item.widget()
                    layout.removeWidget(w)
                    w.setParent(None)
                    w.deleteLater()

        if hasattr(current_tab, "element_registry"):
            current_tab.element_registry.clear()

        t_id = getattr(current_tab, "tab_id", "unknown")
        tab_schema = next((t for t in cfg.UI_SCHEMA.get("tab_structure",
[[]] if t.get("id") == t_id), {}))

```

```

        elements_list = tab_schema.get("elements", [])

        if hasattr(current_tab, "_build_tab_geometry"):
            current_tab._build_tab_geometry(elements_list)
        elif hasattr(main_win, "build_dynamic_tabs"):
            main_win.build_dynamic_tabs()

    i18n = main_win._services.get("i18n")
    if i18n:
        i18n.translate_all()

    if current_tab and hasattr(current_tab, "canvas_widget") and
current_tab.canvas_widget:
        for child in current_tab.canvas_widget.findChildren(QWidget):
            child.show()
            child.raise_()

    QApplication.processEvents()
    if main_win:
        main_win.update()

def show_menu(pos):
    i18n = services.get("i18n")
    lang = getattr(i18n, "_current_lang", "de")
    main_win = get_main_window()

    txt_jump = i18n.text("ide.context.jump_macro") if i18n else ("[IDE]
Zum Makro-Code springen" if lang == "de" else "[IDE] Jump to Macro Code")
    txt_prop = i18n.text("ide.context.properties") if i18n else ("[IDE]
Größe und Position..." if lang == "de" else "[IDE] Size and Position...")
    txt_up = i18n.text("ide.context.move_up") if i18n else ("[IDE] Nach
oben verschieben" if lang == "de" else "[IDE] Move Up")
    txt_down = i18n.text("ide.context.move_down") if i18n else ("[IDE]
Nach unten verschieben" if lang == "de" else "[IDE] Move Down")
    txt_ren = "[IDE] Element umbenennen" if lang == "de" else "[IDE]
Rename Element"
    txt_del = i18n.text("ide.context.delete") if i18n else ("[IDE] Element
löschen" if lang == "de" else "[IDE] Delete Element")

    context_menu = QMenu(main_win or widget)

    act_jump = QAction(txt_jump, context_menu)
    act_jump.triggered.connect(lambda:
WidgetFactory._trigger_live_macro(services, elem_config, None))
    context_menu.addAction(act_jump)
    context_menu.addSeparator()

    act_prop = QAction(txt_prop, context_menu)
    act_prop.triggered.connect(lambda:
open_properties_dialog(get_main_window(), services.get("i18n"), elem_config,
widget))
    context_menu.addAction(act_prop)

```

... [HINWEIS: Datei wurde hier nach 300 Zeilen gekürzt] ...

File: gui\ide\_designer.py

```
# -*- coding: utf-8 -*-
"""Presentation Layer: Adressatengerechte Entwickler-IDE (Formularsteuerung &
Code-Templates)"""

import os
import uuid
import json
from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
QLabel, QComboBox, QLineEdit, QInputDialog, QMessageBox, QFrame
from PyQt6.QtCore import Qt
import infrastructure.cfg as cfg

class IDEDesigner:
    """Isoliertes Steuerungspanel für Entwickler-Werkzeuge (Kapselung)."""

    @staticmethod
    def inject_ide_sidebar_at(main_window, container_widget):
        """Erzeugt das visuelle Layout innerhalb der Sidebar (Koffer-Modus)."""
        panel_layout = QVBoxLayout()
        panel_layout.setContentsMargins(0, 0, 0, 0)
        panel_layout.setAlignment(Qt.AlignmentFlag.AlignTop)
        container_widget.setLayout(panel_layout)

        content_container = QWidget()
        content_container.setStyleSheet("""
            QWidget { border: none; background: transparent; }
            QLabel.group_title {
                font-weight: normal;
                text-transform: uppercase;
                color: #2b2b2b;
                font-size: 11px;
                letter-spacing: 0.5px;
                margin-top: 14px;
                margin-bottom: 4px;
            }
            QPushButton { background-color: #ffffff; border: 1px solid #ced4da;
border-radius: 4px; padding: 7px; text-align: left; font-size: 12px; }
            QPushButton:hover { background-color: #e9ecef; }
            QComboBox { background-color: #ffffff; border: 1px solid #ced4da;
border-radius: 4px; padding: 5px; font-size: 12px; }
            QLineEdit { background-color: #ffffff; border: 1px solid #ced4da;
border-radius: 4px; padding: 7px; font-size: 12px; }
        """)

        content_layout = QVBoxLayout()
        content_layout.setContentsMargins(12, 5, 12, 12)
```

```

content_layout.setSpacing(6)
content_layout.setAlignment(Qt.AlignmentFlag.AlignTop)
content_container.setLayout(content_layout)

def create_separator():
    line = QFrame()
    line.setFrameShape(QFrame.Shape.HLine)
    line.setFrameShadow(QFrame.Shadow.Sunken)
    line.setStyleSheet("color: #ced4da; margin-top: 6px; margin-bottom:
6px;")
    return line

i18n = main_window._services["i18n"]

# =====
# SEKTION 0: SAFETY ZONE (Undo / Redo ganz oben in der IDE-Sidebar)
# =====
lbl_safety = QLabel("SICHERHEITS-VERLAUF (IDE)")
lbl_safety.setStyleSheet("font-weight: bold; color: #1a73e8; font-size:
10px; margin-top: 5px;")
content_layout.addWidget(lbl_safety)

safety_layout = QHBoxLayout()

# Initialisiere den Text sofort JIT (Just-in-Time) mit sicherem Fallback
btn_undo = QPushButton(i18n.text("sidebar.safety.undo") or "Rückgängig")
btn_redo = QPushButton(i18n.text("sidebar.safety.redo") or "Wiederholen")

# Registrierung für zukünftige Live-Sprachwechsel (de <-> en)
i18n.register(btn_undo, "button", "sidebar.safety.undo")
i18n.register(btn_redo, "button", "sidebar.safety.redo")

# Statische Trigger-Brücke zum synchronisierten UIStateManager im RAM
from gui.designer import MenuDesigner
btn_undo.clicked.connect(lambda: MenuDesigner._trigger_undo(main_window))
btn_redo.clicked.connect(lambda: MenuDesigner._trigger_redo(main_window))

safety_layout.addWidget(btn_undo)
safety_layout.addWidget(btn_redo)
content_layout.addLayout(safety_layout)
content_layout.addWidget(create_separator())

# =====
# BEREICH 1: MENÜBEFEHLE ZUORDNEN
# =====
lbl_cmd = QLabel(i18n.text("ide.group.menu_commands") or "📁 Menübefehle
zuordnen")
lbl_cmd.setProperty("class", "group_title")
content_layout.addWidget(lbl_cmd)

btn_assign = QPushButton(i18n.text("sidebar.item.assign_btn") or "Befehl
zuordnen...")
btn_assign.clicked.connect(lambda:

```

```

IDEDesigner._live_assign_macro_to_menu(main_window))
    content_layout.addWidget(btn_assign)

    content_layout.addWidget(create_separator())

    # =====
    # BEREICH 2: REGISTER-STEUERELEMENTE
    # =====
    lbl_vba = QLabel("🔧 Formular-Steuerelemente")
    lbl_vba.setProperty("class", "group_title")
    content_layout.addWidget(lbl_vba)

    combo_type = QComboBox()
    combo_type.addItem("Schaltfläche (VBA-Button)", "button")
    combo_type.addItem("Kombinationsfeld (Dropdown)", "combobox")
    combo_type.addItem("Kontrollkästchen (Checkbox)", "checkbox")
    combo_type.addItem("Optionsfeld (Radio Button)", "radiobutton")
    combo_type.addItem("Gruppenfeld (Group Box)", "groupbox")
    combo_type.addItem("Bezeichnung (Label)", "label")
    combo_type.addItem("Eingabe-Zeile (einzeilig)", "input_line")
    combo_type.addItem("Text-Feld (mehrzeilig)", "input_text")
    combo_type.addItem("Text-Anzeige (Viewer)", "text_viewer")
    content_layout.addWidget(combo_type)

    btn_insert = QPushButton(i18n.text("ide.btn.insert_widget") or "Baustein
auf Tab platzieren")
    btn_insert.clicked.connect(lambda:
IDEDesigner._live_insert_widget(main_window, combo_type.currentData()))
    content_layout.addWidget(btn_insert)

    move_layout = QHBoxLayout()
    move_layout.setSpacing(4)
    btn_move_up = QPushButton("⬆ Nach oben")
    btn_move_down = QPushButton("⬇ Nach unten")
    btn_move_up.setStyleSheet("font-size: 11px; padding: 5px; text-align:
center;")
    btn_move_down.setStyleSheet("font-size: 11px; padding: 5px; text-align:
center;")

    btn_move_up.clicked.connect(lambda:
IDEDesigner._live_move_element(main_window, -1))
    btn_move_down.clicked.connect(lambda:
IDEDesigner._live_move_element(main_window, 1))

    move_layout.addWidget(btn_move_up)
    move_layout.addWidget(btn_move_down)
    content_layout.addLayout(move_layout)

    content_layout.addWidget(create_separator())

    # =====
    # BEREICH 3: GENAI FORMULAR DESIGNER
    # =====
    lbl_ai = QLabel("🤖 GenAI Formular Designer")

```

```

lbl_ai.setProperty("class", "group_title")
content_layout.addWidget(lbl_ai)

prompt_input = QLineEdit()
prompt_input.setPlaceholderText(i18n.text("ide.placeholder.prompt") or
"Formular per Prompt beschreiben...")
content_layout.addWidget(prompt_input)

btn_generate = QPushButton(i18n.text("ide.btn.generate") or "Formular
generieren")
btn_generate.clicked.connect(lambda:
IDEDesigner._live_genai_build(main_window, prompt_input.text()))
content_layout.addWidget(btn_generate)

panel_layout.addWidget(content_container)
content_layout.addStretch()

@staticmethod
def _live_assign_macro_to_menu(main_window):
    """
    Verknüpft dynamisch Hauptmenü-Aktionen mit benutzerdefinierten Makros.
    Unterstützt unbegrenzt tief verschachtelte Submenüs (Rekursion) und
    schützt das Kernmenü 'Datei'.
    """
    from PyQt6.QtWidgets import QInputDialog, QMessageBox

    i18n = main_window._services.get("i18n")
    lang = getattr(i18n, "_current_lang", "de")

    menu_structure = cfg.UI_SCHEMA.get("menu_structure", {})
    action_items = {} # Anzeige-Name -> (Menu_ID, Item_Dict)

    def _extract_actions_recursive(items_list, parent_label_path, menu_id):
        """Durchwandert rekursiv alle Menüebenen, um Action-Items zu
        finden."""
        for item in items_list:
            item_type = item.get("type")
            item_id = item.get("id")

            # Lokalisierung des aktuellen Eintrags
            localized_item = i18n.text(item_id) if i18n else item_id
            current_path = f"{parent_label_path} -> {localized_item}"

            if item_type == "action":
                display_label = f"{current_path} ({item_id})"
                action_items[display_label] = (menu_id, item)

            elif item_type == "submenu" and "items" in item:
                # Tiefer in die Schachtelung eintauchen
                _extract_actions_recursive(item["items"], current_path,
menu_id)

    # Chronologischer WYSIWYG-Scan über alle Hauptmenüs

```



```

        for menu_id, menu_config in menu_structure.items():
            # ABSOLUTES SCHUTZSCHILD: Das komplette 'Datei'-Kernmenü wird
            # ignoriert
            if menu_id == "file" or menu_config.get("i18n_key") == "menu.file":
                continue

            menu_key = menu_config.get("i18n_key", f"menu.{menu_id}")
            localized_menu = i18n.text(menu_key) if i18n else menu_id.upper()

            if "items" in menu_config:
                _extract_actions_recursive(menu_config["items"], localized_menu,
                menu_id)

            if not action_items:
                title_empty = i18n.text("dialog.title.no_entries") if i18n else
                ("Keine Einträge" if lang == "de" else "No Entries")
                msg_empty = "Es wurden keine konfigurierbaren Benutzer-Menüeinträge im
                System gefunden." if lang == "de" else "No configurable user menu items found in
                the system."
                QMessageBox.warning(main_window, title_empty, msg_empty)
                return

            # Dialog 1: Welchem Menüpunkt soll das Makro zugeordnet werden?
            title_sel_item = i18n.text("sidebar.item.assign_btn") if i18n else "Befehl
            zuordnen..."
            label_sel_item = "Ziel-Menüeintrag auswählen:" if lang == "de" else
            "Select target menu item:"

            chosen_item_label, ok1 = QInputDialog.getItem(
                main_window, title_sel_item, label_sel_item,
                list(action_items.keys()), 0, False
            )
            if not ok1 or not chosen_item_label:
                return

            target_menu_id, target_item_dict = action_items[chosen_item_label]

            # 2. SCHRITT: Das Verzeichnis commands_user dynamisch nach .py-Makros
            # scannen
            base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
            user_cmd_dir = os.path.join(base_dir, "business", "commands_user")

            available_macros = []
            if os.path.exists(user_cmd_dir):
                for filename in os.listdir(user_cmd_dir):
                    if filename.endswith(".py") and not filename.startswith("__"):
                        available_macros.append(filename)

            available_macros.sort()

            if not available_macros:
                title_no_macro = "Keine Makros" if lang == "de" else "No Macros"
                msg_no_macro = f"Keine benutzerdefinierten Makros im Ordner
                '{user_cmd_dir}' gefunden." if lang == "de" else f"No user macros found in folder

```

```

'{user_cmd_dir}'."
    QMessageBox.information(main_window, title_no_macro, msg_no_macro)
    return

    # Dialog 2: Welches Makro aus dem Ordner soll zugewiesen werden?
    label_sel_macro = "Verfügbares Benutzer-Makro auswählen:" if lang == "de"
else "Select available user macro:"

    chosen_macro_file, ok2 = QInputDialog.getItem(
        main_window, title_sel_item, label_sel_macro, available_macros, 0,
False
    )
    if not ok2 or not chosen_macro_file:
        return

    # 3. SCHRITT: Modul-Pfad für den dynamischen Klassen-Importer aufbereiten
    macro_module_name = chosen_macro_file.replace(".py", "")
    parts = macro_module_name.split('_')
    class_name_parts = [p.capitalize() for p in parts if p.lower() != 'cmd']
    expected_class_name = "".join(class_name_parts) + "Command"

    full_command_path = f"business.commands_user.{macro_module_name}.
{expected_class_name}"

    # 4. SCHRITT: Transaktionale Aktualisierung & Live-Neuaufbau
    if "ui_state_manager" in main_window._services:
        main_window._services["ui_state_manager"].save_checkpoint()

    target_item_dict["command_class"] = full_command_path

    persistence = main_window._services.get("menu_persistence")
    if persistence and hasattr(persistence, "save_menus"):
        persistence.save_menus(cfg.UI_SCHEMA)

    if hasattr(main_window, "setup_ui"):
        main_window.setup_ui()

    msg_success = f"Makro '{expected_class_name}' erfolgreich mit
'{target_item_dict['id']}' verknüpft." if lang == "de" else f"Macro
'{expected_class_name}' successfully linked to '{target_item_dict['id']}'."
    main_window.statusBar().showMessage(msg_success, 4000)

    @staticmethod
    def _live_insert_widget(main_window, widget_type):
        """
        Platziert ein Excel-Steuerelement auf dem aktiven Tab und
        generiert/erweitert das Sammel-Makro.
        Garantiert die unzerbrechliche Persistenz des Typsystems auf der
        Festplatte.
        """
        from PyQt6.QtWidgets import QMessageBox, QInputDialog
        import uuid
        import infrastructure.cfg as cfg

```

```

i18n = main_window._services["i18n"]
current_tab_index = main_window._tabs.currentIndex()

# Schutzschild: Auf dem Willkommens-Register (Index 0) darf nichts
injiziert werden!
if current_tab_index == 0:
    QMessageBox.warning(main_window, "Schutzschild", "Das Willkommens-
Register ist geschützt und darf nicht modifiziert werden.")
    return

active_tab_widget = main_window._tabs.widget(current_tab_index)
# Ermittle die ID des aktuellen Tabs (z.B. "tab.extraction")
tab_id = getattr(active_tab_widget, "tab_id", f"tab_{current_tab_index}")
# Bereinige den Namen für den Dateinamen (Punkte durch Unterstriche
ersetzen)
clean_tab_name = tab_id.replace(".", "_")

# 1. BILINGUALE ABFRAGE FÜR DIE BESCHRIFTUNG (Zwingend Deutsch und
Englisch)
lang = getattr(i18n, "_current_lang", "de")
prompt_de = "Deutscher Platzhalter/Text:" if lang == "de" else "German
label/placeholder text:"
prompt_en = "Englischer Platzhalter/Text:" if lang == "de" else "English
counterpart text:"

text_de, ok1 = QInputDialog.getText(main_window, "VBA-Designer",
prompt_de)
if not ok1 or not text_de.strip(): return
text_en, ok2 = QInputDialog.getText(main_window, "VBA-Designer",
prompt_en, text=text_de.strip())

... [HINWEIS: Datei wurde hier nach 300 Zeilen gekürzt] ...

```

File: `gui\sidebar.py`

```

"""Presentation Layer: Modulares VS-Code-Style Sidebar-Framework.
Verwaltet eine schmale Icon-Leiste (ActivityBar) und ein StackedWidget für
wechselnde Sidebar-Inhalte."""
from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
QStackedWidget
from PyQt6.QtCore import Qt

class SidebarController(QWidget):
    """Das zentrale Kontrollzentrum für alle linken Sidebars (Icon-Leiste +
Inhalts-Stack)."""

    def __init__(self, main_window):
        super().__init__()
        self.main_window = main_window
        self.active_id = None # Speichert, welche Sidebar gerade offen ist

```

```

# Horizontales Layout: [ Schmale Icon-Leiste | Großer Inhalts-Stack ]
self.layout = QHBoxLayout()
self.layout.setContentsMargins(0, 0, 0, 0)
self.layout.setSpacing(0)
self.setLayout(self.layout)

# 1. Die schmale Icon-Leiste (Activity Bar)
self.activity_bar = QWidget()
self.activity_bar.setFixedWidth(46)
self.activity_bar.setStyleSheet("""
    QWidget { background-color: #f1f3f5; border-right: 1px solid #ced4da;
padding: 10px 0px; }
    QPushButton { background: transparent; border: none; font-size: 14px;
padding: 10px 0px; }
    QPushButton:hover { background-color: #e9ecef; }
    QPushButton:checked { background-color: #ffffff; border-left: 3px
solid #007acc; }
""")
self.activity_layout = QVBoxLayout()
self.activity_layout.setContentsMargins(0, 5, 0, 0)
self.activity_layout.setSpacing(5)
self.activity_layout.setAlignment(Qt.AlignmentFlag.AlignTop)
self.activity_bar.setLayout(self.activity_layout)
self.layout.addWidget(self.activity_bar)

# 2. Der Inhalts-Stack für die eigentlichen Sidebars
self.content_stack = QStackedWidget()
self.content_stack.setFixedWidth(220) # Feste Breite für das ausgeklappte
Menü

self.layout.addWidget(self.content_stack)

self.sidebar_mapping = {} # ID -> (Widget, Button)

def register_sidebar(self, sidebar_id: str, icon_text: str, widget: QWidget,
tooltip: str = ""):
    """Registriert ein neues Sidebar-Panel im System."""
    btn = QPushButton(icon_text)
    btn.setCheckable(True)
    btn.setToolTip(tooltip)
    btn.setFixedSize(46, 32)
    btn.clicked.connect(lambda: self.toggle_sidebar(sidebar_id))

    self.activity_layout.addWidget(btn)
    self.content_stack.addWidget(widget)
    self.sidebar_mapping[sidebar_id] = (widget, btn)

def toggle_sidebar(self, sidebar_id: str):
    """Wechselt die Sidebar oder schließt sie komplett, wenn sie bereits aktiv
war."""
    target_widget, target_btn = self.sidebar_mapping[sidebar_id]

    # Fall 1: Bereits offen -> Schließen
    if self.active_id == sidebar_id:

```

```

        self.content_stack.hide()
        target_btn.setChecked(False)
        self.active_id = None
        self.main_window._main_splitter.setSizes([46, self.main_window.width()
- 46])

# Fall 2: Öffnen / Wechseln
else:
    for sid, (_, btn) in self.sidebar_mapping.items():
        if sid != sidebar_id:
            btn.setChecked(False)

        target_btn.setChecked(True)
        self.content_stack.setCurrentWidget(target_widget)
        self.content_stack.show()
        self.active_id = sidebar_id
        self.main_window._main_splitter.setSizes([266,
self.main_window.width() - 266])

```

File: `gui\tabs.py`

```

# -*- coding: utf-8 -*-
"""Presentation Layer: Dynamisches Registerkarten-Framework (Tabs) mit
automatischer Layout-Segmentierung"""

from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
QSplitter, QMessageBox
from PyQt6.QtCore import Qt
from .factory import WidgetFactory

class DynamicTab(QWidget):
    """Generischer Tab, der seine UI-Elemente rein aus dem Schema generiert und
komplexe Segmente aufbaut."""

    def __init__(self, tab_config: dict, services: dict):
        super().__init__()
        self._services = services
        self._i18n = services["i18n"]
        self._element_registry = {} # Kapselung: Hält Referenzen auf die Felder
für die Businesslogik

        # Speichere die ID des aktuellen Tabs (Wichtig für die automatische Code-
Generierung!)
        self.tab_id = tab_config.get("id", "unknown_tab")
        self._layout_type = tab_config.get("layout_type", "form")

        # =====
        # BOOTSTRAPPING DER SCROLLBAR-INFRASTRUKTUR (Absolut absturzsicher)
        # =====

```

```

from PyQt6.QtWidgets import QScrollArea, QWidget, QVBoxLayout

# 1. Das äußere Haupt-Layout des Tabs bekommt eine ScrollArea injiziert
outer_layout = QVBoxLayout(self)
outer_layout.setContentsMargins(0, 0, 0, 0)

self.scroll_area = QScrollArea(self)
self.scroll_area.setWidgetResizable(True)

self.scroll_area.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAsNeeded
)

self.scroll_area.setVerticalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBarAsNeeded)
self.scroll_area.setStyleSheet("QScrollArea { border: none; background:
transparent; }")
outer_layout.addWidget(self.scroll_area)

# 2. Die eigentliche Leinwand ("Canvas"), auf der die absoluten Elemente
platziert werden
self.canvas_widget = QWidget()
self.scroll_area.setWidget(self.canvas_widget)

# 3. Unser inneres Haupt-Layout wird nun an das Canvas-Widget gekoppelt!
# NEU: Das Layout steuert nicht mehr das canvas_widget direkt!
self._main_layout = QVBoxLayout()
self.canvas_widget.setLayout(self._main_layout)
self._main_layout.setContentsMargins(10, 10, 10, 10)

# Initialisiere die Segment-Container aus der JSON-Datei
self._build_tab_geometry(tab_config.get("elements", []))

def _build_tab_geometry(self, elements_schema: list):
    """Erzeugt rahmenlose Segmente basierend auf dem gewählten Layout-
    Typen."""
    if self._layout_type == "form":
        # Klassisches Formular: Elemente werden direkt untereinander gereiht
        for elem in elements_schema:
            self._create_and_append_widget(elem, self._main_layout)

        # Ein Stretch ganz unten sorgt für die adäquate Ausrichtung
        self._main_layout.addStretch()

    elif self._layout_type == "split_horizontal":
        from PyQt6.QtWidgets import QSplitter
        splitter = QSplitter(Qt.Orientation.Horizontal)
        splitter.setStyleSheet("QSplitter::handle { background-color: #ced4da;
}")

        splitter.setHandleWidth(4)

        left_widget = QWidget()
        left_layout = QVBoxLayout(left_widget)
        left_layout.setContentsMargins(0, 0, 0, 0)

        right_widget = QWidget()

```

```

        right_layout = QVBoxLayout(right_widget)
        right_layout.setContentsMargins(0, 0, 0, 0)

        for elem in elements_schema:
            pane = elem.get("pane", "left")
            target_layout = right_layout if pane == "right" else left_layout
            self._create_and_append_widget(elem, target_layout)

        splitter.addWidget(left_widget)
        splitter.addWidget(right_widget)
        self._main_layout.addWidget(splitter)

    elif self._layout_type == "split_vertical":
        from PyQt6.QtWidgets import QSplitter
        splitter = QSplitter(Qt.Orientation.Vertical)
        splitter.setStyleSheet("QSplitter::handle { background-color: #ced4da;
}"))

        splitter.setHandleWidth(4)

        top_widget = QWidget()
        top_layout = QVBoxLayout(top_widget)
        top_layout.setContentsMargins(0, 0, 0, 0)

        bottom_widget = QWidget()
        bottom_layout = QVBoxLayout(bottom_widget)
        bottom_layout.setContentsMargins(0, 0, 0, 0)

        for elem in elements_schema:
            pane = elem.get("pane", "top")
            target_layout = bottom_layout if pane == "bottom" else top_layout
            self._create_and_append_widget(elem, target_layout)

        splitter.addWidget(top_widget)
        splitter.addWidget(bottom_widget)
        self._main_layout.addWidget(splitter)

    def _create_and_append_widget(self, elem_config: dict, target_layout):
        """Hilfsfunktion zur Platzierung der Elemente auf der Leinwand."""
        # Generierung über deine Widget-Fabrik
        widget = WidgetFactory.create_widget(elem_config, self._services,
self._element_registry)

        if widget:
            # INTERNER TRANSITIONS-SCHUTZ: Prüft, ob canvas_widget existiert
            target_canvas = getattr(self, "canvas_widget", self)

            if "x" in elem_config and "y" in elem_config:
                # Das Widget muss ein Kind unseres Canvas-Widgets sein!
                widget.setParent(target_canvas)
                widget.move(int(elem_config["x"]), int(elem_config["y"]))
                widget.show()

            # Wir erweitern die Leinwand-Größe dynamisch, damit die

```

Scrollbar anspringen

```

        if hasattr(self, "canvas_widget"):
            needed_w = int(elem_config["x"]) + widget.width() + 40
            needed_h = int(elem_config["y"]) + widget.height() + 40
            self.canvas_widget.setMinimumSize(
                max(self.canvas_widget.minimumWidth(), needed_w),
                max(self.canvas_widget.minimumHeight(), needed_h)
            )
        else:
            # Ansonsten wird es normal in den Layout-Sizer eingepflegt
            target_layout.addWidget(widget)

# =====
# LIVE-REPAINT TRIGGER: Erzwingt das physische Rendern auf Klassenebene
# =====
        if widget:
            widget.updateGeometry()
            widget.repaint()

    def _execute_assigned_command(self, command_class_name: str):
        """Sucht das zuständige Command und übergibt die Daten des Tabs an die
        Business-Logik."""
        if not command_class_name:
            return

        discover_func = self._services.get("command_finder")
        if not discover_func:
            return

        available_macros = discover_func(filter_prefix=None)
        for _, cmd_instance in available_macros.items():
            if cmd_instance.__class__.__name__ == command_class_name:
                # Kapselung: Das Business-Command erhält NUR das i18n-entkoppelte
                Datenpaket
                if hasattr(cmd_instance, "execute_with_ui_context"):
                    cmd_instance.execute_with_ui_context(self._services,
                    self._element_registry)
                else:
                    cmd_instance.execute(self._services)
                break

    def add_element_live(self, element_config: dict):
        """Fügt ein neues Excel-Steuerelement zur Laufzeit live in dieses Register
        ein."""
        services = self._services

        # Live-Generierung über deine Fabrik
        new_widget = WidgetFactory.create_widget(element_config, services,
        self._element_registry)

        if new_widget is not None:
            # SCHUTZ-FALLBACK: Falls canvas_widget (noch) nicht existiert, nutzen
            wir das Register selbst

```



```

        target_canvas = getattr(self, "canvas_widget", self)

        if "x" in element_config and "y" in element_config:
            # Absolute Platzierung zur Laufzeit
            new_widget.setParent(target_canvas)
            new_widget.move(int(element_config["x"]),
int(element_config["y"]))

        # Scroll-Leinwand anpassen (nur wenn canvas_widget physisch
existiert)
        if hasattr(self, "canvas_widget"):
            needed_w = int(element_config["x"]) + new_widget.width() + 40
            needed_h = int(element_config["y"]) + new_widget.height() + 40
            self.canvas_widget.setMinimumSize(
                max(self.canvas_widget.minimumWidth(), needed_w),
                max(self.canvas_widget.minimumHeight(), needed_h)
            )
        else:
            # Sizer-Fallklasse (Fallback)
            if hasattr(self, "_main_layout"):
                count = self._main_layout.count()
                if count > 1:
                    self._main_layout.insertWidget(count - 1, new_widget)
                else:
                    self._main_layout.addWidget(new_widget)

        # Element-ID registrieren
        elem_id = element_config.get("id")
        self._element_registry[elem_id] = new_widget

        # Physisches Zeichnen erzwingen
        new_widget.show()
        if hasattr(self, "_main_layout"):
            self._main_layout.invalidate()
            self._main_layout.activate()
        self.updateGeometry()
        self.repaint()

```

File: gui\window.py

```

"""Presentation Layer: Generisches Hauptfenster-Gehäuse (MainWindow) mit
optimierter Pfeil-Faltung"""
import uuid
from PyQt6.QtWidgets import QMainWindow, QTabWidget, QInputDialog, QSplitter,
QWidget, QHBoxLayout, QMessageBox
from PyQt6.QtCore import Qt
from PyQt6.QtCore import QTimer
import copy
import infrastructure.cfg as cfg
from .tabs import DynamicTab
from .designer import MenuDesigner

```

```

from .sidebar import SidebarController

class MainWindow(QMainWindow):

    """Das vollkommen generische Hauptfenster von ClioGraph mit einklappbarer
    Sidebar."""
    def __init__(self, services: dict, ui_schema: dict):
        super().__init__()
        import infrastructure.cfg as cfg # Sicherstellen, dass cfg importiert ist
        import json
        import os

        self._services = services
        self._i18n = services["i18n"]
        self._schema = ui_schema
        self._tab_mapping = []
        self._btn_toggle_sidebar = None
        self._sidebar_title_label = None
        self._designer_content = None
        self._sidebar_is_collapsed = False
        self.resize(1150, 700)

        # =====
        # DATENFLUSS-REPARATUR: Direktes, lückenloses Laden von der Festplatte
        # =====
        file_path = getattr(cfg, "DYNAMIC_MENU_PATH",
"json_storage/dynamic_menu.json")

        # Falls die JSON-Datei existiert, lesen wir das VOLLSTÄNDIGE Live-Schema
ein
        if os.path.exists(file_path):
            try:
                with open(file_path, "r", encoding="utf-8") as f:
                    full_schema = json.load(f)

                    cfg.UI_SCHEMA["menu_structure"] =
full_schema.get("menu_structure", {})
                    cfg.UI_SCHEMA["menu_order"] = full_schema.get("menu_order",
[],)

                    # --- KERN-REPARATUR: Register werden nun garantiert
persistent geladen! ---
                    cfg.UI_SCHEMA["tab_structure"] =
full_schema.get("tab_structure", [])
            except Exception as e:
                print(f"🚨 Fehler beim Laden des Registerschemas aus {file_path}:
{e}")
        else:
            # Fallback, falls die Datei noch gar nicht existiert (z. B. nach dem
Löschen)
            if ui_schema:
                cfg.UI_SCHEMA["menu_structure"] = ui_schema.get("menu_structure",
{})

                cfg.UI_SCHEMA["menu_order"] = ui_schema.get("menu_order", [])
                cfg.UI_SCHEMA["tab_structure"] = ui_schema.get("tab_structure",

```

[ ])

```

# =====
# VISUELLE IDENTITÄT: Echtes Kleeblatt-Icon (Vernichtet das Windows-
Rechteck!)
# =====
self.setWindowIcon(self._generate_clover_icon())

# 1. ZENTRALER SPLITTER
self._main_splitter = QSplitter(Qt.Orientation.Horizontal)
self._main_splitter.setHandleWidth(4)
self._main_splitter.setStyleSheet("QSplitter::handle { background-color:
#ced4da; }")

# 2. Modularen Sidebar-Manager für die linke Seite einsetzen (VS-Code
Style)
self.sidebar_manager = SidebarController(self)
self._main_splitter.addWidget(self.sidebar_manager)

# 3. Das Tab-Widget für die rechte Seite
self._tabs = QTabWidget()
self._tabs.setStyleSheet("QTabWidget::pane { border: none; }")

# =====
# HIER IST JETZT DER RICHTIGE PLATZ (Das Objekt existiert nun sicher!)
# =====
self._tabs.currentChanged.connect(self._on_tab_changed_check_plus)
self._main_splitter.addWidget(self._tabs)
self.setCentralWidget(self._main_splitter)

# Start-Größenverteilung setzen
self._main_splitter.setSizes([220, 1150 - 220])

# Hellgraue vertikale Abgrenzungsstriche und VS-Style Aktiv-Hervorhebung
oben
self.menuBar().setStyleSheet("""
    QMenuBar {
        background-color: #f0f0f0; /* Neutraler, hellgrauer
Menühintergrund */
        border-bottom: 1px solid #ced4da;
    }
    QMenuBar::item {
        border-right: 1px solid #d3d3d3;
        padding-left: 12px;
        padding-right: 12px;
        padding-top: 6px;
        padding-bottom: 6px;
        background-color: transparent;
    }
    QMenuBar::item:last {
        border-right: none;
    }

/* --- NEU: Wenn man mit der Maus drüberfährt (Hover) --- */

```

```

        QMenuBar::item:selected {
            background-color: #e2e8f0;
            color: #000000;
        }

        /* --- NEU: Wenn das Hauptmenü angeklickt wurde / offen ist (Aktiv-
Zustand) --- */
        QMenuBar::item:pressed {
            background-color: #ffffff; /* Reines Weiß wie die aktiven
Registerkarten */
            color: #1a73e8;           /* Clio-Tech-Blau für die aktive
Schrift */

            font-weight: bold;
            border-top-left-radius: 4px;
            border-top-right-radius: 4px;
            border-bottom: 1px solid #ffffff; /* Verschmilzt visuell nach
unten */
        }
    """

# =====
# CORE INJEKTION: UI STATE MANAGER FÜR UNDO / REDO ACTIVATION
# =====
from infrastructure.ui_state_manager import UIStateManager
self._services["ui_state_manager"] = UIStateManager(self._i18n)

# Triggert nun den UI-Aufbau mit der korrekten Register-Injektion im RAM
self.setup_ui()

def setup_ui(self):
    """Baut Menüs, die linke Designer-Sidebar und Tabs strikt nach Schema
auf."""
    import infrastructure.cfg as cfg
    self.menuBar().clear()

    # 1. Obere Menüs aus der Konfiguration generieren
    self._build_menu_recursive(self.menuBar(),
cfg.UI_SCHEMA.get("menu_structure", {}))

    # Excel-Style Plus-Schaltfeld ans Ende hängen (sofern Obergrenze nicht
erreicht)
    current_menu_count = len(cfg.UI_SCHEMA.get("menu_structure", {}).keys())
    if current_menu_count < cfg.APP_CONFIG.get("max_main_menus", 10):
        plus_action = self.menuBar().addAction("+")
        plus_action.setToolTip("Neues Hauptmenü hinzufügen")
        plus_action.triggered.connect(self.create_new_main_menu)

    # 2. Den Designer als Modul in der neuen Icon-Sidebar registrieren (Nur
einmalig)
    if not self.sidebar_manager.sidebar_mapping:
        from .designer import MenuDesigner
        from PyQt6.QtWidgets import QWidget

```

```

# --- KNOPF 1: SYSTEMDESIGNER (FÜR FORSCHER) ---
designer_container = QWidget()
MenuDesigner.inject_designer_sidebar_at(self, designer_container)

self.sidebar_manager.register_sidebar(
    sidebar_id="system_designer",
    icon_text="⚙️",
    widget=designer_container,
    tooltip="System-Designer (Menüs & Register)"
)

# --- KNOPF 2: ENTWICKLERTOOLS (FÜR ENTWICKLER / HYBRID-IDE) ---
from .ide_designer import IDEDesigner

self._ide_container = QWidget() # Sicher als Instanz-Variable
verankert
    IDEDesigner.inject_ide_sidebar_at(self, self._ide_container)

dev_tooltip = self._i18n.text("sidebar.btn.developer_tool") or
"Entwicklertools (VBA & GenAI)"

self.sidebar_manager.register_sidebar(
    sidebar_id="developer_tools",
    icon_text="🧰", # Hier ist das korrekte Koffer-Icon gesetzt!
    widget=self._ide_container,
    tooltip=dev_tooltip
)

# Standardmäßig zugeklappt starten
self.sidebar_manager.content_stack.hide()
self._main_splitter.setSizes([46, self.width() - 46])

# =====
# 3. KERN-REPARATUR: DELEGIERT DAS INTEGRATIVE RENDERN VOLLSTÄNDIG
# =====
# Wir überlassen das saubere Zeichnen komplett der spezialisierten
Methode!
# Das verhindert die doppelte Initiierung des Willkommens-Registers.
self._build_tabs(cfg.UI_SCHEMA.get("tab_structure", []))

# Falls noch kein aktives Menü gesetzt ist, initialisiere es mit dem
ersten dynamischen Eintrag
# --- SICHERHEITSANKER: Gedächtnis mit der exakten ersten Menü-ID
bespielen ---
if not hasattr(self, "_active_menu_uuid") or not self._active_menu_uuid:
    import infrastructure.cfg as cfg
    order = cfg.UI_SCHEMA.get("menu_order", [])
    dynamic_menus = [k for k in order if k != "file"]
    if dynamic_menus:
        # Wir nehmen die echte, rohe ID-Zeichenkette (z.B. "menu_xyz")
        self._active_menu_uuid = dynamic_menus[0]

```

```

# Erzwingt das sofortige Übersetzen aller registrierten UI-Elemente
self.refresh_ui_texts()

def _on_tab_changed_check_plus(self, index):
    """Prüft, ob das allerletzte Tab (das '+' Feld) angeklickt wurde, und
    erzeugt JIT ein neues Register."""
    # 1. SICHERHEITSANKER: Wenn der Index ungültig oder im Minus ist, sofort
    abbrechen
    if index < 0 or index >= self._tabs.count():
        return

    # 2. URSACHEN-KILLER: Beim allerersten Start (Index 0, Willkommen) darf
    NIEMALS der Dialog kommen!
    if index == 0:
        return

    # 3. TEXT-CHECK: Nur wenn das angeklickte Feld wirklich die Plus-
    Schaltfläche ist
    tab_text = self._tabs.tabText(index).strip()
    if tab_text == "+":
        from .designer import MenuDesigner

        # Signale kurz stummschalten, um Endlosschleifen beim Einfügen zu
        unterbinden
        self._tabs.blockSignals(True)

        # Ruft DEINE NEUE sichere Logik im MenuDesigner auf (schreibt direkt
        in de.json/en.json via UUID)
        MenuDesigner._live_add_new_tab(self)

        # Signale sofort wieder freigeben
        self._tabs.blockSignals(False)

def toggle_sidebar(self):
    """Kollabiert oder expandiert die Sidebar im Splitter, schaltet
    Sichtbarkeiten um."""
    if not self._sidebar_is_collapsed:
        if self._designer_content: self._designer_content.hide()
        if self._sidebar_title_label: self._sidebar_title_label.hide()
        self._sidebar_container.setFixedWidth(34)
        self._main_splitter.handle(1).setEnabled(False)
        if self._btn_toggle_sidebar: self._btn_toggle_sidebar.setText("►")
        self._sidebar_is_collapsed = True
    else:
        self._sidebar_container.setMinimumWidth(220)
        self._sidebar_container.setMaximumWidth(16777215)
        self._main_splitter.handle(1).setEnabled(True)
        self._main_splitter.setSizes([220, self.width() - 220])
        if self._designer_content: self._designer_content.show()
        if self._sidebar_title_label: self._sidebar_title_label.show()

```

```

        if self._btn_toggle_sidebar: self._btn_toggle_sidebar.setText("◀")
        self._sidebar_is_collapsed = False

def _build_menu_recursive(self, parent_menu_component, menu_schema: dict):
    """Erlaubt geordnete Generierung der Hauptmenüs nach menu_order."""
    import infrastructure.cfg as cfg

    # Wenn wir auf oberster Ebene (MenuBar) sind, nutzen wir unsere menu_order
    Liste!
    if isinstance(parent_menu_component, type(self.menuBar())):
        order = cfg.UI_SCHEMA.get("menu_order", [])
        if not order:
            order = list(menu_schema.keys())
            cfg.UI_SCHEMA["menu_order"] = order

        # Synchronisation: Neue Menüs hinten anhängen
        for m_key in menu_schema.keys():
            if m_key not in order:
                order.append(m_key)

        for key in order:
            if key not in menu_schema:
                continue
            menu_data = menu_schema[key]
            localized_title = self._i18n.text(menu_data.get("i18n_key",
f"menu.{key}"))

            # Menü erzeugen
            current_menu = parent_menu_component.addMenu(localized_title)

            # --- DIE RETTUNG: Sobald das Menü angeklickt wird, im Gedächtnis
            verankern! ---
            current_menu.aboutToShow.connect(lambda k=key: setattr(self,
"_active_menu_uuid", k))

            # --- KORREKTUR: Diese Schleife MUSS innerhalb der 'key'-Schleife
            eingerückt sein! ---
            for item in menu_data.get("items", []):
                # SCAN-SONDE: Druckt jedes geladene Item ins Terminal
                print(f"[I18N-DEBUG-SCAN] Item-ID: {item.get('id')} | Command:
{item.get('command_class')}")

                if item["type"] == "separator":
                    current_menu.addSeparator()
                elif item["type"] == "action":
                    action_title = self._i18n.text(item["id"])
                    action = current_menu.addAction(action_title)

                    cmd_class = item.get("command_class")
                    if cmd_class == "ChangeLanguageCommand":
                        action.triggered.connect(lambda checked:
self._change_language_dialog())
                    elif cmd_class == "ExitCommand":
                        action.triggered.connect(lambda checked: self.close())

```

```

        elif cmd_class == "ResetSystemCommand":
            action.triggered.connect(lambda checked:
self._execute_system_reset_direct())
        elif cmd_class:
            action.triggered.connect(lambda checked, c=cmd_class:
self._dispatch_custom_user_command(c))
        elif item["type"] == "submenu":
            sub_item = item.copy()
            sub_item["i18n_key"] = item.get("id")
            sub_schema = { item.get("id", "sub"): sub_item }
            self._build_menu_recursive(current_menu, sub_schema)

... [HINWEIS: Datei wurde hier nach 300 Zeilen gekürzt] ...

```

File: `infrastructure\__init__.py`

File: `infrastructure\cfg.py`

```

# Datei: infrastructure/cfg.py
"""Infrastructure Layer: Globale Konfiguration und Übersetzungs-Wörterbuch (I18N)
für ClioGraph"""

import os
import sys
from pathlib import Path
import importlib
import pkgutil
import inspect

# Fügt das Hauptverzeichnis zum Suchpfad von Python hinzu
sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))
from business.commands.cmd_base import UICommand

APP_NAME = "ClioGraph"

# UI-Format für den Fehlerdienst
ERROR_UI_FORMAT = "{prefix} ? Details: {message}"

def get_user_data_dir() -> Path:
    """Ermittelt den offiziellen, beschreibbaren Anwendungsordner des
Betriebssystems."""
    if sys.platform == "win32":
        # Windows: C:\Users\Name\AppData\Roaming\ClioGraph
        base_dir = Path(os.environ.get("APPDATA", Path.home() /
"AppData/Roaming"))
    elif sys.platform == "darwin":

```



```

        # macOS: /Users/Name/Library/Application Support/ClioGraph
        base_dir = Path.home() / "Library/Application Support"
    else:
        # Linux: /home/Name/.config/cliograph
        base_dir = Path.home() / ".config"

    data_dir = base_dir / APP_NAME
    # Erstellt den Ordner automatisch, falls er noch nicht existiert
    data_dir.mkdir(parents=True, exist_ok=True)
    return data_dir

# Zentrale, dynamische Pfade definieren
USER_DATA_PATH = get_user_data_dir()
DB_PATH = str(USER_DATA_PATH / "cliograph_db")
LOG_FILE_PATH = str(USER_DATA_PATH / "cliograph_system.log")

# Ermittelt dynamisch das Hauptverzeichnis deines Projekts (ClioGraph/)
PROJECT_ROOT = Path(__file__).resolve().parent.parent

# ZENTRALE PFAD-DEFINITION FÜR ALLERLEI JSON-DATEIEN (Egal auf welchem Laufwerk
oder PC!)
JSON_STORAGE_DIR = str(PROJECT_ROOT / "json_storage")
JSON_MENU_FILE = os.path.join(JSON_STORAGE_DIR, "dynamic_menu.json")

# =====
# DYNAMISCHER ORDNER FÜR DIE SPRACH-DATEIEN DER FORSCHER
# =====
JSON_LOCALES_DIR = os.path.join(JSON_STORAGE_DIR, "locales")

# Sicherheits-Check: Falls der Ordner jemals fehlen sollte, erstellt Python ihn
automatisch
if not os.path.exists(JSON_LOCALES_DIR):
    os.makedirs(JSON_LOCALES_DIR, exist_ok=True)

# Sicherheits-Check: Falls der Ordner im Projektverzeichnis fehlt, erstellt Python
ihn automatisch
if not os.path.exists(JSON_STORAGE_DIR):
    os.makedirs(JSON_STORAGE_DIR, exist_ok=True)

# 1. UNVERÄNDERLICHE MASTER-SPRACHEN (Systemschutz vor Löschung)
MASTER_LANGUAGES = ["de", "en"]

# =====
# GLOBALER EXLOSIVER EDITOR-BEFEHL (Kapselung für die Community)
# =====
EXTERNAL_EDITOR_COMMAND = "code"

# =====
# GLOBALE ANWENDUNGSEINSTELLUNGEN (MINIMALES AUSGANGSGERÜST)
# =====
APP_CONFIG = {
    "version": "1.0.0",
    "default_language": "de",

```

```

    "supported_languages": ["de", "en"],
    "default_model": "llama3:latest",
    "max_main_menus": 8,          # Reduziert von 10 auf exakt 8 Hauptmenüs (inkl.
Datei)
    "max_tabs": 10,              # Begrenzung auf maximal 10 geöffnete Registerkarten
}

# 2. AKTIVE ANZEIGELUSTE FÜR DIE GUI (Wird später im Betrieb erweitert)
AVAILABLE_LANGUAGES = [
    {"name": "Deutsch", "code": "de"},
    {"name": "English", "code": "en"}
]

# =====
# ZENTRALES WÖRTERBUCH FÜR DIE INTERNATIONALISIERUNG (I18N)
# =====
I18N = {
    # --- Visual Studio Style Start-Texte (GUI-BasisFramework) ---
    "tabs.welcome.title": {
        "de": "Willkommen",
        "en": "Welcome"
    },
    "dialog.welcome.headline": {
        "de": "Exemplarische Vorgehensweisen",
        "en": "Exemplary Procedures"
    },
    # --- Allgemeine Applikations-Texte ---
    "app.title": {
        "de": "ClioGraph — Historische Graphen-Analyse & Graph-RAG",
        "en": "ClioGraph — Historical Graph Analytics & Graph-RAG"
    },
    "app.status.ready": {
        "de": "System-Infrastruktur geladen. Bereit.",
        "en": "System infrastructure loaded. Ready."
    },
    "menu.language": {
        "de": "Sprache",
        "en": "Language"
    },
    # --- Das geschützte Basis-Menü ---
    "menu.file": {
        "de": "Datei",
        "en": "File"
    },
    "menu.file.language": {
        "de": "Sprache wechseln",
        "en": "Change Language"
    },
    "menu.file.exit": {

```

```

        "de": "Beenden",
        "en": "Exit"
    },

    "menu.file.reset_system": {
        "de": "System-Infrastruktur zurücksetzen",
        "en": "Reset System Infrastructure"
    },
    "dialog.reset.title": {
        "de": "System zurücksetzen",
        "en": "Reset System"
    },
    "dialog.reset.text": {
        "de": "Möchten Sie das System wirklich auf die Werkseinstellungen
zurücksetzen?\nAlle dynamischen Menüs und hinzugefügten Sprachen werden
archiviert.",
        "en": "Do you really want to reset the system to default settings?\nAll
dynamic menus and added languages will be archived."
    },

    "menu.file.export": {
        "de": "Projekt exportieren",
        "en": "Export Project"
    },

# --- Die 8 Hauptmenü-Übersetzungen (Vorbereitung für Forscher) ---
"menu.sources": { "de": "Quellenverwaltung", "en": "Source Management" },
"menu.database": { "de": "Datenbank", "en": "Database" },
"menu.apis": { "de": "APIs", "en": "APIs" },
"menu.register": { "de": "Register", "en": "Tabs" },
"menu.visualization": { "de": "Visualisierung", "en": "Visualization" },
"menu.community": { "de": "Community", "en": "Community" },
"menu.help": { "de": "Hilfe", "en": "Help" },

# Tab/Register-Einträge

# --- System-Designer Tab-Steuerung ---
"sidebar.tab.move_left": {
    "de": "Nach links",
    "en": "Move Left"
},
"sidebar.tab.move_right": {
    "de": "Nach rechts",
    "en": "Move Right"
},

# --- Die 4 neuen, strukturierten Fach-Registerkarten (Zweisprachig) ---
"tabs.extraction.title": {"de": "🚀 Wissensextraktion", "en": "🚀 Knowledge
Extraction"},
"tabs.network.title": {"de": "🌐 Netzwerkanalyse", "en": "🌐 Network
Analysis"},
"tabs.graphrag.title": {"de": "🧠 GraphRAG & LLM", "en": "🧠 GraphRAG &
LLM"},
"tabs.ml.title": {"de": "📊 Machine Learning", "en": "📊 Machine Learning"},

```

```

# --- Formularfelder & Platzhalter (Eingaben und Viewer) ---
"fields.source_placeholder": {"de": "Fügen Sie hier den historischen
Quellentext ein...", "en": "Paste the historical source text here..."},
"fields.output_placeholder": {"de": "Hier erscheint das extrahierte
Wissensnetzwerk als JSON...", "en": "The extracted knowledge network will appear
here as JSON..."},

"fields.network_filter_placeholder": {"de": "Knoten- oder Kantenfilter (z.B.
Person, Ort)...", "en": "Node or edge filter (e.g., Person, Location)..."},
"fields.network_log_placeholder": {"de": "Netzwerkanalyse-Metriken
(Zentralitätswerte, Cluster)...", "en": "Network analysis metrics (centrality
values, clusters)..."},

"fields.graphrag_query_placeholder": {"de": "Stellen Sie eine historische
Frage an Ihren Wissensgraphen...", "en": "Ask a historical question to your
knowledge graph..."},
"fields.graphrag_answer_placeholder": {"de": "Die synthetisierte GraphRAG-
Antwort mit Quellenbelegen...", "en": "The synthesized GraphRAG answer with source
citations..."},

"fields.ml_features_placeholder": {"de": "Eingabe-Vektor für Klassifikation
oder Mustererkennung...", "en": "Input vector for classification or pattern
recognition..."},

# --- Interaktions-Schaltflächen (Buttons) ---
"buttons.start_extraction": {"de": "Extraktion starten", "en": "Start
Extraction"},
"buttons.render_network": {"de": "Netzwerk generieren", "en": "Render
Network"},
"buttons.query_graph": {"de": "Wissensgraph abfragen", "en": "Query Knowledge
Graph"},
"buttons.train_model": {"de": "Modell trainieren", "en": "Train Model"},

# --- Standard-Platzhalter für leere Befehle ---
"dummy.test_feature": {
  "de": "Dynamisches Test-Feature",
  "en": "Dynamic Test Feature"
},

# --- Sidebar Haupt-Buttons ---
"sidebar.btn.system_designer": {
  "de": "⚙️ System-Designer",
  "en": "⚙️ System Designer"
},
"sidebar.btn.developer_tools": {
  "de": "🔧 Entwicklertools",
  "en": "🔧 Developer Tools"
},

# --- IDE-Formularbaukasten ---
"ide.headline.vba": {
  "de": "📄 Formular-Steuerelemente",
  "en": "📄 Form Controls"
}

```

```

    },
    "ide.label.widget_type": {
        "de": "Objekt-Typ auswählen:",
        "en": "Select Object Type:"
    },
    "ide.label.element_id": {
        "de": "Technische ID (eindeutig):",
        "en": "Technical ID (unique):"
    },
    },
    "ide.btn.insert_widget": {
        "de": "Baustein auf Tab platzieren",
        "en": "Insert Control on Tab"
    },
    },

# --- Generischer Open-Source GenAI Formular-BUILDER ---
"ide.headline.genai": {
    "de": "🧬 GenAI Formular-BUILDER",
    "en": "🧬 GenAI Formular Builder"
},
"ide.placeholder.prompt": {
    "de": "Beschreiben Sie die gewünschten Felder (z.B. Name, Adresse,
Button)...",
    "en": "Describe the desired fields (e.g., name, address, button)..."
},
"ide.btn.generate": {
    "de": "Layout generieren",
    "en": "Generate Layout"
},
},

# --- Zentraler Fehlerdienst (errors.py) ---
"error.validation": { "de": "[Validierung]", "en": "[Validation]" },
"error.graph_data": { "de": "[Daten-Fehler]", "en": "[Data Error]" },
"error.database": { "de": "[Datenbank-Fehler]", "en": "[Database Error]" },
"error.llm": { "de": "[KI-Dienst-Fehler]", "en": "[AI Service Error]" },
"error.generic": { "de": "[Anwendungs-Fehler]", "en": "[Application Error]" },
"error.unknown": { "de": "[Kritischer Systemfehler]", "en": "[Critical System
Error]" }
}

def discover_all_commands(filter_prefix: str = None) -> dict:
    """Sucht alle Makro-Klassen im commands-Ordner."""
    pool = {}
    import business.commands as cmd_pkg

    for _, mod_name, is_pkg in pkgutil.iter_modules(cmd_pkg.__path__):
        if is_pkg or mod_name == "cmd_base":
            continue

        if filter_prefix:
            allowed_modules = [f"cmd_{filter_prefix}", "cmd_dummy"]
            if mod_name not in allowed_modules:
                continue

```

```

    mod = importlib.import_module(f"business.commands.{mod_name}")

    for cls_name, cls in inspect.getmembers(mod, inspect.isclass):
        if issubclass(cls, UICommand) and cls is not UICommand:
            display_name = f"[{mod_name}.py] -> {cls_name}"
            pool[display_name] = cls()

    return pool

# =====
# DAS GENERISCHE GUI-SCHEMA FÜR CLIOPGRAPH (AUSGANGSGERÜST)
# =====

# =====
# DAS GENERISCHE GUI-SCHEMA FÜR CLIOPGRAPH (REPARIERTES AUSGANGSGERÜST)
# =====
# In infrastructure/cfg.py die Struktur anpassen:

# Datei: infrastructure/cfg.py
# Vollständiges Master-Schema für die hybride IDE-Struktur

# Datei: infrastructure/cfg.py -> Ganz unten ersetzen

UI_SCHEMA = {
    "menu_structure": {
        "file": {
            "i18n_key": "menu.file",
            "items": [
                {"type": "action", "id": "menu.file.language", "command_class":
"ChangeLanguageCommand"},
                {"type": "action", "id": "menu.file.ollama", "command_class":
"OllamaInfoCommand"},
                {"type": "action", "id": "menu.file.validate", "command_class":
"OllamaValidateCommand"},
                {"type": "action", "id": "menu.file.edit_locales",
"command_class": "LanguageEditorCommand"},
            ]
        }
    }
}

... [HINWEIS: Datei wurde hier nach 300 Zeilen gekürzt] ...

```

File: `infrastructure\errors.py`

```

# Datei: infrastructure/errors.py
"""Infrastructure/Utility Layer: Gekapselter und unabhängiger Fehler-Service"""
import logging
import infrastructure.cfg as cfg
from infrastructure.exceptions import (
    ClioGraphError, DatabaseConnectionError,
    LlmTimeoutError, GraphValidationError
)

# Einfaches Logging in eine Datei aktivieren

```

```

logging.basicConfig(
    filename=cfg.LOG_FILE_PATH,
    level=logging.ERROR,
    format="%(asctime)s [%(levelname)s] %(message)s"
)

class ErrorHandlingService:
    def __init__(self, i18n_service):
        self._i18n = i18n_service
        # Zuordnung von Ausnahmen zu ihren Übersetzungs-Schlüsseln oder Präfixen
        self._error_map = {
            ValueError: "error.validation",
            GraphValidationError: "error.graph_data",
            DatabaseConnectionError: "error.database",
            LlmTimeoutError: "error.llm",
            ClioGraphError: "error.generic"
        }

    def handle_exception(self, exception: Exception) -> str:
        """Loggt den Fehler und gibt eine saubere, lokalisierte Nachricht
        zurück."""
        # 1. Fehler im Hintergrund protokollieren
        logging.error(f"Ausnahme abgefangen: {str(exception)}", exc_info=True)

        # 2. Passenden Übersetzungs-Präfix ermitteln
        for error_cls, cfg_key in self._error_map.items():
            if isinstance(exception, error_cls):
                # Nutzen des bestehenden i18n-Service für die Übersetzung
                prefix = self._i18n.text(cfg_key)
                return cfg.ERROR_UI_FORMAT.format(prefix=prefix,
                message=str(exception))

        # Fallback für unvorhergesehene Systemfehler
        fallback_msg = self._i18n.text("error.unknown")
        return cfg.ERROR_UI_FORMAT.format(prefix=fallback_msg,
        message=str(exception))

```

File: `infrastructure\exceptions.py`

```

# Datei: exceptions.py
"""Domain Layer: Spezifische Ausnahme-Klassen für das ClioGraph-System"""

class ClioGraphError(Exception):
    """Basis-Ausnahme für das gesamte ClioGraph-System.
    Alle spezifischen Fehler erben von dieser Klasse.
    """
    pass

class DatabaseConnectionError(ClioGraphError):
    """Wird geworfen, wenn die Kùzu-Datenbank nicht erreichbar ist

```

```

        oder eine Graph-Abfrage fehlschlägt.
        """

    pass

class LlmTimeoutError(ClioGraphError):
    """Wird geworfen, wenn das lokale Ollama-Modell (z.B. llama3:science)
    nicht antwortet oder die API blockiert ist.
    """

    pass

class GraphValidationError(ClioGraphError):
    """Wird geworfen, wenn die vom LLM gelieferten JSON-Daten unvollständig
    sind oder Beziehungen (Kanten) ohne gültige Knoten erzeugt wurden.
    """

    pass

```

File: `infrastructure\i18n.py`

```

# -*- coding: utf-8 -*-
"""
Infrastructure/Utility Layer: Kapselung des JSON-basierten Übersetzungs-Services
mit robustem Code-Fallback
Erweitert um die neuen Übersetzungen für den System-Designer (Sortierung und
System-Schutz).
"""

import os
import json
import infrastructure.cfg as cfg

class TranslationService:
    def __init__(self, default_lang: str = "de"):
        self._registered_widgets = {}
        self._current_lang = default_lang
        self._translations = {} # Hier landen die geladenen JSON-Inhalte

        # Feste Standard-Übersetzungen als Sicherheitsnetz (Damit die GUI niemals
        # leer bleibt!)
        self._fallback_translations = {
            "de": {
                "app.title": "ClioGraph — Historische Graphen-Analyse & Graph-
RAG",
                "menu.file": "Datei",
                "menu.file.language": "Sprache wechseln",
                "menu.file.exit": "Beenden",
                "mod1.title": "Wissens-Extraktion",
                # --- NEU: Übersetzungen für die Sicherheits-Knöpfe ---
                "sidebar.safety.undo": "Rückgängig",
                "sidebar.safety.redo": "Wiederholen",
            }
        }

```



```

# --- System-Designer (Hauptmenü) ---
"sidebar.designer.title": "⚙️ SYSTEM-DESIGNER",
"sidebar.menu.group": "⚙️ HAUPTMENÜ",
"sidebar.menu.add": "+ Neues Hauptmenü",
"sidebar.menu.delete": "- Hauptmenü löschen",
"sidebar.menu.rename": "✏️ Hauptmenü umbenennen",
"sidebar.menu.move_left": "◀ Nach links",
"sidebar.menu.move_right": "▶ Nach rechts",

# --- System-Designer (Schaltflächen / Items) ---
"sidebar.item.group": "📄 MENÜEINTRÄGE",
"sidebar.item.add": "+ Neues Item",
"sidebar.item.delete": "- Item löschen",
"sidebar.item.rename": "✏️ Item umbenennen",
"sidebar.item.assign": "⚡ Befehl zuordnen",
"sidebar.item.move_up": "▲ Nach oben",
"sidebar.item.move_down": "▼ Nach unten",

# --- System-Designer (Registerkarten / Tabs) ---
"sidebar.tab.group": "📁 REGISTERKARTEN",
"sidebar.tab.add": "+ Neues Register",
"sidebar.tab.delete": "- Register löschen",
"sidebar.tab.rename": "✏️ Register umbenennen",
"sidebar.tab.move_left": "◀ Links verschieben",
"sidebar.tab.move_right": "▶ Rechts verschieben",

# --- System-Designer (Schutz- und Warnmeldungen) ---
"msg.protection.title": "System-Schutzfunktion",
"msg.protection.core_deny": "Dieses Element gehört zum geschützten
Systemkern und darf weder gelöscht, verschoben noch umbenannt werden!"
},
"en": {
  "app.title": "ClioGraph — Historical Graph Analytics & Graph-
RAG",

  "menu.file": "File",
  "menu.file.language": "Change Language",
  "menu.file.exit": "Exit",
  "mod1.title": "Knowledge Extraction",

# --- NEU: Übersetzungen für die Sicherheits-Knöpfe ---
"sidebar.safety.undo": " Undo",
"sidebar.safety.redo": " Redo",

# --- System Designer (Main Menu) ---
"sidebar.designer.title": "⚙️ SYSTEM DESIGNER",
"sidebar.menu.group": "⚙️ MAIN MENU",
"sidebar.menu.add": "+ New Main Menu",
"sidebar.menu.delete": "- Delete Main Menu",
"sidebar.menu.rename": "✏️ Rename Main Menu",
"sidebar.menu.move_left": "◀ Move Left",
"sidebar.menu.move_right": "▶ Move Right",

# --- System Designer (Buttons / Items) ---
"sidebar.item.group": "📄 MENU ITEMS",

```

```

        "sidebar.item.add": "+ New Item",
        "sidebar.item.delete": "- Delete Item",
        "sidebar.item.rename": "✎ Rename Item",
        "sidebar.item.assign": "⚡ Assign Command",
        "sidebar.item.move_up": "▲ Move Up",
        "sidebar.item.move_down": "▼ Move Down",

        # --- System Designer (Tabs) ---
        "sidebar.tab.group": "📁 TABS",
        "sidebar.tab.add": "+ New Tab",
        "sidebar.tab.delete": "- Delete Tab",
        "sidebar.tab.rename": "✎ Rename Tab",
        "sidebar.tab.move_left": "◀ Move Left",
        "sidebar.tab.move_right": "▶ Move Right",

        # --- System Designer (Protection Messages) ---
        "msg.protection.title": "System Protection",
        "msg.protection.core_deny": "This element belongs to the protected
system core and cannot be deleted, moved, or renamed!"
    }
}

self._locales_dir = cfg.JSON_LOCALES_DIR
self._load_all_locales()

def _load_all_locales(self):
    """Lädt alle verfügbaren JSON-Sprachdateien dynamisch aus dem
json_storage."""
    if not os.path.exists(self._locales_dir):
        os.makedirs(self._locales_dir, exist_ok=True)

    for lang_code in cfg.APP_CONFIG.get("supported_languages", ["de", "en"]):
        file_path = os.path.join(self._locales_dir, f"{lang_code}.json")

        # Starten mit den eingebauten Code-Fallbacks
        self._translations[lang_code] =
dict(self._fallback_translations.get(lang_code, {}))

        # Wenn eine Datei auf der Festplatte existiert, überschreiben wir die
Werte damit
        if os.path.exists(file_path):
            try:
                with open(file_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
                    if file_data:
                        self._translations[lang_code].update(file_data)
                        print(f"📁 [I18N] Sprachdatei '{lang_code}.json' erfolgreich
geladen und synchronisiert.")
            except Exception as e:
                print(f"❌ [I18N-FEHLER] Konnte {lang_code}.json nicht lesen:
{e}")

def register(self, widget, widget_type: str, i18n_key: str):
    """Registriert ein GUI-Widget für den dynamischen Sprachwechsel."""

```

```

        self._registered_widgets[widget] = (widget_type, i18n_key)

    def change_language(self, lang_code: str):
        """Wechselt die Sprache global im Service und aktualisiert alle
        Komponenten."""
        if lang_code in cfg.APP_CONFIG.get("supported_languages", ["de", "en"]):
            self._current_lang = lang_code
            self.translate_all()

    def text(self, key: str) -> str:
        """Holt den übersetzten Text für einen technischen Schlüssel aus den JSON-
        Daten."""
        lang_dict = self._translations.get(self._current_lang, {})
        if key in lang_dict:
            return lang_dict[key]

        default_dict = self._translations.get(cfg.APP_CONFIG["default_language"],
        {}))
        if key in default_dict:
            return default_dict[key]

        return key

    def translate_all(self):
        """Übersetzt alle dynamisch angemeldeten GUI-Widgets zur Laufzeit."""
        for widget, (w_type, key) in self._registered_widgets.items():
            try:
                translated_text = self.text(key)

                if w_type == "headline":
                    widget.setText(f"<h3>{translated_text}</h3>")

                elif w_type == "placeholder":
                    # Deckt ab (laut Grafik): 'Kombinationsfeld', 'Eingabe-Zeile',
                    'Text-Feld', 'Text-Anzeige'
                    widget.setPlaceholderText(translated_text)

                elif w_type in ["button", "label", "checkbox", "radiobutton"]:
                    # Deckt ab (laut Grafik): 'Schaltfläche', 'Bezeichnung',
                    'Kontrollkästchen', 'Optionsfeld'
                    widget.setText(translated_text)

                elif w_type == "groupbox":
                    # Deckt ab (laut Grafik): 'Gruppenfeld' (benötigt .setTitle)
                    widget.setTitle(translated_text)

                elif w_type == "welcome_html":
                    # Verhindert ein Überschreiben des Forscher-Eingabetextes und
                    lädt reines HTML
                    widget.setHtml(translated_text)
            except Exception:
                pass

    def update_or_append_key(self, key: str, de_text: str, en_text: str):

```

```

        """Fügt zur Laufzeit einen neuen GUI-Schlüssel direkt in die physischen
        JSON-Dateien ein."""
        if "de" not in self._translations: self._translations["de"] = {}
        if "en" not in self._translations: self._translations["en"] = {}

        self._translations["de"][key] = de_text
        self._translations["en"][key] = en_text

        de_file = os.path.join(self._locales_dir, "de.json")
        try:
            with open(de_file, "w", encoding="utf-8") as f:
                json.dump(self._translations["de"], f, indent=4,
ensure_ascii=False)
        except Exception as e:
            print(f"✗ Fehler beim Schreiben in de.json: {e}")

        en_file = os.path.join(self._locales_dir, "en.json")
        try:
            with open(en_file, "w", encoding="utf-8") as f:
                json.dump(self._translations["en"], f, indent=4,
ensure_ascii=False)
        except Exception as e:
            print(f"✗ Fehler beim Schreiben in en.json: {e}")

```

File: `infrastructure\menu_storage.py`

```

# Datei: infrastructure/menu_storage.py
import json
import os
import infrastructure.cfg as cfg

class MenuPersistenceManager:
    """
    Kapselt das Speichern und Laden der dynamischen Menüpunkte in einer JSON-
    Datei.
    Greift strikt auf die zentralen Pfad-Definitionen in der cfg.py zu.
    """

    def __init__(self, discover_func):
        self._discover_func = discover_func

        # --- ABSOLUTE TRENNUNG: KEINE HART CODIERTEN PFADBERECHNUNGEN ---
        # Wir lesen die Pfade direkt aus der zentralen Konfiguration aus
        self._json_folder = cfg.JSON_STORAGE_DIR
        self._storage_file = cfg.JSON_MENU_FILE

        # Robustheits-Check: Falls der Ordner physisch fehlt, wird er angelegt
        if not os.path.exists(self._json_folder):
            os.makedirs(self._json_folder)
            print(f"[PERSISTENZ] Zentraler Ordner angelegt: {self._json_folder}")

```

```

def save_menus(self, dynamic_menu_store: dict):
    """Speichert das komplette UI_SCHEMA-Menüformat direkt und sauber ab."""
    try:
        # ERWEITERT: Jetzt wird auch die tab_structure dauerhaft gesichert!
        data_to_save = {
            "menu_structure": dynamic_menu_store.get("menu_structure", {}),
            "menu_order": dynamic_menu_store.get("menu_order", []),
            "tab_structure": dynamic_menu_store.get("tab_structure", []),
            "supported_languages":
dynamic_menu_store.get("supported_languages", ["de", "en"]),
            "available_languages":
dynamic_menu_store.get("available_languages", [])
        }

        with open(self._storage_file, "w", encoding="utf-8") as f:
            json.dump(data_to_save, f, indent=4, ensure_ascii=False)
            print("✓ [PERSISTENZ] Menüstruktur, Reihenfolge und Registerkarten im
UI_SCHEMA-Format gesichert.")
        except Exception as e:
            print(f"✗ [PERSISTENZ-FEHLER] Konnte Menüs nicht sichern: {e}")

def load_menus(self) -> dict:
    """Liest die JSON-Datei aus und gibt die rohe menu_structure zurück."""
    if not os.path.exists(self._storage_file):
        # KORREKTUR: Wenn keine Datei da ist, nehmen wir das frische Kern-
        Schema aus der cfg.py!
        return {"menu_structure": cfg.UI_SCHEMA["menu_structure"]}
    try:
        with open(self._storage_file, "r", encoding="utf-8") as f:
            saved_data = json.load(f)
            if saved_data and "menu_structure" in saved_data:
                print("📁 [PERSISTENZ] Menüstruktur erfolgreich geladen.")
                return saved_data["menu_structure"]
    except Exception as e:
        print(f"✗ [PERSISTENZ-FEHLER] Fehler beim Laden: {e}")
    return {"menu_structure": cfg.UI_SCHEMA["menu_structure"]}

```

File: infrastructure\ui\_state\_manager.py

```

# -*- coding: utf-8 -*-
"""
ClioGraph - UI State Manager (Synchronized Undo / Redo)
Verwaltet Snapshots des UI-Schemas UND der physischen JSON-Sprachdateien.
Garantiert, dass Sprach-IDs und GUI-Elemente immer synchron zurückgesetzt werden.
"""

import copy
import os
import json

```

```

import infrastructure.cfg as cfg

class UIStateManager:
    def __init__(self, translation_service=None):
        self._undo_stack = []
        self._redo_stack = []
        self._i18n = translation_service
        self.save_checkpoint()

    def save_checkpoint(self):
        """Schiesst ein synchronisiertes Backup-Foto von Schema und
        Übersetzungen."""
        snapshot = {
            "ui_schema": copy.deepcopy(cfg.UI_SCHEMA),
            "translations": {}
        }

        # Falls der Übersetzungsservice bereitsteht, sichern wir den aktuellen
        RAM-Zustand der Sprachen
        if self._i18n and hasattr(self._i18n, "_translations"):
            snapshot["translations"] = copy.deepcopy(self._i18n._translations)

        self._undo_stack.append(snapshot)
        self._redo_stack.clear()

    def undo_action(self):
        """Geht exakt einen Schritt im System-Designer zurück (Sichert den Live-
        Zustand)."""
        if self._undo_stack:
            # 1. Sichere den AKTUELLEN Live-Zustand vom Bildschirm für das Redo
            current_live = {
                "ui_schema": copy.deepcopy(cfg.UI_SCHEMA),
                "translations": copy.deepcopy(self._i18n._translations) if
self._i18n else {}
            }
            self._redo_stack.append(current_live)

            # 2. Hole den exakt vorherigen Zustand vom Undo-Stapel
            previous = self._undo_stack.pop()

            # 3. Überschreibe das Schema ohne Referenzverlust
            cfg.UI_SCHEMA.clear()
            cfg.UI_SCHEMA.update(copy.deepcopy(previous["ui_schema"]))

            # Synchronisiere Sprachdateien
            if self._i18n and previous["translations"]:
                self._i18n._translations = copy.deepcopy(previous["translations"])
                self._write_translations_to_disk()

            return True
        return False

    def redo_action(self):
        """Wiederholt exakt einen Layout-Schritt (Sichert den Live-Zustand)."""

```

```

        if self._redo_stack:
            # 1. Sichere den aktuellen Zustand vor dem Sprung auf den Undo-Stapel
            current_live = {
                "ui_schema": copy.deepcopy(cfg.UI_SCHEMA),
                "translations": copy.deepcopy(self._i18n._translations) if
self._i18n else {}
            }
            self._undo_stack.append(current_live)

            # 2. Hole den Zustand aus der Zukunft
            future = self._redo_stack.pop()

            # 3. Überschreibe das Schema ohne Referenzverlust
            cfg.UI_SCHEMA.clear()
            cfg.UI_SCHEMA.update(copy.deepcopy(future["ui_schema"]))

            # Synchronisiere Sprachdateien
            if self._i18n and future["translations"]:
                self._i18n._translations = copy.deepcopy(future["translations"])
                self._write_translations_to_disk()

        return True
    return False

def _write_translations_to_disk(self):
    """Hilfsfunktion: Schreibt den wiederhergestellten Sprachpool physisch in
    die JSON-Dateien."""
    if not self._i18n or not hasattr(self._i18n, "_locales_dir"): return

    for lang in ["de", "en"]:
        if lang in self._i18n._translations:
            file_path = os.path.join(self._i18n._locales_dir, f"{lang}.json")
            try:
                with open(file_path, "w", encoding="utf-8") as f:
                    json.dump(self._i18n._translations[lang], f, indent=4,
ensure_ascii=False)
            except Exception as e:
                print(f"✗ Synchronisations-Fehler beim Schreiben von
{lang}.json: {e}")

```

File: infrastructure\languages\\_\_init\_\_.py

File: infrastructure\languages\translator.py

```
# Datei: infrastructure/languages/translator.py
"""Infrastructure Layer: KI-gestützte Offline-Übersetzung und automatischer
Validierungs-Service"""

import os
import json
import urllib.request
import requests # <-- DIESE ZEILE HIER DIREKT ERGÄNZEN!
import infrastructure.cfg as cfg

class DynamicLanguageManager:
    """Verwaltet dynamische Sprachvorschläge via Ollama und generiert/validiert
    Sprach-Vorlagen zur Laufzeit."""

    def __init__(self, services: dict = None):
        self._services = services
        self.active_languages = cfg.APP_CONFIG.get("supported_languages", ["de",
"en"])

    def check_ollama_status(self) -> bool:
        """Prüft schnell und lautlos, ob der lokale Ollama-Server erreichbar
ist."""
        import requests
        try:
            # Nutzt die Standard-Ollama-URL aus deiner Konfiguration
            url = getattr(self, "base_url", "http://localhost:11434")
            # Ein kurzes Timeout verhindert, dass die GUI einfriert, falls Ollama
hängt

            response = requests.get(url, timeout=15.0)
            return response.status_code == 200
        except Exception:
            return False

    def suggest_translation(self, text: str, target_lang_name: str) -> str:
        """Generiert offline mithilfe des lokalen Ollama-LLMs einen intelligenten
Übersetzungsvorschlag."""
        model_name = cfg.APP_CONFIG.get("default_model", "llama3")

        if self._services and "biz" in self._services:
            biz = self._services["biz"]
            if hasattr(biz, "_llm") and hasattr(biz._llm, "model_name"):
                model_name = biz._llm.model_name

        if not self.check_ollama_status():
            return text

        # Umstellung auf die schnellere generate-API (identisch zum erfolgreichen
Testskript)
        url = "http://localhost:11434/api/generate"

        # Ein kompakter, klarer englisch-deutscher Hybrid-Prompt, den Llama3
```



```

perfekt versteht
    prompt = (
        f"You are a professional translator for the historical software
framework ClioGraph.\n"
        f"Translate the German term '{text}' precisely, scholarly, and
extremely compact into the target language: '{target_lang_name}'.\n\n"
        f"Examples:\n"
        f"- 'Quellenverwaltung' -> 'Source Management' / 'Archivio' / matching
terms\n"
        f"- 'Datenbank' -> 'Database'\n"
        f"- 'Hilfe' -> 'Help'\n\n"
        f"CRITICAL RULE: Reply ONLY with the raw translated term or matching
short phrase. "
        f"Do not repeat the original text if a translation exists. "
        f"No explanations, no quotes, no conversational text. Only the direct
translation."
    )

    payload = {
        "model": model_name,
        "prompt": prompt,
        "stream": False,
        "options": { "temperature": 0.1 }
    }

    try:
        # Nutzung der installierten requests-Bibliothek mit robustem 15-
        Sekunden-Timeout
        response = requests.post(url, json=payload, timeout=15.0)

        if response.status_code == 200:
            result_json = response.json()
            message_content = result_json.get("response", "").strip()
            # Bereinigung von störenden Satzzeichen, falls die KI welche
mitsendet
            return message_content.replace("'", '').replace('"',
            "").replace(".", "")
        else:
            print(f"⚠️ [KI-Fehler] Server antwortet mit Status-Code
{response.status_code}")

    except requests.exceptions.Timeout:
        print("🕒 [Ollama-Timeout] Abfrage übersprungen. Das Zeitlimit von 15
        Sekunden wurde überschritten.")
    except Exception as e:
        print(f"⚠️ [KI-Fehler] Abfrage fehlgeschlagen: {e}")

    return text

def build_new_language_pack(self, target_code: str, target_name: str,
i18n_service) -> bool:
    """Klont die de.json, übersetzt alle Schlüssel offline via Ollama und
speichert sie ab."""

```

```

source_translations = i18n_service._translations.get("de", {})
if not source_translations: return False

new_package = {}
print(f"🤖 [Sprach-Designer] Starte Offline-KI-Gesamtexport für
'{target_name}''...")

for key, de_text in source_translations.items():
    new_package[key] = self.suggest_translation(de_text, target_name)

target_file = os.path.join(cfg.JSON_LOCALES_DIR, f"{target_code}.json")
try:
    with open(target_file, "w", encoding="utf-8") as f:
        json.dump(new_package, f, indent=4, ensure_ascii=False)
    i18n_service._translations[target_code] = new_package
    if target_code not in cfg.APP_CONFIG["supported_languages"]:
        cfg.APP_CONFIG["supported_languages"].append(target_code)
    if {"name": target_name, "code": target_code} not in
cfg.AVAILABLE_LANGUAGES:
        cfg.AVAILABLE_LANGUAGES.append({"name": target_name, "code":
target_code})
    return True
except Exception:
    return False

def validate_and_heal_locales(self, target_code: str, target_name: str,
i18n_service) -> int:
    """Scannt die Sprachdatei der gewählten Zielsprache und heilt alle
    identischen/fehlenden Einträge via KI."""
    de_dict = i18n_service._translations.get("de", {})

    # 1. Die Übersetzungsdatei der Zielsprache laden (z.B. fr.json oder
    fa.json)
    target_file = os.path.join(cfg.JSON_LOCALES_DIR, f"{target_code}.json")
    target_dict = {}

    if os.path.exists(target_file):
        try:
            with open(target_file, "r", encoding="utf-8") as f:
                target_dict = json.load(f)
        except Exception:
            target_dict = {}

    repairstatistiken = 0
    print(f"🔍 [KI-Validierung] Starte Überprüfung für Zielsprache:
    {target_name} ({target_code})'...")

    # 2. Schleife über alle existierenden deutschen Master-Schlüssel
    for key, de_text in de_dict.items():
        if key in ["app.title", "dialog.language.select.title"]: continue

        # WANN MUSS DIE KI HEILEN?
        # Wenn der Schlüssel in der Zielsprache fehlt ODER noch den deutschen
        Text enthält (Soll-Bruchstelle)

```

```

        if key not in target_dict or target_dict[key] == de_text or
target_dict[key].strip() == "":
            print(f"🔧 [Validierung -> {target_code}] Repariere:
'{de_text}')"
            geheiltes_wort = self.suggest_translation(de_text, target_name)

            if geheiltes_wort != de_text:
                target_dict[key] = geheiltes_wort
                repairstatistiken += 1
            else:
                target_dict[key] = de_text

# 3. Wenn Reparaturen stattfanden, die Datei physisch zurückschreiben
if repairstatistiken > 0 or not os.path.exists(target_file):
    try:
        with open(target_file, "w", encoding="utf-8") as f:
            json.dump(target_dict, f, indent=4, ensure_ascii=False)
            i18n_service._translations[target_code] = target_dict # RAM
aktualisieren
    except Exception as e:
        print(f"❌ Fehler beim Schreiben der Reparatur-Datei
{target_code}.json: {e}")

    return repairstatistiken

```

File: infrastructure\llm\_ml\\_\_init\_\_.py

File: infrastructure\llm\_ml\llm.py

```

# Datei: infrastructure/llm_ml/llm.py
"""Infrastructure Layer: Kapselung des lokalen LLM-Dienstes via Ollama"""
from infrastructure.exceptions import LlmTimeoutError
import infrastructure.cfg as cfg

class LlmExtractionService:
    """Verwaltet die Abfragen an das lokale Ollama-Modell zur
    Entitätenextraktion."""

    def __init__(self, model_name: str = cfg.APP_CONFIG["default_model"]):
        # Falls kein Modell übergeben wird, nimm automatisch den Standard aus der
        cfg
        self._model_name = model_name

    def extract_entities_and_relations(self, text: str) -> list:
        """Sendet Rohtext an Ollama und erwartet eine strukturierte Liste zurück.

```

```

        Fängt Verbindungs- und Timeout-Fehler ab und reicht sie als Business-Error
weiter.
    """
    try:
        # Platzhalter für den echten Ollama API-Aufruf
        # Später: response = ollama.chat(model=self._model_name, ...)
        print(f"[Infrastructure] Sende Text an Ollama-Modell
'{self._model_name}'...")

        # Temporäre Test-Rückgabe (wird bald durch echte LLM-Logik ersetzt)
        return []
    except Exception as e:
        # Technische API-Abstürze in saubere Business-Exception verpacken
        raise LlmTimeoutError(f"Ollama-Modell '{self._model_name}' antwortet
nicht: {str(e)}")

```

File: json\_storage\dynamic\_menu.json

```

{
  "menu_structure": {
    "file": {
      "i18n_key": "menu.file",
      "items": [
        {
          "type": "action",
          "id": "menu.file.language",
          "command_class": "ChangeLanguageCommand"
        },
        {
          "type": "action",
          "id": "menu.file.ollama",
          "command_class": "OllamaInfoCommand"
        },
        {
          "type": "action",
          "id": "menu.file.validate",
          "command_class": "OllamaValidateCommand"
        },
        {
          "type": "action",
          "id": "menu.file.edit_locales",
          "command_class": "LanguageEditorCommand"
        },
        {
          "type": "separator"
        },
        {
          "type": "action",
          "id": "menu.file.reset_system",
          "command_class": "ResetSystemCommand"
        }
      ]
    }
  }
}

```

```

    },
    {
        "type": "separator"
    },
    {
        "type": "action",
        "id": "menu.file.exit",
        "command_class": "ExitCommand"
    }
]
}
},
"menu_order": [
    "file"
],
"tab_structure": [
    {
        "id": "tab.welcome",
        "i18n_key": "tabs.welcome.title",
        "label": "Willkommen",
        "layout_type": "form",
        "elements": [
            {
                "id": "welcome_headline",
                "type": "label",
                "i18n_key": "welcome.headline.text",
                "width": 400,
                "height": 40,
                "x": 20,
                "y": 20
            },
            {
                "id": "search_input_line",
                "type": "input_line",
                "placeholder_i18n": "welcome.search.placeholder",
                "width": 250,
                "height": 30,
                "x": 20,
                "y": 80
            },
            {
                "id": "search_execute_button",
                "type": "button",
                "i18n_key": "welcome.button.search",
                "macro_file": "macros.search_macro",
                "macro_event": "execute_search",
                "width": 120,
                "height": 30,
                "x": 280,
                "y": 80
            }
        ]
    }
],

```

```

"supported_languages": [
    "de",
    "en"
],
"available_languages": []
}

```

File: json\_storage\locales\de.json

```

{
  "app.status.ready": "System-Infrastruktur geladen. Bereit.",
  "app.title": "ClioGraph — Historische Graphen-Analyse & Graph-RAG",
  "dialog.assistant.btn.cancel": "Beenden",
  "dialog.assistant.btn.ki": "🚀 KI--Übersetzungsvorschlag",
  "dialog.assistant.btn.manual": "📝 Manuell generieren",
  "dialog.assistant.btn.ok": "OK / Bestätigen",
  "dialog.assistant.dropdown.none": "--- Keine weitere Sprache ---",
  "dialog.assistant.group.eu": "Europäische Sprachen einbeziehen:",
  "dialog.assistant.group.intl": "Zusätzliche internationale Sprache vorbereiten (ISO-Standard):",
  "dialog.assistant.info": "Wählen Sie die europäischen Hauptsprachen zur Optimierung aus und/oder bestimmen Sie eine internationale Zielsprache aus der standardisierten ISO-Systemliste:",
  "dialog.assistant.msg.status.calc": "🧮 System berechnet Strukturvorlage...",
  "dialog.assistant.msg.status.ki": "🧠 Lokale KI generiert präzise Fachübersetzungen...",
  "dialog.assistant.msg.success": "<b>Der Sprachen-Assistent hat das Sprachpaket erfolgreich vorbereitet!</b><br><br>Folgende Projektsprachen wurden im System registriert:<br>• {log}<br><br><b>Nächster Schritt:</b> Nutzen Sie das Menü <i>Datei → 📝 Sprachdateien bearbeiten</i>. Dort können Sie die Begriffe komfortabel manuell anpassen oder per KI-Übersetzungsvorschlag befüllen lassen.",
  "dialog.assistant.title": "🌐 Sprachen-Assistent & Internationalisierung",
  "dialog.btn.save_generic": "💾 Speichern",
  "dialog.editor.btn.save": "💾 Änderungen in Tabelle speichern",
  "dialog.editor.col.de": "Referenz: Deutsch",
  "dialog.editor.col.en": "Referenz: Englisch",
  "dialog.editor.col.key": "System-Schlüssel (Gesperrt)",
  "dialog.editor.col.target": "Ihre Übersetzung (Editierbar)",
  "dialog.editor.label.select": "Wählen Sie die zu bearbeitende Projektsprache:",
  "dialog.editor.msg.success": "💾 Die Sprachdatei wurde erfolgreich aktualisiert und die Benutzeroberfläche zur Laufzeit neu gezeichnet!",
  "dialog.editor.title": "📝 In-App Sprachdateien-Editor",
  "dialog.item.add.de_prompt": "Deutscher Name des neuen Items:",
  "dialog.item.add.en_prompt": "Englischer Name des neuen Items:",
  "dialog.item.add.title": "Neues Item hinzufügen",
  "dialog.item.add.type_action": "Aktion (Klickbar)",
  "dialog.item.add.type_prompt": "Typ bestimmen:",
  "dialog.item.add.type_submenu": "Submenü (Verschachtelt)",
  "dialog.item.rename.de_prompt": "Neuen DEUTSCHEN Namen eingeben:",
  "dialog.item.rename.en_prompt": "Enter new ENGLISH name:",

```

```

"dialog.item.select.target": "Ziel-Menü wählen:",
"dialog.item.select_label": "Menüeintrag wählen:",
"dialog.language.add_option": "✚ [Neue Sprache hinzufügen...]",
"dialog.language.designer.text": "Welche wissenschaftliche Zielsprache soll
die KI generieren?",
"dialog.language.designer.title": "Language Designer",
"dialog.language.select.text": "Wähle eine Sprache:",
"dialog.language.select.title": "Language / Sprache",
"dialog.language.switch.hint.text": "<b>Hinweis zum wissenschaftlichen
Arbeiten:</b><br><br>Sie haben zu einer neu vorbereiteten Sprache gewechselt.
<br>Da diese noch nicht vollständig übersetzt wurde, sehen Sie aktuell Platzhalter
in der Benutzeroberfläche.<br><br><b>Nächster Schritt:</b> Nutzen Sie das Menü
<i>Datei → ✏ Sprachdateien bearbeiten</i>, um die Begriffe komfortabel und
direkt innerhalb von ClioGraph zu übersetzen.",
"dialog.language.switch.hint.title": "Sprachwechsel aktiv",
"dialog.maintenance.prompt": "Möchten Sie alle zusätzlichen Sprachdateien
unwiderruflich löschen und das System auf Deutsch/Englisch zurücksetzen?
\\n\\n(Hinweis: Es wird automatisch ein Sicherheits-Backup im Ordner angelegt).",
"dialog.maintenance.success": "<b>Sprach-Reset erfolgreich durchgeführt!</b>
<br><br>Die dynamic_menu.json wurde synchronisiert und alle Sicherungsdateien
wurden im Ordner <i>backup_locales</i> hinterlegt.",
"dialog.maintenance.title": "System-Bereinigung",
"dialog.menu.bilingual.prompt.de": "Geben Sie hier das DEUTSCHE Gegenstück für
das neue Menü ein:\\n(Aktueller Titel: '{title}']",
"dialog.menu.bilingual.prompt.en": "Geben Sie hier die ENGLISCHE Übersetzung
für das neue Menü ein:\\n(Bisheriger DE-Titel: '{title}']",
"dialog.menu.bilingual.title": "Zweisprachigkeit garantieren",
"dialog.menu.delete.confirm": "Wollen Sie das Hauptmenü '{0}' mit den gesamten
Items und Untermenüs wirklich löschen?",
"dialog.menu.delete.title": "Hauptmenü löschen",
"dialog.menu.limit_reached": "Die maximale Anzahl von {0} Hauptmenüs wurde
erreicht!",
"dialog.menu.prompt.de": "Deutscher Name des Hauptmenüs:",
"dialog.menu.prompt.en": "Englischer Name des Hauptmenüs:",
"dialog.menu.select.default": "Hauptmenü wählen:",
"dialog.menu.select.delete": "Zu löschendes Hauptmenü wählen:",
"dialog.menu.select.rename": "Zu umbenennendes Hauptmenü wählen:",
"dialog.menu.select_label": "Hauptmenü wählen:",
"dialog.ollama.error.text": "<b>Fehler: Lokale KI-Infrastruktur nicht
erreichbar!</b><br><br>Um eine neue Sprache vollautomatisch zu generieren,
benötigt ClioGraph den lokalen Dienst Ollama.<br><br><b>Bitte gehen Sie wie folgt
vor:</b><br>1. Installieren und starten Sie die Software Ollama auf diesem PC.
<br>2. Öffnen Sie das Menü <i>Datei -> Ollama-Dienst (KI)</i> für die Download-
Informationen.",
"dialog.ollama.text": "Für die intelligenten Funktionen wie automatische
Übersetzungsvorschläge und die Generierung neuer Sprachen (Language Designer)
nutzt ClioGraph den lokalen Dienst <b>Ollama</b>.<br><br><b>Vorteile für die
Forschung:</b><br>• 100% Offline-Betrieb (kein Internet notwendig)<br>• Maximaler
Datenschutz für vertrauliche Archivquellen<br><br>Falls der Dienst auf diesem PC
noch nicht installiert ist, können Sie die Software hier kostenfrei herunterladen:
<br><a href='https://ollama.com'>https://ollama.com</a><br><br><i>Tipp: Starten
Sie nach der Installation Ollama und laden Sie das Basismodell im Windows-Terminal
mit dem Befehl: <b>ollama run llama3</b></i>",
"dialog.ollama.title": "Offline-KI Infrastruktur (Ollama)",

```

```

"dialog.reset.text": "Möchten Sie das System wirklich auf die
Werkseinstellungen zurücksetzen?\nAlle dynamischen Menüs und hinzugefügten
Sprachen werden archiviert.",
"dialog.reset.title": "System zurücksetzen",
"dialog.tab.delete.confirm": "Möchten Sie das Register \"{name}\" wirklich
löschen?",
"dialog.tab.select_label": "Registerkarte wählen:",
"dialog.welcome.headline": "Exemplarische Vorgehensweisen",
"ide.btn.generate": "Formular generieren",
"ide.btn.insert_widget": "Baustein auf Tab platzieren",
"ide.context.delete": "[IDE] Element löschen",
"ide.context.jump_macro": "[IDE] Zum Makro-Code springen",
"ide.context.move_down": "[IDE] Nach unten verschieben",
"ide.context.move_up": "[IDE] Nach oben verschieben",
"ide.context.properties": "[IDE] Größe und Position...",
"ide.dialog.height": "Höhe (px):",
"ide.dialog.pos_x": "Position X (px):",
"ide.dialog.pos_y": "Position Y (px):",
"ide.dialog.prop_title": "Form formatieren",
"ide.dialog.save": "Speichern",
"ide.dialog.width": "Breite (px):",
"ide.group.form_controls": "🔧 Formular-Steuerelemente",
"ide.group.genai_designer": "🤖 GenAI Formular Designer",
"ide.group.menu_commands": "📁 Menübefehle zuordnen",
"ide.headline.genai": "GenAI Formular-Builder",
"ide.headline.vba": "Formular-Steuerelemente",
"ide.label.element_id": "Technische ID (eindeutig):",
"ide.label.widget_type": "Objekt-Typ auswählen:",
"ide.placeholder.prompt": "Formular per Prompt beschreiben...",
"ide.sonde.btn_close": "Schließen",
"ide.sonde.btn_edit": "Code bearbeiten",
"ide.sonde.data": "Übergebene Daten: {data}",
"ide.sonde.element": "Steuerelement: '{label}'",
"ide.sonde.hint": "Bitte bearbeiten Sie das Modul: '{module}'",
"ide.sonde.question": "Möchten Sie den Makro-Code im Editor bearbeiten?",
"ide.sonde.success": "Event '{event}' erfolgreich ausgelöst!",
"ide.sonde.title": "ClioGraph IDE Event-Sonde",
"menu.apis": "APIs",
"menu.community": "Community",
"menu.database": "Datenbank",
"menu.dynamic.menu_47d178f74018": "Hauptmenü1_de",
"menu.file": "Datei",
"menu.file.edit_locales": "✎ Sprachdateien bearbeiten",
"menu.file.exit": "Beenden",
"menu.file.export": "Projekt exportieren",
"menu.file.language": "🔄 Sprache wechseln",
"menu.file.ollama": "🤖 Ollama-Dienst (KI)",
"menu.file.reset_system": "🔧 System-Infrastruktur zurücksetzen",
"menu.file.validate": "🌐 Sprachen-Assistent",
"menu.help": "Hilfe",
"menu.language": "Sprache",
"menu.quellenverwaltung": "Quellenverwaltung",
"menu.register": "Register",
"menu.sources": "Quellenverwaltung",

```



```

"menu.visualization": "Visualisierung",
"mod1.btn.execute": "Analyse-Prozess starten",
"mod1.headline": "Daten-Erfassung und Textanalyse",
"mod1.placeholder.ref": "Quellen-Referenz (z.B. Signatur, Mappe, ID)...",
"mod1.placeholder.text": "Quellentext hier einfügen...",
"mod1.title": "Wissens-Extraktion",
"mod2.btn.execute": "Netzwerk-Metriken berechnen",
"mod2.headline": "Historische Netzwerkanalyse (HNA)",
"mod2.placeholder.text": "Metriken berechnen (Zentralitäten, Dichte,
Cluster)...",
"mod2.title": "Netzwerkanalyse",
"mod3.btn.execute": "Graph-RAG Suche ausführen",
"mod3.headline": "Interaktives LLM-Wissensgraphen-Training & GraphRAG
Abfragen",
"mod3.placeholder.text": "Graph-RAG Frage an die historischen Daten
eingeben...",
"mod3.title": "Graph-RAG Training",
"mod4.btn.execute": "GNN-Training starten",
"mod4.headline": "Prädiktive Graphenanalyse & Graph Neural Networks (GNN)",
"mod4.placeholder.text": "GNN-Modell konfigurieren (Link-Prediction / Node-
Classification)...",
"mod4.title": "Graph Machine Learning",
"msg.protection.core_deny": "Dieses Element gehört zum geschützten Systemkern
und darf weder gelöscht, verschoben noch umbenannt werden!",
"msg.protection.title": "System-Schutzfunktion",
"notification.redo.empty": "Keine weiteren Schritte zum Wiederholen.",
"notification.undo.empty": "Keine weiteren Schritte zum Rückgängigmachen.",
"sidebar.btn.developer_tool": "Entwicklertools (VBA & GenAI)",
"sidebar.designer.title": "🌸 SYSTEM-DESIGNER",
"sidebar.item.add": "+ Neues Item",
"sidebar.item.assign": "⚡ Befehl zuordnen",
"sidebar.item.assign_btn": "Befehl zuordnen...",
"sidebar.item.delete": "- Item löschen",
"sidebar.item.group": "📁 MENÜEINTRÄGE",
"sidebar.item.move_down": "▼ Nach unten",
"sidebar.item.move_up": "▲ Nach oben",
"sidebar.item.rename": "✏ Item umbenennen",
"sidebar.menu.add": "+ Neues Hauptmenü",
"sidebar.menu.assign_command": "Menübefehl zuordnen",
"sidebar.menu.delete": "- Hauptmenü löschen",
"sidebar.menu.group": "🌸 HAUPTMENÜ",
"sidebar.menu.move_left": "◀ Nach links",
"sidebar.menu.move_right": "▶ Nach rechts",
"sidebar.menu.rename": "✏ Hauptmenü umbenennen",
"sidebar.safety.redo": "Wiederholen",
"sidebar.safety.undo": "Rückgängig",
"sidebar.tab.add": "+ Neues Register",
"sidebar.tab.delete": "- Register löschen",
"sidebar.tab.group": "📁 REGISTERKARTEN",
"sidebar.tab.move_left": "◀ Nach links",
"sidebar.tab.move_right": "▶ Nach rechts",
"sidebar.tab.rename": "✏ Register umbenennen",
"sonde.msg.1": "Bereinigung wird initialisiert...",
"sonde.msg.2": "Lösche Konfigurationsdateien (dynamic_menu.json)...",

```

```

"sonde.msg.3": "Archiviere zusätzliche Sprachdateien...",
"sonde.msg.4": "Setze Systemkonfigurationen im RAM zurück...",
"sonde.msg.5": "Initialisiere Benutzeroberfläche neu...",
"sonde.msg.done": "Das System wurde erfolgreich in den Urzustand
zurückgesetzt!",
"sonde.title.1": "Schritt 1/5",
"sonde.title.2": "Schritt 2/5",
"sonde.title.3": "Schritt 3/5",
"sonde.title.4": "Schritt 4/5",
"sonde.title.5": "Schritt 5/5",
"sonde.title.done": "Erfolg",
"tabs.welcome.info_text": "<h2>Willkommen bei ClioGraph 

```

File: json\_storage\locales\de\_factory.json

```

{
  "app.status.ready": "System-Infrastruktur geladen. Bereit.",
  "app.title": "ClioGraph — Historische Graphen-Analyse & Graph-RAG",
  "dialog.assistant.btn.cancel": "Beenden",
  "dialog.assistant.btn.ki": "<img alt='rocket icon' data-bbox='412 802 432 816'/> KI--Übersetzungsvorschlag",
  "dialog.assistant.btn.manual": "<img alt='document icon' data-bbox='452 818 472 832'/> Manuell generieren",
  "dialog.assistant.btn.ok": "OK / Bestätigen",
  "dialog.assistant.dropdown.none": "--- Keine weitere Sprache ---",
  "dialog.assistant.group.eu": "Europäische Sprachen einbeziehen:",
  "dialog.assistant.group.intl": "Zusätzliche internationale Sprache vorbereiten
(ISO-Standard):",
  "dialog.assistant.info": "Wählen Sie die europäischen Hauptsprachen zur
Optimierung aus und/oder bestimmen Sie eine internationale Zielsprache aus der

```

```

standardisierten ISO-Systemliste:",
  "dialog.assistant.msg.status.calc": "🧮 System berechnet Strukturvorlage...",
  "dialog.assistant.msg.status.ki": "🤖 Lokale KI generiert präzise
Fachübersetzungen...",
  "dialog.assistant.msg.success": "<b>Der Sprachen-Assistent hat das Sprachpaket
erfolgreich vorbereitet!</b><br><br>Folgende Projektsprachen wurden im System
registriert:<br>• {log}<br><br><b>Nächster Schritt:</b> Nutzen Sie das Menü
<i>Datei → 📝 Sprachdateien bearbeiten</i>. Dort können Sie die Begriffe
komfortabel manuell anpassen oder per KI-Übersetzungsvorschlag befüllen lassen.",
  "dialog.assistant.title": "🌐 Sprachen-Assistent & Internationalisierung",
  "dialog.btn.save_generic": "💾 Speichern",
  "dialog.editor.btn.save": "💾 Änderungen in Tabelle speichern",
  "dialog.editor.col.de": "Referenz: Deutsch",
  "dialog.editor.col.en": "Referenz: Englisch",
  "dialog.editor.col.key": "System-Schlüssel (Gesperrt)",
  "dialog.editor.col.target": "Ihre Übersetzung (Editierbar)",
  "dialog.editor.label.select": "Wählen Sie die zu bearbeitende
Projektsprache:",
  "dialog.editor.msg.success": "💾 Die Sprachdatei wurde erfolgreich
aktualisiert und die Benutzeroberfläche zur Laufzeit neu gezeichnet!",
  "dialog.editor.title": "📝 In-App Sprachdateien-Editor",
  "dialog.item.add.de_prompt": "Deutscher Name des neuen Items:",
  "dialog.item.add.en_prompt": "Englischer Name des neuen Items:",
  "dialog.item.add.title": "Neues Item hinzufügen",
  "dialog.item.add.type_action": "Aktion (Klickbar)",
  "dialog.item.add.type_prompt": "Typ bestimmen:",
  "dialog.item.add.type_submenu": "Submenü (Verschachtelt)",
  "dialog.item.rename.de_prompt": "Neuen DEUTSCHEN Namen eingeben:",
  "dialog.item.rename.en_prompt": "Enter new ENGLISH name:",
  "dialog.item.select.target": "Ziel-Menü wählen:",
  "dialog.item.select_label": "Menüeintrag wählen:",
  "dialog.language.add_option": "➕ [Neue Sprache hinzufügen...]",
  "dialog.language.designer.text": "Welche wissenschaftliche Zielsprache soll
die KI generieren?",
  "dialog.language.designer.title": "Language Designer",
  "dialog.language.select.text": "Wähle eine Sprache:",
  "dialog.language.select.title": "Language / Sprache",
  "dialog.language.switch.hint.text": "<b>Hinweis zum wissenschaftlichen
Arbeiten:</b><br><br>Sie haben zu einer neu vorbereiteten Sprache gewechselt.
<br>Da diese noch nicht vollständig übersetzt wurde, sehen Sie aktuell Platzhalter
in der Benutzeroberfläche.<br><br><b>Nächster Schritt:</b> Nutzen Sie das Menü
<i>Datei → 📝 Sprachdateien bearbeiten</i>, um die Begriffe komfortabel und
direkt innerhalb von ClioGraph zu übersetzen.",
  "dialog.language.switch.hint.title": "Sprachwechsel aktiv",
  "dialog.maintenance.prompt": "Möchten Sie alle zusätzlichen Sprachdateien
unwiderruflich löschen und das System auf Deutsch/Englisch zurücksetzen?
\\n\\n(Hinweis: Es wird automatisch ein Sicherheits-Backup im Ordner angelegt).",
  "dialog.maintenance.success": "<b>Sprach-Reset erfolgreich durchgeführt!</b>
<br><br>Die dynamic_menu.json wurde synchronisiert und alle Sicherungsdateien
wurden im Ordner <i>backup_locales</i> hinterlegt.",
  "dialog.maintenance.title": "System-Bereinigung",
  "dialog.menu.bilingual.prompt.de": "Geben Sie hier das DEUTSCHE Gegenstück für
das neue Menü ein:\\n(Aktueller Titel: '{title}')" ,
  "dialog.menu.bilingual.prompt.en": "Geben Sie hier die ENGLISCHE Übersetzung

```

```
für das neue Menü ein:\n(Bisheriger DE-Titel: '{title}'),
"dialog.menu.bilingual.title": "Zweisprachigkeit garantieren",
"dialog.menu.delete.confirm": "Wollen Sie das Hauptmenü '{0}' mit den gesamten
Items und Untermenüs wirklich löschen?",
"dialog.menu.delete.title": "Hauptmenü löschen",
"dialog.menu.limit_reached": "Die maximale Anzahl von {0} Hauptmenüs wurde
erreicht!",
"dialog.menu.prompt.de": "Deutscher Name des Hauptmenüs:",
"dialog.menu.prompt.en": "Englischer Name des Hauptmenüs:",
"dialog.menu.select.default": "Hauptmenü wählen:",
"dialog.menu.select.delete": "Zu löschendes Hauptmenü wählen:",
"dialog.menu.select.rename": "Zu umbenennendes Hauptmenü wählen:",
"dialog.menu.select_label": "Hauptmenü wählen:",
"dialog.ollama.error.text": "<b>Fehler: Lokale KI-Infrastruktur nicht
erreichbar!</b><br><br>Um eine neue Sprache vollautomatisch zu generieren,
benötigt ClioGraph den lokalen Dienst Ollama.<br><br><b>Bitte gehen Sie wie folgt
vor:</b><br>1. Installieren und starten Sie die Software Ollama auf diesem PC.
<br>2. Öffnen Sie das Menü <i>Datei -> Ollama-Dienst (KI)</i> für die Download-
Informationen.",
"dialog.ollama.text": "Für die intelligenten Funktionen wie automatische
Übersetzungsvorschläge und die Generierung neuer Sprachen (Language Designer)
nutzt ClioGraph den lokalen Dienst <b>Ollama</b>.<br><br><b>Vorteile für die
Forschung:</b><br>• 100% Offline-Betrieb (kein Internet notwendig)<br>• Maximaler
Datenschutz für vertrauliche Archivquellen<br><br>Falls der Dienst auf diesem PC
noch nicht installiert ist, können Sie die Software hier kostenfrei herunterladen:
<br><a href='https://ollama.com'>https://ollama.com</a><br><br><i>Tipp: Starten
Sie nach der Installation Ollama und laden Sie das Basismodell im Windows-Terminal
mit dem Befehl: <b>ollama run llama3</b></i>",
"dialog.ollama.title": "Offline-KI Infrastruktur (Ollama)",
"dialog.reset.text": "Möchten Sie das System wirklich auf die
Werkseinstellungen zurücksetzen?<br>Alle dynamischen Menüs und hinzugefügten
Sprachen werden archiviert.",
"dialog.reset.title": "System zurücksetzen",
"dialog.tab.delete.confirm": "Möchten Sie das Register \"{name}\" wirklich
löschen?",
"dialog.tab.select_label": "Registerkarte wählen:",
"dialog.welcome.headline": "Exemplarische Vorgehensweisen",
"ide.btn.generate": "Formular generieren",
"ide.btn.insert_widget": "Baustein auf Tab platzieren",
"ide.context.delete": "[IDE] Element löschen",
"ide.context.jump_macro": "[IDE] Zum Makro-Code springen",
"ide.context.move_down": "[IDE] Nach unten verschieben",
"ide.context.move_up": "[IDE] Nach oben verschieben",
"ide.context.properties": "[IDE] Größe und Position...",
"ide.dialog.height": "Höhe (px):",
"ide.dialog.pos_x": "Position X (px):",
"ide.dialog.pos_y": "Position Y (px):",
"ide.dialog.prop_title": "Form formatieren",
"ide.dialog.save": "Speichern",
"ide.dialog.width": "Breite (px):",
"ide.group.form_controls": "🛠 Formular-Steuerelemente",
"ide.group.genai_designer": "🤖 GenAI Formular Designer",
"ide.group.menu_commands": "📁 Menübefehle zuordnen",
"ide.headline.genai": "GenAI Formular-Builder",
```

```

"ide.headline.vba": "Formular-Steuerelemente",
"ide.label.element_id": "Technische ID (eindeutig):",
"ide.label.widget_type": "Objekt-Typ auswählen:",
"ide.placeholder.prompt": "Formular per Prompt beschreiben...",
"ide.sonde.btn_close": "Schließen",
"ide.sonde.btn_edit": "Code bearbeiten",
"ide.sonde.data": "Übergebene Daten: {data}",
"ide.sonde.element": "Steuerelement: '{label}'",
"ide.sonde.hint": "Bitte bearbeiten Sie das Modul: '{module}'",
"ide.sonde.question": "Möchten Sie den Makro-Code im Editor bearbeiten?",
"ide.sonde.success": "Event '{event}' erfolgreich ausgelöst!",
"ide.sonde.title": "ClioGraph IDE Event-Sonde",
"menu.apis": "APIs",
"menu.community": "Community",
"menu.database": "Datenbank",
"menu.dynamic.menu_47d178f74018": "Hauptmenü1_de",
"menu.file": "Datei",
"menu.file.edit_locales": "📝 Sprachdateien bearbeiten",
"menu.file.exit": "Beenden",
"menu.file.export": "Projekt exportieren",
"menu.file.language": "🔄 Sprache wechseln",
"menu.file.ollama": "🖨️ Ollama-Dienst (KI)",
"menu.file.reset_system": "🔄 System-Infrastruktur zurücksetzen",
"menu.file.validate": "🌐 Sprachen-Assistent",
"menu.help": "Hilfe",
"menu.language": "Sprache",
"menu.quellenverwaltung": "Quellenverwaltung",
"menu.register": "Register",
"menu.sources": "Quellenverwaltung",
"menu.visualization": "Visualisierung",
"mod1.btn.execute": "Analyse-Prozess starten",
"mod1.headline": "Daten-Erfassung und Textanalyse",
"mod1.placeholder.ref": "Quellen-Referenz (z.B. Signatur, Mappe, ID)...",
"mod1.placeholder.text": "Quellentext hier einfügen...",
"mod1.title": "Wissens-Extraktion",
"mod2.btn.execute": "Netzwerk-Metriken berechnen",
"mod2.headline": "Historische Netzwerkanalyse (HNA)",
"mod2.placeholder.text": "Metriken berechnen (Zentralitäten, Dichte,
Cluster)...",
"mod2.title": "Netzwerkanalyse",
"mod3.btn.execute": "Graph-RAG Suche ausführen",
"mod3.headline": "Interaktives LLM-Wissensgraphen-Training & GraphRAG
Abfragen",
"mod3.placeholder.text": "Graph-RAG Frage an die historischen Daten
eingeben...",
"mod3.title": "Graph-RAG Training",
"mod4.btn.execute": "GNN-Training starten",
"mod4.headline": "Prädiktive Graphenanalyse & Graph Neural Networks (GNN)",
"mod4.placeholder.text": "GNN-Modell konfigurieren (Link-Prediction / Node-
Classification)...",
"mod4.title": "Graph Machine Learning",
"msg.protection.core_deny": "Dieses Element gehört zum geschützten Systemkern
und darf weder gelöscht, verschoben noch umbenannt werden!",
"msg.protection.title": "System-Schutzfunktion",

```



```

"notification.redo.empty": "Keine weiteren Schritte zum Wiederholen.",
"notification.undo.empty": "Keine weiteren Schritte zum Rückgängigmachen.",
"sidebar.btn.developer_tool": "Entwicklertools (VBA & GenAI)",
"sidebar.designer.title": "🔧 SYSTEM-DESIGNER",
"sidebar.item.add": "+ Neues Item",
"sidebar.item.assign": "⚡ Befehl zuordnen",
"sidebar.item.assign_btn": "Befehl zuordnen...",
"sidebar.item.delete": "- Item löschen",
"sidebar.item.group": "📁 MENÜEINTRÄGE",
"sidebar.item.move_down": "▼ Nach unten",
"sidebar.item.move_up": "▲ Nach oben",
"sidebar.item.rename": "✏ Item umbenennen",
"sidebar.menu.add": "+ Neues Hauptmenü",
"sidebar.menu.assign_command": "Menübefehl zuordnen",
"sidebar.menu.delete": "- Hauptmenü löschen",
"sidebar.menu.group": "⚙️ HAUPTMENÜ",
"sidebar.menu.move_left": "◀ Nach links",
"sidebar.menu.move_right": "▶ Nach rechts",
"sidebar.menu.rename": "✏ Hauptmenü umbenennen",
"sidebar.safety.redo": "Wiederholen",
"sidebar.safety.undo": "Rückgängig",
"sidebar.tab.add": "+ Neues Register",
"sidebar.tab.delete": "- Register löschen",
"sidebar.tab.group": "📄 REGISTERKARTEN",
"sidebar.tab.move_left": "◀ Nach links",
"sidebar.tab.move_right": "▶ Nach rechts",
"sidebar.tab.rename": "✏ Register umbenennen",
"sonde.msg.1": "Bereinigung wird initialisiert...",
"sonde.msg.2": "Löschte Konfigurationsdateien (dynamic_menu.json)...",
"sonde.msg.3": "Archiviere zusätzliche Sprachdateien...",
"sonde.msg.4": "Setze Systemkonfigurationen im RAM zurück...",
"sonde.msg.5": "Initialisiere Benutzeroberfläche neu...",
"sonde.msg.done": "Das System wurde erfolgreich in den Urzustand
zurückgesetzt!",
"sonde.title.1": "Schritt 1/5",
"sonde.title.2": "Schritt 2/5",
"sonde.title.3": "Schritt 3/5",
"sonde.title.4": "Schritt 4/5",
"sonde.title.5": "Schritt 5/5",
"sonde.title.done": "Erfolg",
"tabs.welcome.info_text": "<h2>Willkommen bei ClioGraph 🏛️🇩🇪</h2><p>Dieses
dynamisch erweiterbare GUI-Basisframework dient als hochdisziplinierter,
entkoppelter Arbeitsplatz (n-tier) für wissenschaftliche Analysen in den Digital
Humanities.</p><h3>🌀 Geplante Fachlogik der Vollversion:</h3><ul><li><b>Quellen-
Verwaltung & Textscanning:</b> Automatisierte Strukturierung und Analyse
historischer Dokumente.</li><li><b>Historische Netzwerkanalyse:</b>
Hochperformante Beziehungs-Berechnungen via Kùzu-Cypher-Queries.</li><li><b>LLM-
Wissensgraphen & GraphRAG:</b> Lokales, datenschutzkonformes KI-Modell-Training
und Deep Prompting via Ollama.</li><li><b>Machine Learning (ML):</b> Innovative,
integrierte Modelle zur automatischen Handschriftenkennung (HTR).</li></ul><h3>🔧
Integrierte Entwickler-Werkzeuge:</h3><ul><li><b>System- & IDE-Designer
(Sidebars):</b> Hauptmenüs, Registerkarten und Formular-Bausteine dynamisch zur
Laufzeit erstellen, verschieben und zweisprachig umbenennen.</li><li>
<b>Unzerbrechlicher Code-Editor-Sprung:</b> Per Rechtsklick-Kontextmenü direkt aus

```

```

jedem Widget zur zugehörigen Makro-Methode in VS Code springen.</li></ul><br>
<b>Hinweis zur Dokumentation:</b> Das Projekthandbuch und die vollständige
Systemdokumentation (PDF & Markdown) finden Sie lokal im Projektverzeichnis unter
dem relativen Pfad: <code style=\"background-color: #f1f3f5; padding: 2px 6px;
border-radius: 4px; font-family: monospace;\">./_Dokumentation/</code><br><br><p
style=\"font-size: 11px; color: #666;\">This code is provided for
educational/utility purposes only without any warranty. Use at your own risk.<br>
<br><b>@2026 Copyright ClioGraph Schweiz</b></p>\",
  \"tabs.welcome.title\": \"Willkommen\"
}

```

File: `json_storage\locales\en.json`

```

{
  \"app.status.ready\": \"System infrastructure loaded. Ready.\",
  \"app.title\": \"ClioGraph — Historical Graph Analytics & Graph-RAG\",
  \"dialog.assistant.btn.cancel\": \"Close\",
  \"dialog.assistant.btn.ki\": \"🚀 AI Translation Suggestion\",
  \"dialog.assistant.btn.manual\": \"📄 Generate Manually\",
  \"dialog.assistant.btn.ok\": \"OK / Confirm\",
  \"dialog.assistant.dropdown.none\": \"--- No additional language ---\",
  \"dialog.assistant.group.eu\": \"Include European languages:\",
  \"dialog.assistant.group.intl\": \"Prepare additional international language (ISO
standard):\",
  \"dialog.assistant.info\": \"Select the main European languages for optimization
and/or choose an international target language from the standardized ISO system
list:\",
  \"dialog.assistant.msg.status.calc\": \"🧮 System is calculating structure
template...\",
  \"dialog.assistant.msg.status.ki\": \"🧠 Local AI is generating precise
professional translations...\",
  \"dialog.assistant.msg.success\": \"<b>The Language Assistant has successfully
prepared the language pack!</b><br><br>The following project languages have been
registered:<br>• {log}<br><br><b>Next Step:</b> Navigate to <i>File → 📝 Edit
Language Files</i> to translate and refine the terms manually or via AI
suggestions.\",
  \"dialog.assistant.title\": \"🌐 Language Assistant & Internationalization\",
  \"dialog.btn.save_generic\": \"💾 Save\",
  \"dialog.editor.btn.save\": \"💾 Save Table Changes\",
  \"dialog.editor.col.de\": \"Reference: German\",
  \"dialog.editor.col.en\": \"Reference: English\",
  \"dialog.editor.col.key\": \"System Key (Locked)\",
  \"dialog.editor.col.target\": \"Your Translation (Editable)\",
  \"dialog.editor.label.select\": \"Select the project language to edit:\",
  \"dialog.editor.msg.success\": \"💾 The language file has been successfully
updated and the user interface re-rendered in real-time!\",
  \"dialog.editor.title\": \"📝 In-App Language File Editor\",
  \"dialog.item.add.de_prompt\": \"German name of the new item:\",
  \"dialog.item.add.en_prompt\": \"English name of the new item:\",
  \"dialog.item.add.title\": \"Add New Item\",
  \"dialog.item.add.type_action\": \"Action (Clickable)\",

```

```

"dialog.item.add.type_prompt": "Select type:",
"dialog.item.add.type_submenu": "Submenu (Nested)",
"dialog.item.rename.de_prompt": "Neuen DEUTSCHEN Namen eingeben:",
"dialog.item.rename.en_prompt": "Enter new ENGLISH name:",
"dialog.item.select.target": "Select target menu:",
"dialog.item.select_label": "Select Menu Item:",
"dialog.language.add_option": "✚ [Add new language...]",
"dialog.language.designer.text": "Which scholarly target language should the
AI generate?",
"dialog.language.designer.title": "Language Designer",
"dialog.language.select.text": "Select a language:",
"dialog.language.select.title": "Language / Sprache",
"dialog.language.switch.hint.text": "<b>Note on Scholarly Workflow:</b><br>
<br>You have switched to a newly prepared language.<br>Since it has not been fully
translated yet, you will currently see placeholders in the user interface.<br><br>
<b>Next Step:</b> Use the menu <i>File → ✏ Edit Language Files</i> to translate
the terms comfortably and directly within ClioGraph.",
"dialog.language.switch.hint.title": "Language Switch Active",
"dialog.maintenance.prompt": "Do you want to permanently delete all additional
language files and reset the system to German/English?\n\n(Note: A safety backup
will be created automatically in the 'backup_locales' folder.)",
"dialog.maintenance.success": "<b>System reset successfully completed!</b><br>
<br>The dynamic_menu.json has been synchronized and all recovery files have been
stored in the <i>backup_locales</i> folder.",
"dialog.maintenance.title": "System Maintenance",
"dialog.menu.bilingual.prompt.de": "Please enter the GERMAN counterpart for
the new menu here:\n(Current title: '{title}')",
"dialog.menu.bilingual.prompt.en": "Please enter the ENGLISH translation for
the new menu here:\n(Previous DE title: '{title}')",
"dialog.menu.bilingual.title": "Ensure Bilingualism",
"dialog.menu.delete.confirm": "Are you sure you want to delete the main menu
'{0}' along with all its items and submenus?",
"dialog.menu.delete.title": "Delete Main Menu",
"dialog.menu.limit_reached": "The maximum number of {0} main menus has been
reached!",
"dialog.menu.prompt.de": "German name of the main menu:",
"dialog.menu.prompt.en": "English name of the main menu:",
"dialog.menu.select.default": "Select main menu:",
"dialog.menu.select.delete": "Select main menu to delete:",
"dialog.menu.select.rename": "Select main menu to rename:",
"dialog.menu.select_label": "Select Main Menu:",
"dialog.ollama.error.text": "<b>Error: Local AI Infrastructure not reachable!
</b><br><br>To generate a new language automatically, ClioGraph requires the local
service Ollama.<br><br><b>Please proceed as follows:</b><br>1. Install and start
the Ollama software on this PC.<br>2. Open the menu <i>File -> Ollama Service (AI)
</i> for download information.",
"dialog.ollama.text": "For intelligent features like automated translation
suggestions and creating new language packages (Language Designer), ClioGraph
utilizes the local service <b>Ollama</b>.<br><br><b>Advantages for Research:</b>
<br>• 100% Offline operation (no internet required)<br>• Maximum data privacy for
confidential archival sources<br><br>If the service is not yet installed on this
PC, you can download the software for free here:<br><a
href='https://ollama.com'>https://ollama.com</a><br><br><i>Tip: After
installation, start Ollama and download the base model in your Windows terminal

```



```

using the command: <b>ollama run llama3</b></i>",
  "dialog.ollama.title": "Offline AI Infrastructure (Ollama)",
  "dialog.reset.text": "Do you really want to reset the system to default
settings?\nAll dynamic menus and added languages will be archived.",
  "dialog.reset.title": "Reset System",
  "dialog.tab.delete.confirm": "Are you sure you want to delete the tab \"
{name}\"?",
  "dialog.tab.select_label": "Select Tab:",
  "dialog.welcome.headline": "Exemplary Procedures",
  "ide.btn.generate": "Generate Form",
  "ide.btn.insert_widget": "Insert Control on Tab",
  "ide.context.delete": "[IDE] Delete Element",
  "ide.context.jump_macro": "[IDE] Jump to Macro Code",
  "ide.context.move_down": "[IDE] Move Down",
  "ide.context.move_up": "[IDE] Move Up",
  "ide.context.properties": "[IDE] Size and Position...",
  "ide.dialog.height": "Height (px):",
  "ide.dialog.pos_x": "Position X (px):",
  "ide.dialog.pos_y": "Position Y (px):",
  "ide.dialog.prop_title": "Format Shape",
  "ide.dialog.save": "Save",
  "ide.dialog.width": "Width (px):",
  "ide.group.form_controls": "🔧 Form Controls",
  "ide.group.genai_designer": "🤖 GenAI Form Designer",
  "ide.group.menu_commands": "📁 Assign Menu Commands",
  "ide.headline.genai": "GenAI Formular Builder",
  "ide.headline.vba": "Form Controls",
  "ide.label.element_id": "Technical ID (unique):",
  "ide.label.widget_type": "Select Object Type:",
  "ide.placeholder.prompt": "Describe the desired form...",
  "ide.sonde.btn_close": "Close",
  "ide.sonde.btn_edit": "Edit Code",
  "ide.sonde.data": "Passed data: {data}",
  "ide.sonde.element": "Control element: '{label}'",
  "ide.sonde.hint": "Please edit the module: '{module}'",
  "ide.sonde.question": "Do you want to edit the macro code in the editor?",
  "ide.sonde.success": "Event '{event}' triggered successfully!",
  "ide.sonde.title": "ClioGraph IDE Event Probe",
  "menu.apis": "APIs",
  "menu.community": "Community",
  "menu.database": "Database",
  "menu.dynamic.menu_47d178f74018": "Hauptmenü1_en",
  "menu.file": "File",
  "menu.file.edit_locales": "✎ Edit Language Files",
  "menu.file.exit": "Exit",
  "menu.file.export": "Export Project",
  "menu.file.language": "🔄 Change Language",
  "menu.file.ollama": "🤖 Ollama Service (AI)",
  "menu.file.reset_system": "🔄 Reset System Infrastructure",
  "menu.file.validate": "🌐 Language Assistant",
  "menu.help": "Help",
  "menu.language": "Language",
  "menu.quellenverwaltung": "Sources",
  "menu.register": "Tabs",

```

```

"menu.sources": "Source Management",
"menu.visualization": "Visualization",
"mod1.btn.execute": "Start analysis process",
"mod1.headline": "Data Capture and Text Analysis",
"mod1.placeholder.ref": "Source reference (e.g., signature, folder, ID)...",
"mod1.placeholder.text": "Insert source text here...",
"mod1.title": "Knowledge Extraction",
"mod2.btn.execute": "Calculate network metrics",
"mod2.headline": "Historical Network Analysis (HNA)",
"mod2.placeholder.text": "Calculate metrics (centralities, density,
clusters)...",
"mod2.title": "Network Analytics",
"mod3.btn.execute": "Execute Graph-RAG search",
"mod3.headline": "Interactive LLM Knowledge Graph Training & GraphRAG
Queries",
"mod3.placeholder.text": "Enter Graph-RAG query for historical data...",
"mod3.title": "Graph-RAG Training",
"mod4.btn.execute": "Start GNN training",
"mod4.headline": "Predictive Graph Analysis & Graph Neural Networks (GNN)",
"mod4.placeholder.text": "Configure GNN model (Link Prediction / Node
Classification)...",
"mod4.title": "Graph Machine Learning",
"msg.protection.core_deny": "This element belongs to the protected system core
and cannot be deleted, moved, or renamed!",
"msg.protection.title": "System Protection",
"notification.redo.empty": "No further steps to redo.",
"notification.undo.empty": "No further steps to undo.",
"sidebar.btn.developer_tool": "Developer Tools (VBA & GenAI)",
"sidebar.designer.title": "🌀 SYSTEM DESIGNER",
"sidebar.item.add": "+ New Item",
"sidebar.item.assign": "⚡ Assign Command",
"sidebar.item.assign_btn": "Assign Command...",
"sidebar.item.delete": "- Delete Item",
"sidebar.item.group": "📁 MENU ITEMS",
"sidebar.item.move_down": "▼ Move Down",
"sidebar.item.move_up": "▲ Move Up",
"sidebar.item.rename": "✏️ Rename Item",
"sidebar.menu.add": "+ New Main Menu",
"sidebar.menu.assign_command": "Assign Menu Command",
"sidebar.menu.delete": "- Delete Main Menu",
"sidebar.menu.group": "🌀 MAIN MENU",
"sidebar.menu.move_left": "◀ Move Left",
"sidebar.menu.move_right": "▶ Move Right",
"sidebar.menu.rename": "✏️ Rename Main Menu",
"sidebar.safety.redo": "Redo",
"sidebar.safety.undo": "Undo",
"sidebar.tab.add": "+ New Tab",
"sidebar.tab.delete": "- Delete Tab",
"sidebar.tab.group": "📁 TABS",
"sidebar.tab.move_left": "◀ Move Left",
"sidebar.tab.move_right": "Move Right",
"sidebar.tab.rename": "✏️ Rename Tab",
"sonde.msg.1": "Initializing cleanup...",
"sonde.msg.2": "Deleting configuration files (dynamic_menu.json)...",

```

```

"sonde.msg.3": "Archiving additional language files...",
"sonde.msg.4": "Resetting system configurations in RAM...",
"sonde.msg.5": "Re-initializing user interface...",
"sonde.msg.done": "The system has been successfully reset to its original
state!",
"sonde.title.1": "Step 1/5",
"sonde.title.2": "Step 2/5",
"sonde.title.3": "Step 3/5",
"sonde.title.4": "Step 4/5",
"sonde.title.5": "Step 5/5",
"sonde.title.done": "Success",
"tabs.welcome.info_text": "<h2>Welcome to ClioGraph 🏛️🌐</h2><p>This
dynamically extensible GUI base framework serves as a highly disciplined,
decoupled workspace (n-tier) for historical analysis in the Digital Humanities.
</p><h3>🚀 Planned Business Logic for the Full Version:</h3><ul><li><b>Source
Management & Text Scanning:</b> Automated structuring and analysis of historical
documents.</li><li><b>Historical Network Analysis:</b> High-performance
relationship calculations via Kùzu-Cypher queries.</li><li><b>LLM Knowledge Graphs
& GraphRAG:</b> Local, privacy-compliant AI model training and deep prompting via
Ollama.</li><li><b>Machine Learning (ML):</b> Innovative, integrated models for
automatic handwritten text recognition (HTR).</li></ul><h3>🔧 Integrated
Developer Tools:</h3><ul><li><b>System- & IDE-Designer (Sidebars):</b> Dynamically
create, move, and bilingually rename main menus, tabs, and form elements at
runtime.</li><li><b>Unbreakable Code-Editor Jump:</b> Right-click any widget to
jump directly to its corresponding macro method in VS Code.</li></ul><br>📁
<b>Documentation Note:</b> The project manual and system documentation (PDF &
Markdown) can be found locally within the project directory at the relative path:
<code style=\"background-color: #f1f3f5; padding: 2px 6px; border-radius: 4px;
font-family: monospace;\">./_Dokumentation/</code><br><br><p style=\"font-size:
11px; color: #666;\">This code is provided for educational/utility purposes only
without any warranty. Use at your own risk.<br><br><b>@2026 Copyright ClioGraph
Schweiz</b></p>",
"tabs.welcome.title": "Welcome"
}

```

File: json\_storage\locales\en\_factory.json

```

{
  "app.status.ready": "System infrastructure loaded. Ready.",
  "app.title": "ClioGraph — Historical Graph Analytics & Graph-RAG",
  "dialog.assistant.btn.cancel": "Close",
  "dialog.assistant.btn.ki": "🚀 AI Translation Suggestion",
  "dialog.assistant.btn.manual": "📄 Generate Manually",
  "dialog.assistant.btn.ok": "OK / Confirm",
  "dialog.assistant.dropdown.none": "--- No additional language ---",
  "dialog.assistant.group.eu": "Include European languages:",
  "dialog.assistant.group.intl": "Prepare additional international language (ISO
standard):",
  "dialog.assistant.info": "Select the main European languages for optimization
and/or choose an international target language from the standardized ISO system
list:",

```

```

"dialog.assistant.msg.status.calc": "🧮 System is calculating structure
template...",
"dialog.assistant.msg.status.ki": "🧠 Local AI is generating precise
professional translations...",
"dialog.assistant.msg.success": "<b>The Language Assistant has successfully
prepared the language pack!</b><br><br>The following project languages have been
registered:<br>• {log}<br><br><b>Next Step:</b> Navigate to <i>File → 📝 Edit
Language Files</i> to translate and refine the terms manually or via AI
suggestions.",
"dialog.assistant.title": "🌐 Language Assistant & Internationalization",
"dialog.btn.save_generic": "💾 Save",
"dialog.editor.btn.save": "💾 Save Table Changes",
"dialog.editor.col.de": "Reference: German",
"dialog.editor.col.en": "Reference: English",
"dialog.editor.col.key": "System Key (Locked)",
"dialog.editor.col.target": "Your Translation (Editable)",
"dialog.editor.label.select": "Select the project language to edit:",
"dialog.editor.msg.success": "💾 The language file has been successfully
updated and the user interface re-rendered in real-time!",
"dialog.editor.title": "📝 In-App Language File Editor",
"dialog.item.add.de_prompt": "German name of the new item:",
"dialog.item.add.en_prompt": "English name of the new item:",
"dialog.item.add.title": "Add New Item",
"dialog.item.add.type_action": "Action (Clickable)",
"dialog.item.add.type_prompt": "Select type:",
"dialog.item.add.type_submenu": "Submenu (Nested)",
"dialog.item.rename.de_prompt": "Neuen DEUTSCHEN Namen eingeben:",
"dialog.item.rename.en_prompt": "Enter new ENGLISH name:",
"dialog.item.select.target": "Select target menu:",
"dialog.item.select_label": "Select Menu Item:",
"dialog.language.add_option": "➕ [Add new language...]",
"dialog.language.designer.text": "Which scholarly target language should the
AI generate?",
"dialog.language.designer.title": "Language Designer",
"dialog.language.select.text": "Select a language:",
"dialog.language.select.title": "Language / Sprache",
"dialog.language.switch.hint.text": "<b>Note on Scholarly Workflow:</b><br>
<br>You have switched to a newly prepared language.<br>Since it has not been fully
translated yet, you will currently see placeholders in the user interface.<br><br>
<b>Next Step:</b> Use the menu <i>File → 📝 Edit Language Files</i> to translate
the terms comfortably and directly within ClioGraph.",
"dialog.language.switch.hint.title": "Language Switch Active",
"dialog.maintenance.prompt": "Do you want to permanently delete all additional
language files and reset the system to German/English?\n\n(Note: A safety backup
will be created automatically in the 'backup_locales' folder.)",
"dialog.maintenance.success": "<b>System reset successfully completed!</b><br>
<br>The dynamic_menu.json has been synchronized and all recovery files have been
stored in the <i>backup_locales</i> folder.",
"dialog.maintenance.title": "System Maintenance",
"dialog.menu.bilingual.prompt.de": "Please enter the GERMAN counterpart for
the new menu here:\n(Current title: '{title}']",
"dialog.menu.bilingual.prompt.en": "Please enter the ENGLISH translation for
the new menu here:\n(Previous DE title: '{title}']",
"dialog.menu.bilingual.title": "Ensure Bilingualism",

```

```

"dialog.menu.delete.confirm": "Are you sure you want to delete the main menu
'{0}' along with all its items and submenus?",
"dialog.menu.delete.title": "Delete Main Menu",
"dialog.menu.limit_reached": "The maximum number of {0} main menus has been
reached!",
"dialog.menu.prompt.de": "German name of the main menu:",
"dialog.menu.prompt.en": "English name of the main menu:",
"dialog.menu.select.default": "Select main menu:",
"dialog.menu.select.delete": "Select main menu to delete:",
"dialog.menu.select.rename": "Select main menu to rename:",
"dialog.menu.select_label": "Select Main Menu:",
"dialog.ollama.error.text": "<b>Error: Local AI Infrastructure not reachable!
</b><br><br>To generate a new language automatically, ClioGraph requires the local
service Ollama.<br><br><b>Please proceed as follows:</b><br>1. Install and start
the Ollama software on this PC.<br>2. Open the menu <i>File -> Ollama Service (AI)
</i> for download information.",
"dialog.ollama.text": "For intelligent features like automated translation
suggestions and creating new language packages (Language Designer), ClioGraph
utilizes the local service <b>Ollama</b>.<br><br><b>Advantages for Research:</b>
<br>• 100% Offline operation (no internet required)<br>• Maximum data privacy for
confidential archival sources<br><br>If the service is not yet installed on this
PC, you can download the software for free here:<br><a
href='https://ollama.com'>https://ollama.com</a><br><br><i>Tip: After
installation, start Ollama and download the base model in your Windows terminal
using the command: <b>ollama run llama3</b></i>",
"dialog.ollama.title": "Offline AI Infrastructure (Ollama)",
"dialog.reset.text": "Do you really want to reset the system to default
settings?\nAll dynamic menus and added languages will be archived.",
"dialog.reset.title": "Reset System",
"dialog.tab.delete.confirm": "Are you sure you want to delete the tab \"
{name}\"?",
"dialog.tab.select_label": "Select Tab:",
"dialog.welcome.headline": "Exemplary Procedures",
"ide.btn.generate": "Generate Form",
"ide.btn.insert_widget": "Insert Control on Tab",
"ide.context.delete": "[IDE] Delete Element",
"ide.context.jump_macro": "[IDE] Jump to Macro Code",
"ide.context.move_down": "[IDE] Move Down",
"ide.context.move_up": "[IDE] Move Up",
"ide.context.properties": "[IDE] Size and Position...",
"ide.dialog.height": "Height (px):",
"ide.dialog.pos_x": "Position X (px):",
"ide.dialog.pos_y": "Position Y (px):",
"ide.dialog.prop_title": "Format Shape",
"ide.dialog.save": "Save",
"ide.dialog.width": "Width (px):",
"ide.group.form_controls": "🔧 Form Controls",
"ide.group.genai_designer": "🤖 GenAI Form Designer",
"ide.group.menu_commands": "📁 Assign Menu Commands",
"ide.headline.genai": "GenAI Formular Builder",
"ide.headline.vba": "Form Controls",
"ide.label.element_id": "Technical ID (unique):",
"ide.label.widget_type": "Select Object Type:",
"ide.placeholder.prompt": "Describe the desired form...",

```

```

"ide.sonde.btn_close": "Close",
"ide.sonde.btn_edit": "Edit Code",
"ide.sonde.data": "Passed data: {data}",
"ide.sonde.element": "Control element: '{label}'",
"ide.sonde.hint": "Please edit the module: '{module}'",
"ide.sonde.question": "Do you want to edit the macro code in the editor?",
"ide.sonde.success": "Event '{event}' triggered successfully!",
"ide.sonde.title": "ClioGraph IDE Event Probe",
"menu.apis": "APIs",
"menu.community": "Community",
"menu.database": "Database",
"menu.dynamic.menu_47d178f74018": "Hauptmenü1_en",
"menu.file": "File",
"menu.file.edit_locales": "✎ Edit Language Files",
"menu.file.exit": "Exit",
"menu.file.export": "Export Project",
"menu.file.language": "🔄 Change Language",
"menu.file.ollama": "🖥️ Ollama Service (AI)",
"menu.file.reset_system": "🔄 Reset System Infrastructure",
"menu.file.validate": "🌐 Language Assistant",
"menu.help": "Help",
"menu.language": "Language",
"menu.quellenverwaltung": "Sources",
"menu.register": "Tabs",
"menu.sources": "Source Management",
"menu.visualization": "Visualization",
"mod1.btn.execute": "Start analysis process",
"mod1.headline": "Data Capture and Text Analysis",
"mod1.placeholder.ref": "Source reference (e.g., signature, folder, ID)...",
"mod1.placeholder.text": "Insert source text here...",
"mod1.title": "Knowledge Extraction",
"mod2.btn.execute": "Calculate network metrics",
"mod2.headline": "Historical Network Analysis (HNA)",
"mod2.placeholder.text": "Calculate metrics (centralities, density,
clusters)...",
"mod2.title": "Network Analytics",
"mod3.btn.execute": "Execute Graph-RAG search",
"mod3.headline": "Interactive LLM Knowledge Graph Training & GraphRAG
Queries",
"mod3.placeholder.text": "Enter Graph-RAG query for historical data...",
"mod3.title": "Graph-RAG Training",
"mod4.btn.execute": "Start GNN training",
"mod4.headline": "Predictive Graph Analysis & Graph Neural Networks (GNN)",
"mod4.placeholder.text": "Configure GNN model (Link Prediction / Node
Classification)...",
"mod4.title": "Graph Machine Learning",
"msg.protection.core_deny": "This element belongs to the protected system core
and cannot be deleted, moved, or renamed!",
"msg.protection.title": "System Protection",
"notification.redo.empty": "No further steps to redo.",
"notification.undo.empty": "No further steps to undo.",
"sidebar.btn.developer_tool": "Developer Tools (VBA & GenAI)",
"sidebar.designer.title": "⚙️ SYSTEM DESIGNER",
"sidebar.item.add": "+ New Item",

```



```

"sidebar.item.assign": "⚡ Assign Command",
"sidebar.item.assign_btn": "Assign Command...",
"sidebar.item.delete": "- Delete Item",
"sidebar.item.group": "📁 MENU ITEMS",
"sidebar.item.move_down": "▼ Move Down",
"sidebar.item.move_up": "▲ Move Up",
"sidebar.item.rename": "✏️ Rename Item",
"sidebar.menu.add": "+ New Main Menu",
"sidebar.menu.assign_command": "Assign Menu Command",
"sidebar.menu.delete": "- Delete Main Menu",
"sidebar.menu.group": "⚙️ MAIN MENU",
"sidebar.menu.move_left": "◀ Move Left",
"sidebar.menu.move_right": "▶ Move Right",
"sidebar.menu.rename": "✏️ Rename Main Menu",
"sidebar.safety.redo": "Redo",
"sidebar.safety.undo": "Undo",
"sidebar.tab.add": "+ New Tab",
"sidebar.tab.delete": "- Delete Tab",
"sidebar.tab.group": "📁 TABS",
"sidebar.tab.move_left": "◀ Move Left",
"sidebar.tab.move_right": "▶ Move Right",
"sidebar.tab.rename": "✏️ Rename Tab",
"sonde.msg.1": "Initializing cleanup...",
"sonde.msg.2": "Deleting configuration files (dynamic_menu.json)...",
"sonde.msg.3": "Archiving additional language files...",
"sonde.msg.4": "Resetting system configurations in RAM...",
"sonde.msg.5": "Re-initializing user interface...",
"sonde.msg.done": "The system has been successfully reset to its original
state!",
"sonde.title.1": "Step 1/5",
"sonde.title.2": "Step 2/5",
"sonde.title.3": "Step 3/5",
"sonde.title.4": "Step 4/5",
"sonde.title.5": "Step 5/5",
"sonde.title.done": "Success",
"tabs.welcome.info_text": "<h2>Welcome to ClioGraph 🏠🌐</h2><p>This
dynamically extensible GUI base framework serves as a highly disciplined,
decoupled workspace (n-tier) for historical analysis in the Digital Humanities.
</p><h3>🔗 Planned Business Logic for the Full Version:</h3><ul><li><b>Source
Management & Text Scanning:</b> Automated structuring and analysis of historical
documents.</li><li><b>Historical Network Analysis:</b> High-performance
relationship calculations via Kùzu-Cypher queries.</li><li><b>LLM Knowledge Graphs
& GraphRAG:</b> Local, privacy-compliant AI model training and deep prompting via
Ollama.</li><li><b>Machine Learning (ML):</b> Innovative, integrated models for
automatic handwritten text recognition (HTR).</li></ul><h3>🔧 Integrated
Developer Tools:</h3><ul><li><b>System- & IDE-Designer (Sidebars):</b> Dynamically
create, move, and bilingually rename main menus, tabs, and form elements at
runtime.</li><li><b>Unbreakable Code-Editor Jump:</b> Right-click any widget to
jump directly to its corresponding macro method in VS Code.</li></ul><br>📁
<b>Documentation Note:</b> The project manual and system documentation (PDF &
Markdown) can be found locally within the project directory at the relative path:
<code style=\"background-color: #f1f3f5; padding: 2px 6px; border-radius: 4px;
font-family: monospace;\">./_Dokumentation/</code><br><br><p style=\"font-size:
11px; color: #666;\">>This code is provided for educational/utility purposes only

```

```
without any warranty. Use at your own risk.<br><br><b>@2026 Copyright ClioGraph  
Schweiz</b></p>",  
    "tabs.welcome.title": "Welcome"  
}
```

File: `json_storage\locales\start_status.json`

```
{  
    "active_languages": [  
        "de",  
        "en"  
    ]  
}
```

File: `tools\convert_to_pdf.py`

```
import sys  
import os  
from fpdf import FPDF  
  
def py_to_pdf(file_path):  
    if not os.path.exists(file_path):  
        return  
  
    # PDF-Objekt erstellen  
    pdf = FPDF()  
    pdf.add_page()  
    pdf.set_font("Courier", size=10) # Feste Schriftart für Code  
  
    # Datei auslesen und in PDF schreiben  
    with open(file_path, 'r', encoding='utf-8') as f:  
        for line in f:  
            pdf.cell(0, 5, txt=line.encode('latin-1', 'replace').decode('latin-  
1'), ln=1)  
  
    # Speicherort festlegen (neben der Originaldatei)  
    pdf_path = os.path.splitext(file_path)[0] + ".pdf"  
    pdf.output(pdf_path)  
  
if __name__ == "__main__":  
    if len(sys.argv) > 1:  
        py_to_pdf(sys.argv[1])
```

File: `tools\json_syntax_fixer.py`



```

# -*- coding: utf-8 -*-
"""
Utility-Tool für ClioGraph: Automatischer JSON-Syntax-Reparierer.
Sucht nach fehlenden Trennungskommas am Zeilenende und korrigiert diese.
"""

import os
import sys

# Hauptverzeichnis ermitteln und im Suchpfad registrieren
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

import infrastructure.cfg as cfg

def repair_json_syntax():
    print("=== CLIGRAPH JSON SYNTAX FIXER ===")
    project_base = getattr(cfg, "PROJECT_ROOT", PROJECT_ROOT)

    # Pfade ermitteln basierend auf deiner Speicherstruktur
    locales_folder = os.path.join(project_base, "json_storage", "locales")
    locales = {
        "de": os.path.join(locales_folder, "de.json"),
        "en": os.path.join(locales_folder, "en.json")
    }

    for lang, file_path in locales.items():
        if not os.path.exists(file_path):
            print(f"❗ Datei nicht gefunden (wird übersprungen): {file_path}")
            continue

        print(f"Analysiere {lang}.json auf Syntaxfehler...")

        with open(file_path, "r", encoding="utf-8") as f:
            lines = f.readlines()

        fixed_lines = []
        corrections_count = 0

        for i, line in enumerate(lines):
            stripped = line.strip()

            # Prüfen, ob es eine normale Key-Value-Zeile ist, die auf ein
            # Anführungszeichen endet
            # aber kein Komma hat und nicht die schließende Klammer oder das Ende
            # der Datei ist
            if (
                stripped.startswith('"')
                and stripped.endswith('"')
                and not stripped.endswith(",")
            ):
                # Sicherheitscheck: Wenn die NÄCHSTE Zeile eine schließende

```

```

Klammer ist, darf KEIN Komma hin
    next_is_end = False
    if i + 1 < len(lines):
        next_stripped = lines[i+1].strip()
        if next_stripped.startswith("{}") or
next_stripped.startswith("]"):
            next_is_end = True

    if not next_is_end:
        # Komma anfügen (unter Beibehaltung der Zeilenumbrüche)
        if line.endswith("\r\n"):
            line = line.replace('\r\n', ',\r\n')
        elif line.endswith("\n"):
            line = line.replace('\n', ',\n')
        else:
            line += ","
        corrections_count += 1
        print(f" → [Zeile {i+1}] Fehlendes Trennungskomma
automatisch repariert.")

    fixed_lines.append(line)

    if corrections_count > 0:
        with open(file_path, "w", encoding="utf-8") as f:
            f.writelines(fixed_lines)
        print(f"✅ {lang}.json erfolgreich mit {corrections_count}
Korrektur(en) gesichert.")
    else:
        print(f" → Keine Syntaxfehler in {lang}.json gefunden.")

if __name__ == "__main__":
    repair_json_syntax()

```

File: `tools\locale_cleaner.py`

```

# -*- coding: utf-8 -*-
import json
import os
import sys

# ONCE-ONLY: Hauptverzeichnis dynamisch ermitteln und im Suchpfad registrieren
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

import infrastructure.cfg as cfg
# NEU: Das dauerhafte, eigenständige Rettungs-Werkzeug importieren
from tools.json_syntax_fixer import repair_json_syntax

def clean_sort_and_generate_pdf():

```

```

"""
Sucht de.json und en.json, reinigt sie von Altlasten und
erzwingt eine 100%ige strukturelle Kongruenz (gleiche Keys) in beiden Dateien.
"""

# ONCE-ONLY: Flickt eventuell vergessene Kommas vor dem Einlesen
vollaufautomatisch zusammen!
repair_json_syntax()

print("\nStarte ClioGraph Lokalisierungs-Utility mit Synchronisations-
Spiegel...")

project_base = getattr(cfg, "PROJECT_ROOT", PROJECT_ROOT)
found_locales = {}

# RADAR-SCAN: Findet deine Dateien sicher in json_storage/locales/
for root, dirs, files in os.walk(project_base):
    if ".venv" in root or ".git" in root or "__pycache__" in root:
        continue
    if "de.json" in files:
        found_locales["de"] = os.path.join(root, "de.json")
    if "en.json" in files:
        found_locales["en"] = os.path.join(root, "en.json")

if "de" not in found_locales or "en" not in found_locales:
    print("❌ Fehler: Es müssen sowohl de.json als auch en.json im Projekt
vorhanden sein!")
    return

# Framework-Core-Defaults für die neuen Dialog-Texte (inkl. Limit-Warnfenster)
defaults_de = {
    "dialog.item.rename.de_prompt": "Neuen DEUTSCHEN Namen eingeben:",
    "dialog.item.rename.en_prompt": "Enter new ENGLISH name:",
    "dialog.item.select_label": "Menüeintrag wählen:",
    "dialog.menu.limit_reached": "Die maximale Anzahl von {0} Hauptmenüs wurde
erreicht!",
    "dialog.menu.select_label": "Hauptmenü wählen:",
    "dialog.tab.select_label": "Registerkarte wählen:"
}

defaults_en = {
    "dialog.item.rename.de_prompt": "Neuen DEUTSCHEN Namen eingeben:",
    "dialog.item.rename.en_prompt": "Enter new ENGLISH name:",
    "dialog.item.select_label": "Select Menu Item:",
    "dialog.menu.limit_reached": "The maximum number of {0} main menus has
been reached!",
    "dialog.menu.select_label": "Select Main Menu:",
    "dialog.tab.select_label": "Select Tab:"
}

try:
    # 1. Beide Dateien einlesen
    with open(found_locales["de"], "r", encoding="utf-8") as f:
        de_data = json.load(f)
    with open(found_locales["en"], "r", encoding="utf-8") as f:

```

```

en_data = json.load(f)

# 2. Altlasten aus beiden Datensätzen entfernen
for data_dict in [de_data, en_data]:
    keys_to_remove = [k for k in data_dict.keys() if
k.startswith("menu.test") or "test_deutsch" in k]
    for k in keys_to_remove:
        del data_dict[k]

# 3. Core-Designer-Defaults injizieren falls abwesend
for k, val in defaults_de.items():
    if k not in de_data: de_data[k] = val
for k, val in defaults_en.items():
    if k not in en_data: en_data[k] = val

# 4. KONGRAUENZ-SCHLEIFE: en.json exakt an de.json angleichen
# Jeder Key aus de.json MUSS auch in en.json existieren
for de_key, de_val in de_data.items():
    if de_key not in en_data:
        # Falls der Key in en.json fehlt, nutzen wir den vordefinierten
Default
        # oder spiegeln als Sicherheitsnetz erst mal den deutschen Inhalt
en_data[de_key] = defaults_en.get(de_key, de_val)

# Umgekehrt: Wenn in en.json etwas ist, das in de.json fehlt, bereinigen
wir es ebenfalls
en_keys_current = list(en_data.keys())
for en_key in en_keys_current:
    if en_key not in de_data:
        del en_data[en_key]

# 5. Beide Datensätze alphabetisch sortieren
de_sorted = {k: de_data[k] for k in sorted(de_data.keys())}
en_sorted = {k: en_data[k] for k in sorted(en_data.keys())}

# 6. Synchronisiert auf die Festplatte schreiben
with open(found_locales["de"], "w", encoding="utf-8") as f:
    json.dump(de_sorted, f, ensure_ascii=False, indent=4)
with open(found_locales["en"], "w", encoding="utf-8") as f:
    json.dump(en_sorted, f, ensure_ascii=False, indent=4)

print(f"🎉 KONGRLUENZ ERREICHT! Beide Dateien haben jetzt exakt
{len(de_sorted)} identische Keys.")
print(f"    -> de.json aktualisiert ({found_locales['de']})")
print(f"    -> en.json aktualisiert ({found_locales['en']})")

# Protokoll-Text für die Review-Datei schreiben
pdf_lines = ["=== CLIOPGRAPH KONGRLUENZ-EXPORT ===", f"Gesamtanzahl
synchronisierter Keys: {len(de_sorted)}", ""]
pdf_lines.append("--- SPRACHDATEI: DE.JSON ---")
for k, v in de_sorted.items(): pdf_lines.append(f'    "{k}": "{v}"')
pdf_lines.append("\n--- SPRACHDATEI: EN.JSON ---")
for k, v in en_sorted.items(): pdf_lines.append(f'    "{k}": "{v}"')

```

```

        tools_dir = os.path.dirname(os.path.abspath(__file__))
        with open(os.path.join(tools_dir, "clioGraph_locales_review.txt"), "w",
encoding="utf-8") as text_file:
            text_file.write("\n".join(pdf_lines))

    except Exception as e:
        print(f"❌ Fehler bei der Spiegelung: {e}")

if __name__ == "__main__":
    clean_sort_and_generate_pdf()

```

File: `tools\system_actions.py`

```

# Datei: tools/system_actions.py
import os
import shutil
from PyQt6.QtWidgets import QMessageBox
import infrastructure.cfg as cfg

class SystemMaintenance:
    """Kapselt alle destruktiven System-Operationen wie Sprach-Cleanups und
Factory Resets."""

    @staticmethod
    def reset_languages_to_factory(main_window):
        """Löscht Dritt-Sprachen mit automatischem Backup, setzt RAM zurück und
synchronisiert."""
        i18n = main_window._i18n

        # 1. Sicherheitsabfrage über das I18N-System (Zweisprachig!)
        antwort = QMessageBox.question(
            main_window,
            i18n.text("dialog.maintenance.title"),
            i18n.text("dialog.maintenance.prompt"),
            QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No
        )

        if antwort != QMessageBox.StandardButton.Yes:
            return

        try:
            # 2. BACKUP-SYSTEM: Ordner für Wiederherstellungsdateien vorbereiten
            backup_dir = os.path.join(cfg.JSON_STORAGE_DIR, "backup_locales")
            if not os.path.exists(backup_dir):
                os.makedirs(backup_dir, exist_ok=True)

            # 3. Physisches Sichern und anschliessendes Löschen im locales-Ordner
            if os.path.exists(cfg.JSON_LOCALES_DIR):
                for filename in os.listdir(cfg.JSON_LOCALES_DIR):
                    if filename.endswith(".json"):

```

```

        lang_code = filename.replace(".json", "")
        if lang_code not in cfg.MASTER_LANGUAGES:
            src_file = os.path.join(cfg.JSON_LOCALES_DIR,
filename)

            dst_file = os.path.join(backup_dir, filename)

            # Kopieren (Sichern) vor dem Löschen
            shutil.copy2(src_file, dst_file)
            # Physisch löschen
            os.remove(src_file)

# 4. RAM-Pools hart auf Ausgangszustand zurücksetzen
cfg.APP_CONFIG["supported_languages"] = ["de", "en"]
cfg.AVAILABLE_LANGUAGES = [
    {"name": "Deutsch", "code": "de"},
    {"name": "English", "code": "en"}
]

# Fallback falls die App auf einer gelöschten Sprache stand
if i18n._current_lang not in cfg.MASTER_LANGUAGES:
    main_window._change_language_trigger("de")

# 5. JUST-IN-TIME PERSISTENZ (Überschreibt dynamic_menu.json
blitzsauber)
persistence = main_window._services.get("menu_persistence")
if persistence and hasattr(persistence, "save_menus"):
    persistence.save_menus(cfg.UI_SCHEMA)

# 6. UI-Neuaufbau erzwingen
if hasattr(main_window, "refresh_ui_texts"):
    main_window.refresh_ui_texts()
elif hasattr(main_window, "setup_ui"):
    main_window.setup_ui()

# Erfolgsmeldung über das I18N-System ausgeben
QMessageBox.information(
    main_window,
    i18n.text("dialog.maintenance.title"),
    i18n.text("dialog.maintenance.success")
)

except Exception as e:
    QMessageBox.critical(main_window, "Fehler", f"Fehler bei der
Systembereinigung:\n{str(e)}")

```

File: `tools\test_ollama.py`

```

import requests
import json

```

```

def test_translation():
    # 1. Lokale URL deines Ollama-Dienstes
    url = "http://localhost:11434/api/generate"

    # 2. Präziser Prompt für die Übersetzung
    prompt = "Translate the German word 'Guten Tag' to Italian. Reply ONLY with the direct translation, nothing else."

    # 3. Payload-Struktur (Wir nutzen llama3 als Basismodell)
    payload = {
        "model": "llama3",
        "prompt": prompt,
        "stream": False # Wichtig: Verhindert, dass die KI wortweise streamt
    }

    print("🕒 Sende Anfrage an den lokalen Ollama-Server... Bitte warten.")

    try:
        # Wir geben dem lokalen PC 15 Sekunden Zeit zum Antworten
        response = requests.post(url, json=payload, timeout=15.0)

        if response.status_code == 200:
            result_json = response.json()
            translation = result_json.get("response", "").strip()
            print("\n✅ VERBINDUNG ERFOLGREICH!")
            print(f"Original (DE): 'Guten Tag'")
            print(f'Ollama (IT):  '{translation}''')
        else:
            print(f"\n❌ FEHLER: Server antwortet mit Status-Code {response.status_code}")
            print(response.text)

    except requests.exceptions.ConnectionError:
        print("\n❌ VERBINDUNG FEHLGESCHLAGEN!")
        print("Der Ollama-Dienst läuft anscheinend nicht im Hintergrund.")
        print("Bitte öffne ein Windows-Terminal (cmd) und starte den Dienst mit: ollama run llama3")
    except Exception as e:
        print(f"\n❌ UNERWARTETER FEHLER: {str(e)}")

if __name__ == "__main__":
    test_translation()

```